QRSathi

QR Code, Barcode and Inventory Management System

A Mini Project Report

submitted in partial fulfilment of the requirements for the award of the degree of **[Degree, e.g. Bachelor of Technology] in [Branch, e.g. Computer Science and Engineering]**

Submitted by

[Student Name]

Roll No. [Roll Number]

Under the guidance of

[Guide Name], [Designation]

[Institution logo]

[Department Name]

[Institution Name]

[University Name]

[City, State]

[Month Year]

A Mini Project Report on

QRSathi

QR Code, Barcode and Inventory Management System

A full-stack web application for generating QR codes and barcodes,
managing products and stock, and administering subscriptions,
built with Next.js, TypeScript and Prisma

by

[Student Name] ([Roll Number])

Guide: **[Guide Name]**

[Department Name]

[Institution Name]

[University Name]

Academic year [Academic Year]

Certificate

This is to certify that the mini project report entitled **"QRSathi — QR Code, Barcode and Inventory Management System"** is a bonafide record of the work carried out by **[Student Name]** (Roll No. **[Roll Number]**) in partial fulfilment of the requirements for the award of the degree of **[Degree and Branch]** of **[University Name]**, during the academic year **[Academic Year]**, under my supervision and guidance.

The work embodied in this report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma.

Project Guide [Guide Name], [Designation]	**Head of the Department** [HoD Name], [Department]
Internal Examiner Name and signature	**External Examiner** Name and signature

Date: [Date of submission] Place: [Place]

Declaration

I, **[Student Name]**, Roll No. **[Roll Number]**, hereby declare that the mini project report entitled **"QRSathi — QR Code, Barcode and Inventory Management System"**, submitted to **[Department Name], [Institution Name]**, affiliated to **[University Name]**, is a record of original work carried out by me under the guidance of **[Guide Name]**.

I further declare that the source code, measurements and test results reported in this document were obtained from the project repository and test runs described in Chapter 5, that results which could not be verified are explicitly marked as such, and that this report has not been submitted elsewhere for the award of any degree or diploma.

Signature of the student	**Date**
[Student Name]	[Date]

Acknowledgement

I express my sincere gratitude to **[Guide Name]**, **[Designation]**, **[Department Name]**, for guidance, encouragement and constructive feedback throughout the development of this project.

I am thankful to **[HoD Name]**, Head of the **[Department Name]**, and to **[Principal / Director Name]**, **[Institution Name]**, for providing the facilities and environment required to complete this work.

I also thank the faculty and laboratory staff of the department, and my family and friends, for their support. Finally, I acknowledge the maintainers of the open-source projects on which QRSathi is built — Next.js, React, Prisma, TypeScript, Tailwind CSS, bwip-js, zxing-wasm and the other libraries listed in Chapter 3 — whose documentation and software made this project possible.

[Student Name]

Abstract

Quick Response (QR) codes and one-dimensional barcodes are now the everyday interface between printed material and digital systems: shops display UPI payment QR codes, products carry EAN-13 barcodes, and marketing material links to websites through QR codes. Small businesses in India, however, typically depend on several unrelated tools for these tasks — a free QR generator, a separate barcode label tool, a spreadsheet for stock, and no means of changing a printed QR code's destination after it has been distributed. This fragmentation leads to duplicated data, invalid barcodes, untraceable stock movements and printed codes that cannot be corrected.

QRSathi is a web application that integrates these functions in one system. Its objectives are to generate standards-compliant QR codes and barcodes, to support editable ("dynamic") QR codes with privacy-respecting scan analytics, to manage products, barcodes and stock as an auditable inventory ledger, and to provide a secure administration console with role-based access control and an optional subscription billing module.

The system is implemented as a full-stack **Next.js 15** application written in **TypeScript 5.9**, using **React 19** for the user interface, **Tailwind CSS 3.4** for styling and **Prisma ORM 6.19** for data access. SQLite is used for development and a generated MySQL schema with versioned migrations is provided for production. The architecture is a layered monolith: App Router pages and 64 API route modules (89 HTTP method handlers) delegate to server-side domain modules (QR service, products, billing, entitlements), which access a relational schema of 22 models through Prisma. QR images are rendered as SVG in the browser; barcodes are rendered with bwip-js; scanning runs entirely in the browser using the zxing-wasm decoder.

The audit carried out for this report verified the following principal modules in source code and automated tests: a QR generator supporting 27 content types (URL, Wi-Fi, vCard, UPI payment, WhatsApp and others) with design customisation and export to PNG, JPG, SVG, PDF and ZIP; dynamic QR codes with short links, expiry, pause, UTM parameters and scan analytics; a barcode generator for 23 symbologies with check-digit validation and printable labels; a camera and image scanner with phishing and UPI-fraud heuristics; product and inventory management with per-user SKU and barcode uniqueness and an append-only stock ledger; bulk generation and spreadsheet import; an administration console with three roles (user, admin, super administrator), TOTP multi-factor authentication, audit logging and branding/pricing configuration; and Razorpay subscription billing with signature verification and idempotent webhooks.

Testing was performed by executing the project's own test suites against an isolated production build and a throw-away database: TypeScript type checking and linting completed without errors; the unit suites passed (134 security assertions, the QR

rendering/decoding suite and 166 barcode assertions); and the API (73), administration/security (99), billing and browser suites were executed with the results reported in Chapter 5. The audit also identified defects and risks, the most significant being that uploads larger than 10 MB fail for paid plans that advertise 25–50 MB limits, and that a development database containing password hashes remains in the Git history.

QRSathi demonstrates that a single, well-structured web application can replace several disconnected tools while applying sound security engineering — server-side sessions, CSRF origin checks, rate limiting, encrypted secrets and audit trails. Its limitations, such as local-disk file storage, the absence of e-mail verification and the lack of a measured performance baseline, are documented together with future enhancements.

Keywords: QR code, barcode, GS1, inventory management, Next.js, TypeScript, Prisma, role-based access control, multi-factor authentication, Razorpay.

Table of Contents

Chapter 1 — Introduction **1**
1.1 Introduction to the Project 1
1.2 Background of QR Code Technology 1
1.3 Background of Barcode Technology 2
1.4 Introduction to Inventory and Product Identification Systems 2
1.5 Problem Statement 2
1.6 Motivation for Developing QRSathi 3
1.7 Project Objectives 3
1.8 Project Scope 3
1.9 Significance of the Project 4
1.10 Intended Users and Stakeholders 4
1.11 Existing System Overview 4
1.12 Proposed System Overview 4
1.13 Advantages of the Proposed System 4
1.14 Project Limitations 5
1.15 Organization of the Report 5
Chapter 2 — System Analysis **6**
2.1 Introduction to System Analysis 6
2.2 Analysis of the Existing System 6
2.3 Problems and Limitations of the Existing Approach 6
2.4 Proposed System 6
2.5 Feasibility Study 7
2.5.1 Technical Feasibility 7
2.5.2 Economic Feasibility 7
2.5.3 Operational Feasibility 7
2.5.4 Schedule Feasibility 7
2.5.5 Security and Maintenance Considerations 7
2.6 Stakeholder Analysis 8
2.7 Functional Requirements Overview 8
2.8 Non-Functional Requirements Overview 8
2.9 User Roles and Permissions 8
2.10 Use Cases and User Interactions 9
2.11 Business Rules 9
2.12 Assumptions and Constraints 10
2.13 Risk Analysis 10
2.14 Comparative Analysis of Existing and Proposed Systems 10
Chapter 3 — System Requirements **12**
3.1 Introduction 12

3.2 Hardware Requirements 12
3.3 Software Requirements 12
3.4 Development Environment 12
3.5 Runtime Environment 13
3.6 Frontend Requirements 13
3.7 Backend Requirements 13
3.8 Database Requirements 13
3.9 Network and Connectivity Requirements 13
3.10 Browser Compatibility 14
3.11 Functional Requirements 14
FR-01 User registration 14
FR-02 User sign-in and sign-out 14
FR-03 Password reset 14
FR-04 Account management 14
FR-05 Static QR code generation 15
FR-06 Save and manage QR codes 15
FR-07 Dynamic QR codes and hosted pages 15
FR-08 Scan analytics 15
FR-09 Barcode generation and labels 15
FR-10 Universal scanner 16
FR-11 Product management 16
FR-12 Stock movements and inventory 16
FR-13 Bulk generation and import 16
FR-14 File uploads for QR content 16
FR-15 Plans and entitlements 17
FR-16 Subscription billing 17
FR-17 Administrator authentication and MFA 17
FR-18 User administration 17
FR-19 Content moderation 17
FR-20 Site configuration 18
FR-21 Audit, logs and system status 18
FR-22 UPI payment QR 18
3.12 Non-Functional Requirements 18
3.13 User Roles and Access-Control Matrix 19
3.14 External Dependencies and Third-Party Services 19
3.15 Configuration and Environment Variables 19
3.16 Requirement Traceability Matrix 20
Chapter 4 — System Design 22
4.1 Introduction to System Design 22
4.2 System Architecture 22
4.3 Architectural Style and Rationale 22
4.4 High-Level System Architecture Diagram 23

4.5 Module Decomposition 25
4.6 Use-Case Diagram 26
4.7 Data-Flow Diagrams 27
4.7.1 Context-Level DFD (Level 0) 27
4.7.2 Level-1 DFD 27
4.7.3 Level-2 DFD: QR Creation and Dynamic Resolution 29
4.8 Flowcharts 29
4.8.1 User Registration and Login 29
4.8.2 Authentication and Authorization of a Request 31
4.8.3 QR Code Generation 33
4.8.4 Barcode Generation and Scanning 35
4.8.5 Product and Inventory Workflow 37
4.8.6 Dynamic QR Resolution 39
4.8.7 Subscription and Payment Workflow 41
4.8.8 Administrator Configuration Workflow 43
4.9 Database Design 44
4.9.1 Conceptual Data Model 44
4.9.2 Entity-Relationship Diagrams 44
4.9.3 Logical Design: Entities and Relationships 50
4.9.4 Physical Design 50
4.9.5 Indexes and Constraints 51
4.9.6 Standalone Models 51
4.9.7 Enumerated and Status Fields 51
4.9.8 Audit-Related Fields 52
4.10 Database Normalization 52
4.11 Class Diagram 52
4.12 Sequence Diagrams 54
4.13 Component Diagram 64
4.14 Deployment Diagram 66
4.15 UI/UX Design 68
4.16 API Design 68
4.17 Security Design 69
4.18 Design Decisions and Trade-offs 70
Chapter 5 — System Testing 72
5.1 Introduction 72
5.2 Testing Objectives 72
5.3 Testing Strategy 72
5.4 Testing Environment 72
5.5 Unit Testing 73
5.6 Integration Testing 73
5.7 System Testing 73
5.8 End-to-End Testing 73

5.9 Database Testing 74
5.10 Authentication and Authorization Testing 74
5.11 Security Testing 74
5.12 Usability and Responsive Testing 74
5.13 Performance Testing 74
5.14 Regression Testing 75
5.15 Compatibility Testing 75
5.16 Test Cases and Test Results 75
5.17 Defect Log 78
5.18 Test Coverage and Limitations 78
5.19 Testing Summary 79
Chapter 6 — Implementation and Evaluation 80
6.1 Introduction 80
6.2 Development Environment Setup 80
6.3 Installation and Configuration 80
6.4 Project Folder Structure 80
6.5 Frontend Implementation 81
6.6 Backend Implementation 81
6.7 Database Implementation 81
6.8 Authentication Implementation 82
6.9 QR Code Module Implementation 82
6.10 Barcode Module Implementation 83
6.11 Product and Inventory Implementation 84
6.12 Administrative Functionality 84
6.13 Dynamic QR Implementation 84
6.14 Subscription and Payment Implementation 85
6.15 Branding and Configuration 85
6.16 Error Handling and Validation 85
6.17 Important Code Walkthroughs 85
6.18 Database Initialization and Migration 86
6.19 Application Execution Procedure 86
6.20 Deployment Procedure 86
6.21 User Interface Screenshots 87
6.22 Evaluation of Implemented Features 97
6.23 Evaluation Against Project Objectives 99
6.24 Known Limitations 99
6.25 Maintenance and Operational Considerations 99
6.26 Implementation Summary 99
Chapter 7 — Conclusion and Future Enhancements 100
7.1 Conclusion 100
7.2 Achievement of Objectives 100
7.3 Major Contributions 100

7.4 Current Limitations 101
7.5 Future Enhancements 101
7.6 Long-Term Scope 102
7.7 Final Conclusion 102
References **103**
Appendix A: Project Directory Structure **105**
Appendix B: API Endpoint Inventory **112**
Appendix C: Database Data Dictionary **116**
Appendix D: Detailed Test Cases **125**
Appendix E: Installation and Execution Guide **132**
Appendix F: Configuration Reference **133**
Appendix G: Screenshot Index **134**
Appendix H: Requirement Traceability Matrix **136**
Appendix I: Glossary **139**

List of Figures

Figure 4.1: High-level system architecture of QRSathi 24
Figure 4.2: Use-case diagram (actors and principal use cases) 26
Figure 4.3: Context-level data-flow diagram (Level 0) 27
Figure 4.4: Level-1 data-flow diagram 28
Figure 4.5: Level-2 DFD: QR code creation and dynamic resolution (processes 2.0 and 3.0) 29
Figure 4.6: Flowchart: user registration and login 30
Figure 4.7(a): Request authorization — CSRF check and workspace guard 31
Figure 4.7(b): Request authorization — administrator guard 32
Figure 4.8(a): QR generation in the browser 33
Figure 4.8(b): Saving a QR code on the server 34
Figure 4.9(a): Barcode generation 35
Figure 4.9(b): Barcode and QR scanning 36
Figure 4.10(a): Product creation 37
Figure 4.10(b): Stock movement 38
Figure 4.11(a): Dynamic QR resolution — status checks 39
Figure 4.11(b): Dynamic QR resolution — scan recording and redirect 40
Figure 4.12(a): Subscription checkout — starting checkout 41
Figure 4.12(b): Subscription checkout — confirmation and webhook 42
Figure 4.13: Flowchart: administrator configuration workflow 43
Figure 4.14: Conceptual data model (entity groups) 44
Figure 4.15: Entity-relationship overview (15 related models; 7 standalone models listed in Table 4.6) 45
Figure 4.16: ERD detail: accounts, sessions, QR codes and analytics 47
Figure 4.17: ERD detail: products, barcodes and inventory ledger 48
Figure 4.18: ERD detail: pricing plans, subscriptions and payments 49
Figure 4.19: Class diagram: domain entities and server-side service modules 53
Figure 4.20: Sequence diagram: workspace login and administrator MFA login 55
Figure 4.21: Sequence diagram: saving a dynamic QR code 57
Figure 4.22: Sequence diagram: database-backed product creation 59
Figure 4.23: Sequence diagram: super administrator saves payment-gateway credentials 61
Figure 4.24: Sequence diagram: subscription checkout, verification and webhook 63
Figure 4.25: Component diagram 65
Figure 4.26: Deployment diagram: verified development set-up and recommended production topology 67
Figure 6.1: Landing page (desktop, 1280 px) 87
Figure 6.2: Sign-in page 88
Figure 6.3: QR code generator with live preview and scan test 88
Figure 6.4: Barcode generator 89
Figure 6.5: Universal scanner 89

Figure 6.6: Workspace overview dashboard (new account) 90
Figure 6.7: Products list (new account) 90
Figure 6.8: New product form 91
Figure 6.9: Inventory 91
Figure 6.10: Bulk generator 92
Figure 6.11: UPI payment QR tool 92
Figure 6.12: Pricing page 93
Figure 6.13: Admin console overview 93
Figure 6.14: Admin user management 94
Figure 6.15: Admin branding settings 94
Figure 6.16: Admin pricing plans editor 95
Figure 6.17: Admin payment-gateway configuration 95
Figure 6.18: Admin security and audit log 96
Figure 6.19: Mobile views (375 px): landing page, generator and scanner 96

List of Tables

Table 2.1: Problems of the existing approach 6
Table 2.2: How the proposed system addresses each problem 6
Table 2.3: Stakeholder analysis 8
Table 2.4: Functional requirements by module 8
Table 2.5: User roles 9
Table 2.6: Use-case descriptions 9
Table 2.7: Risk analysis 10
Table 2.8: Comparison of the existing approach and QRSathi 11
Table 3.1: Hardware requirements 12
Table 3.2: Software requirements (versions verified from `package.json`, `npm ls` and the environment) 12
Table 3.3: Non-functional requirements 18
Table 3.4: Access-control matrix (✓ allowed, ✗ denied, own = only the actor's own records) 19
Table 3.5: External services 19
Table 3.6: Environment variables (from `.env.example` and `src/lib/env-check.ts`) 19
Table 3.7: Requirement traceability matrix (summary; full matrix in Appendix H and the XLSX file) 20
Table 4.1: Architectural layers 22
Table 4.2: Module decomposition 25
Table 4.3: Relationships and cardinalities 50
Table 4.4: Physical model summary (generated from `prisma/schema.prisma`) 50
Table 4.5: Unique constraints and indexes, and their purpose 51
Table 4.6: Models without foreign-key relationships 51
Table 4.7: Enumerated fields 52
Table 4.8: Normalization assessment 52
Table 4.9: Principal pages 68
Table 4.10: HTTP status codes used by the API 69
Table 4.11: Security controls present in the implementation 69
Table 4.12: Security gaps identified by the audit 70
Table 4.13: Design decisions 70
Table 5.1: Test environment 72
Table 5.2: Commands executed and results (final run unless stated) 75
Table 5.3: Test-case summary (full detail in Appendix D) 75
Table 5.4: Defect log 78
Table 6.1: Code walkthrough index 85
Table 6.3: Execution commands 86
Table 6.2: Feature verification matrix 97
Table 7.1: Objectives versus results 100
Table 7.2: Proposed future enhancements 101

Table B.1: API endpoint inventory 112
Table C.1: Data dictionary — User 116
Table C.2: Data dictionary — PasswordReset 116
Table C.3: Data dictionary — QrCode 117
Table C.4: Data dictionary — Scan 117
Table C.5: Data dictionary — Download 118
Table C.6: Data dictionary — Upload 118
Table C.7: Data dictionary — AbuseReport 118
Table C.14: Data dictionary — Log 121
Table C.15: Data dictionary — Setting 121
Table C.16: Data dictionary — Product 121
Table C.17: Data dictionary — Barcode 122
Table C.18: Data dictionary — InventoryTransaction 122
Table C.20: Data dictionary — MfaRecoveryCode 123
Table D.1: Detailed test cases 126
Table F.1: Configuration files 133
Table G.1: Screenshot index 134
Table H.1: Requirement traceability matrix 137
Table I.1: Glossary of terms 139

List of Abbreviations

Abbreviation	Expansion
2FA / MFA	Two-factor / Multi-factor authentication
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode
API	Application Programming Interface
CI	Continuous Integration
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
DFD	Data-Flow Diagram
DNT / GPC	Do Not Track / Global Privacy Control
DPDP	Digital Personal Data Protection (Act, 2023, India)
ECC	Error-Correction Capacity (QR code)
EAN	European Article Number (International Article Number)
ERD	Entity-Relationship Diagram
GS1	Global standards organisation for supply-chain identification
GTIN	Global Trade Item Number
HMAC	Hash-based Message Authentication Code
HSTS	HTTP Strict Transport Security
HTTP(S)	Hypertext Transfer Protocol (Secure)
IDOR	Insecure Direct Object Reference
IFSC	Indian Financial System Code
ISBN / ISSN	International Standard Book / Serial Number
JSON	JavaScript Object Notation
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
OIDC	OpenID Connect
ORM	Object-Relational Mapping
PDF	Portable Document Format
QR	Quick Response (code)
RBAC	Role-Based Access Control
RFC	Request for Comments (IETF standard)
RSC	React Server Components
RTM	Requirement Traceability Matrix
SKU	Stock-Keeping Unit

Abbreviation	Expansion
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SVG	Scalable Vector Graphics
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UI / UX	User Interface / User Experience
UPC	Universal Product Code
UPI	Unified Payments Interface
URL	Uniform Resource Locator
UTM	Urchin Tracking Module (campaign parameters)
VPA	Virtual Payment Address (UPI ID)
WASM	WebAssembly
XSS	Cross-Site Scripting

CHAPTER 1
Introduction

1.1 Introduction to the Project

QRSathi ("Sathi" means "companion" in Hindi) is a web-based platform for creating QR codes and barcodes and for managing the products and stock those codes identify. It combines, in one authenticated workspace, a QR code generator, a dynamic (editable) QR service with scan analytics, a barcode and label generator, a browser-based scanner, a product catalogue with an inventory ledger, bulk generation tools, an administration console and an optional subscription billing module.

The project is implemented as a single Next.js application written in TypeScript. The browser renders QR codes and barcodes as SVG images and decodes codes from the camera using WebAssembly, while the server stores accounts, codes, products and stock movements in a relational database through the Prisma ORM. The application is designed for Indian small businesses — for example, it generates UPI payment QR codes, stores prices in paise and formats numbers in the Indian locale — but nothing in its architecture is specific to one country.

This report documents the project as it exists in the repository at the time of the audit (branch `main`, latest commit `9b5dae7`, plus uncommitted working-tree changes — see Section 1.14) [30]. Every functional claim in this report was checked against the source code, and wherever possible against automated tests that were executed for this report (Chapter 5). The report distinguishes clearly between general background (Sections 1.2–1.4) and what QRSathi actually implements (Sections 1.12 onward).

1.2 Background of QR Code Technology

This section describes QR codes in general; it does not describe QRSathi specifically.

A QR (Quick Response) code is a two-dimensional matrix symbology invented in 1994 by Denso Wave, a Toyota group company, to track automotive parts. It was later standardised internationally as ISO/IEC 18004 [8], [11]. A QR symbol consists of dark and light square modules arranged in a grid; three large *finder patterns* in the corners allow a reader to locate the symbol and determine its orientation from any angle, and *timing* and *alignment* patterns help correct for distortion.

Key properties of QR codes relevant to this project are:

- **Versions.** Symbols range from version 1 (21 × 21 modules) to version 40 (177 × 177 modules). Larger versions hold more data.
- **Error correction.** Reed–Solomon error correction allows a symbol to be decoded even when part of it is damaged or covered. Four levels are defined — L, M, Q and H — recovering roughly 7%, 15%, 25% and 30% of the codewords respectively. Higher levels make it possible to place a logo in the centre of a QR code, at the cost of a denser symbol.
- **Encoding modes.** Numeric, alphanumeric, byte and Kanji modes allow efficient encoding of different kinds of content.
- **Content conventions.** A QR code only stores text. Phones interpret that text according to informal conventions: `https://…` opens a browser, `WIFI:T:WPA;S:…;P:…;;` joins a Wi-Fi network, `BEGIN:VCARD…` adds a contact, and `upi://pay?pa=…` opens a UPI payment app [23].

A distinction that is central to QRSathi is between **static** and **dynamic** QR codes. A static QR code encodes its final content directly; once printed it can never change. A *dynamic* QR code encodes a short URL controlled by a service; the service redirects the visitor to a destination that can be edited, paused or expired later, and can count scans. Dynamic codes are a service convention rather than a different symbology — the printed symbol is an ordinary QR code containing a URL.

1.3 Background of Barcode Technology

This section describes barcodes in general.

Linear (one-dimensional) barcodes encode data in the widths of parallel bars and spaces. The Universal Product Code (UPC) was adopted by the US grocery industry in 1973 and first scanned at a retail checkout in 1974; the European Article Number (EAN, now "International Article Number") followed in 1977. Today the GS1 organisation administers the global numbering system on which retail barcodes depend [9], [10].

Barcode *symbologies* differ in the characters they can encode and their intended use:

- **EAN-13, EAN-8, UPC-A and UPC-E** encode Global Trade Item Numbers (GTINs) for retail products. They are numeric-only and end with a *check digit* calculated with the GS1 modulo-10 algorithm, which lets a scanner reject most misreads.
- **ITF-14** encodes GTINs for cartons and shipping units.
- **GS1-128** encodes GS1 *Application Identifiers* such as (01) GTIN, (10) batch/lot and (17) expiry date.
- **Code 128, Code 39, Code 93 and Codabar** are general-purpose symbologies commonly used for internal labels, asset tags and libraries.
- **Two-dimensional symbologies** such as Data Matrix, PDF417 and Aztec store much more data in a small area and tolerate damage.

An important business rule is that GTINs used on goods sold through retail channels must be allocated by GS1 to the brand owner. Numbers beginning with the prefix 20–29 are reserved for *restricted circulation* (internal use within a company), which makes them safe for a business's own labels.

1.4 Introduction to Inventory and Product Identification Systems

An inventory system records which items an organisation holds and how many units of each are available. Its core concepts are:

- a **product master** (catalogue) describing each stock-keeping unit (SKU): name, code, category, price, supplier and attributes;
- **identifiers** linking a physical item to its record — SKUs, barcodes, QR codes or RFID tags;
- **stock movements** (receipts, issues and adjustments) that change the quantity on hand; and
- **controls** such as re-order (low-stock) levels, batch and expiry tracking, and an audit trail of who changed what and when.

Barcode scanning reduces manual data entry and transcription errors: scanning an item retrieves its record directly, after which a movement can be recorded. A well-designed system treats the stock quantity as the result of recorded movements rather than as a free-form number, so that every change can be explained.

1.5 Problem Statement

Small retailers, shops, creators and organisations increasingly need QR codes (for payments, menus, links and contact details) and barcodes (for products and labels), but commonly face the following problems:

1. **Fragmented tools.** QR generation, barcode/label printing, scanning and stock tracking are performed in separate applications or spreadsheets with no shared data.
2. **Unchangeable printed codes.** A static QR code printed on posters or packaging cannot be corrected when a URL changes; reprinting is costly.
3. **Invalid identifiers.** Barcode numbers created by hand often have wrong check digits, duplicate other products, or use GTIN ranges that should be reserved for GS1-allocated numbers.
4. **Untraceable stock.** Stock quantities kept in spreadsheets are overwritten without a history, so discrepancies cannot be explained.

5. **Scanning risks.** QR-code fraud — fake payment QR codes, phishing links and malicious app downloads — is common, and generic scanner apps rarely warn the user.
6. **Security and privacy.** Many free online tools require no account, offer no access control, and track scanners extensively.

The problem addressed by this project is therefore: *to design and implement a secure, integrated web application through which a small business can generate valid QR codes and barcodes, keep printed QR codes editable, scan codes safely, and manage products and stock with a complete audit trail.*

1.6 Motivation for Developing QRSathi

The motivation came from observing everyday use of codes in Indian retail: UPI QR standees at counters, hand-written price labels next to barcoded goods, and QR codes on menus and posters that lead to outdated pages. Building a single product that handles all of these use cases also provided an opportunity to study, in a realistic codebase, topics central to a computer-science curriculum: relational database design, web application architecture, authentication and authorization, input validation, secure handling of payments and secrets, automated testing and deployment.

1.7 Project Objectives

The objectives of QRSathi, as reflected in its implementation, are:

- **O1.** Generate QR codes for common content types with validation of the content and customisable, scannable designs.
- **O2.** Provide dynamic QR codes whose destination can be edited, paused or expired after printing, with privacy-preserving scan analytics.
- **O3.** Generate barcodes in standard symbologies with correct check digits and printable labels.
- **O4.** Scan QR codes and barcodes from a camera or image in the browser, and warn users about risky content.
- **O5.** Manage products, their barcodes and stock through an auditable inventory ledger with low-stock indication.
- **O6.** Support bulk generation and spreadsheet import for larger catalogues.
- **O7.** Protect accounts and data with secure authentication, role-based access control and a separate, MFA-protected administration console.
- **O8.** Offer configurable plans with server-enforced limits and optional online subscription payments.
- **O9.** Support SQLite for development and MySQL for production from one schema.
- **O10.** Verify behaviour through automated tests.

Section 7.2 evaluates each objective against the audit evidence.

1.8 Project Scope

In scope (implemented): web user interface for desktop and mobile browsers; account registration and sign-in (password and optional Google sign-in); QR generation, saving, editing and export; dynamic QR codes and hosted landing pages; scan analytics; barcode and label generation; in-browser scanning; products, barcodes and stock movements; bulk QR/barcode generation and product import; plan entitlements; administration console; Razorpay subscriptions; SQLite and MySQL database support; automated tests and a CI workflow.

Out of scope: native mobile applications; point-of-sale billing (sales invoices, GST); multi-warehouse inventory; purchase orders and suppliers; hardware integration with dedicated barcode printers or scanners beyond what the browser supports; and GS1 number allocation itself (QRSathi validates GS1 numbers but cannot issue official ones).

1.9 Significance of the Project

The project is significant in three ways. *Practically*, it gives small businesses one tool for tasks that otherwise require several, and it enforces correctness (check digits, unique SKUs, non-negative stock) that ad-hoc tools do not. *Socially*, its scanner warns about common QR-code frauds such as "scan to receive money" UPI scams. *Academically*, it is a complete, tested example of a modern full-stack application that applies security practices — server-side sessions, CSRF defence, rate limiting, encrypted secrets, MFA and audit logging — which are often omitted from student projects.

1.10 Intended Users and Stakeholders

- **Registered users** — shop owners, small businesses, creators and organisations who create codes and manage products.
- **Visitors** — members of the public who scan a QR code produced with QRSathi and are redirected or shown a hosted page; they need no account.
- **Administrators** — platform staff who moderate users and content, view statistics and configure the site.
- **Super administrators** — owners of the platform who additionally manage roles, pricing plans, payment credentials and account deletion.
- **External service providers** — Razorpay (payments), Google (sign-in) and an SMTP provider (e-mail), each optional.
- **Academic stakeholders** — the project guide and examiners evaluating the work.

1.11 Existing System Overview

Before QRSathi, the typical "system" used by the target users is a combination of: free online QR generators (often static-only, sometimes with expiring "dynamic" codes that require a paid plan), separate barcode generator websites, generic phone camera scanners, and spreadsheets for stock. Section 2.2 analyses this approach in detail. No claim is made in this report about the internal behaviour of any specific commercial product.

1.12 Proposed System Overview

QRSathi replaces the fragmented approach with one web application. Its verified modules are:

- **QR Generator and Design Studio** — 27 content types in five categories (essentials, payments, social, business, files & events), with colours, module shapes, logos, frames and export to PNG, JPG, SVG, PDF and ZIP.
- **Dynamic QR and hosted pages** — short links of the form `/r/{code}` and landing pages at `/p/{code}`, with pause, expiry, custom aliases, UTM parameters and scan analytics.
- **Barcode Generator** — 23 symbologies with validation and six label sizes.
- **Universal Scanner** — camera and image scanning in the browser with content classification and safety warnings, and an on-device scan history.
- **Products and Inventory** — product catalogue with multiple barcodes per product, stock in/out/adjust movements, low-stock flags and a per-product activity ledger.
- **Bulk tools** — batch QR and barcode generation and CSV/XLSX product import.
- **Administration console** — a separate sign-in at `/admin` with roles, MFA, user and content moderation, audit log, analytics, branding, pricing, e-mail and payment configuration.
- **Billing** — plans and entitlements stored in the database, with Razorpay subscription checkout and signed webhooks.

1.13 Advantages of the Proposed System

- One account and one data store for codes, products and stock.

- Printed dynamic QR codes remain correctable.
- Barcodes are validated (check digits, restricted-circulation ranges, per-user uniqueness) before being stored.
- Stock changes only through recorded movements, giving a complete history.
- Scanning happens on the user's device; images are not uploaded to a server.
- Scan analytics avoid storing raw IP addresses and honour Do-Not-Track signals.
- Administration is separated from the workspace and protected by MFA.
- The same code runs on SQLite (easy local set-up) and MySQL (production).

These are design advantages derived from the implementation; no comparative user study or benchmark was conducted.

1.14 Project Limitations

Limitations identified in the audit (detailed in Sections 6.24 and 7.4) include: uploaded files are stored on the local disk of a single server; uploads over 10 MB fail even where a plan allows more (defect D-01); e-mail addresses are not verified at sign-up; the in-memory rate limiter is per process unless the database store is enabled; there is no measured performance baseline; real Razorpay, Google and SMTP integrations were not exercised against live services in this audit; and a large part of the most recent work (billing, branding, pricing administration) was uncommitted in Git at the time of the audit.

1.15 Organization of the Report

- **Chapter 2 — System Analysis** analyses the existing approach, feasibility, stakeholders, requirements, roles, use cases, business rules and risks.
- **Chapter 3 — System Requirements** lists hardware and software requirements, detailed functional and non-functional requirements, the access-control matrix, configuration and the requirement traceability matrix.
- **Chapter 4 — System Design** presents the architecture, data-flow diagrams, flowcharts, database design, UML diagrams, UI, API and security design.
- **Chapter 5 — System Testing** reports the tests actually executed, test cases, results and defects.
- **Chapter 6 — Implementation and Evaluation** explains how each module is implemented, with code excerpts, set-up and deployment procedures and screenshots.
- **Chapter 7 — Conclusion and Future Enhancements** evaluates the objectives and proposes future work.
- **References and Appendices** provide sources, the directory structure, API inventory, data dictionary, test cases, installation guide, configuration reference, screenshot index, traceability matrix and glossary.

CHAPTER 2
System Analysis

2.1 Introduction to System Analysis

System analysis is the study of a problem domain and of the existing way of working, carried out to define *what* a new system must do before deciding *how* it will do it. For QRSathi the analysis was performed in two directions. First, the everyday workflow of the target users (Section 1.10) was examined to identify problems and requirements. Second, because the system already exists, the analysis was validated against the implementation: every requirement, role and business rule stated in this chapter corresponds to code that was located in the repository, and the source location is recorded in the traceability matrix (Section 3.16 and Appendix H).

2.2 Analysis of the Existing System

The "existing system" for a typical small shop or creator is not a single product but a set of manual steps and unrelated tools:

1. **QR codes** are made with a free web generator. The resulting image is downloaded and printed. The generator usually keeps no account history, so the same code is recreated when needed again.
2. **UPI payment QR codes** are obtained from a payment app or bank and printed. Creating a standee with a fixed amount or a merchant note requires another tool.
3. **Barcodes** are created on a separate website by typing a number. Whether the number is valid, unique, or allowed for internal use is left to the user.
4. **Labels** are designed in a word processor or image editor.
5. **Stock** is tracked in a spreadsheet or notebook; the quantity cell is overwritten when goods arrive or are sold.
6. **Scanning** uses the phone camera app, which opens links or payment apps without much warning.
7. **Changing a printed QR code** requires creating and reprinting a new one.

2.3 Problems and Limitations of the Existing Approach

Table 2.1: Problems of the existing approach

#	Problem	Consequence
P1	Data is spread across tools with no shared identity	Product, barcode and QR information is duplicated and diverges
P2	Static QR codes cannot be edited	Reprinting cost; broken links in circulation
P3	No validation of barcode numbers	Wrong check digits, duplicates, misuse of GS1 ranges
P4	Stock overwritten without history	Discrepancies cannot be explained or audited
P5	No scan statistics, or invasive tracking	Either no insight into campaigns, or privacy concerns
P6	No access control on shared spreadsheets	Any person with the file can change everything
P7	Scanning without safety warnings	Exposure to UPI "collect" scams and phishing links
P8	Manual, one-at-a-time creation	Large catalogues are slow to label

2.4 Proposed System

The proposed system, QRSathi, addresses these problems as follows:

Table 2.2: How the proposed system addresses each problem

Problem	How QRSathi addresses it (verified in source)
P1	One account owns QR codes, products, barcodes and stock records in a single relational database (`prisma/schema.prisma`).
P2	Dynamic QR codes encode a short link (`/r/{code}`); the destination can be edited, paused or expired (`src/lib/qr-service.ts`, `src/app/r/[code]/route.ts`).

Problem	How QRSathi addresses it (verified in source)
P3	Each symbology validates length, character set and check digit; internal GTINs must use the 20–29 prefix; SKUs and barcodes are unique per user (`src/lib/barcodes.ts`, `src/lib/products.ts`).
P4	Stock changes only through `in`, `out` and `adjust` movements, each recorded in `InventoryTransaction` with a signed quantity and resulting balance.
P5	Scans record device type, browser, OS and country but never raw IP addresses; Do-Not-Track and Global Privacy Control reduce a scan to a count (`src/lib/scans.ts`).
P6	Sessions, roles and ownership checks on every API route (`src/lib/session.ts`).
P7	The scanner classifies content and runs heuristics for phishing URLs and UPI fraud (`src/lib/content-safety.ts`).
P8	Bulk QR/barcode generation and CSV/XLSX product import (`src/app/(app)/bulk`, `src/app/api/products/import`).

2.5 Feasibility Study

2.5.1 Technical Feasibility

The project uses mature, widely documented open-source technologies: Next.js 15.5 and React 19 for the web application, TypeScript 5.9 for static typing, Prisma 6.19 as the ORM, and SQLite/MySQL as databases. Specialised functions are provided by established libraries — `qrcode` for QR matrices, `bwip-js` for barcode encoding, and `zxing-wasm` for decoding. The technical feasibility is demonstrated by the fact that the application builds successfully (`next build` completed in 67 seconds during the audit) and passes its type checks and automated tests (Chapter 5). The only runtime requirement is Node.js 22.5 or later.

2.5.2 Economic Feasibility

All libraries in the dependency tree are open source; the licence report script (`npm run license:check`) ran successfully during the audit. Development requires only a personal computer. A small production deployment can run on a single virtual server with MySQL. Optional services have usage-based costs: Razorpay charges a fee per transaction, and an SMTP provider may charge for e-mail volume. No cost estimate for hosting was measured in this project; the statements above are qualitative.

2.5.3 Operational Feasibility

The user interface follows familiar patterns: a sidebar with grouped navigation (Workspace, Business, More tools, Account), a command palette, light/dark themes and responsive layouts that the browser test suite checks at 375, 768 and 1280 pixel widths. Administrators are created with a command-line tool (`npm run admin`), and operational documents for deployment, backup and recovery exist in `docs/`. No formal usability study with real users was conducted.

2.5.4 Schedule Feasibility

The Git history shows five commits, all dated 2026-10-25, followed by a large set of uncommitted changes. The repository history therefore does not provide a reliable project timeline, and no schedule data is reported. A recommended plan for completing outstanding work is given in Chapter 7.

2.5.5 Security and Maintenance Considerations

Security requirements are significant because the system stores personal data (e-mail addresses), payment records and administrator credentials. The implementation addresses them with server-side sessions, role checks, rate limits, CSRF origin checks, encrypted secrets and an audit log (Section 4.17). Maintainability is supported by TypeScript, a modular `src/lib` layer, ESLint, a CI workflow and Dependabot configuration (`.github/`). Maintenance risks include dependency vulnerabilities that are currently accepted with an expiry date (`security/audit-allowlist.json`), and the need to keep the SQLite and MySQL schemas synchronised — which is automated by `npm run db:mysql:schema -- --check`.

2.6 Stakeholder Analysis

Table 2.3: Stakeholder analysis

Stakeholder	Interest	Influence	Key needs
Registered user (shop owner, creator)	High	Medium	Fast, valid codes; editable QR; simple stock tracking; data privacy
Visitor scanning a code	Medium	Low	Fast redirect; safe destination; no tracking without consent
Administrator	High	High	Moderation tools; visibility; protection against account takeover
Super administrator / owner	High	High	Control of roles, pricing, payments; audit trail
Payment provider (Razorpay)	Medium	Medium	Correct API use; webhook acknowledgement
Project guide / examiners	High	High	Correct, evidenced documentation; working demonstration

2.7 Functional Requirements Overview

The functional requirements are grouped into modules. Detailed specifications with identifiers appear in Section 3.11.

Table 2.4: Functional requirements by module

Module	Summary of functions
Accounts	Sign up, sign in (password, Google), sign out, forgot/reset password, profile and password change, sign out other devices, delete account
QR codes	Create static/dynamic/hosted QR codes for 27 types; design; export; save; list, search, favourite, edit, pause, duplicate, delete
Dynamic QR	Short-link redirect, hosted page, expiry, UTM, abuse reporting
Analytics	Scan counts by day, device, browser, OS, country; CSV export (plan-dependent)
Barcodes	Generate 23 symbologies; validate; labels; export
Scanner	Camera/image decoding; classification; safety notes; local history; product lookup
Products & inventory	Product CRUD with barcodes; archive/restore; stock in/out/adjust; low stock; history; import
Bulk	Batch QR and barcode generation from tables
Billing	View plans; subscribe; verify; cancel; payment history
Administration	Users, roles, suspension, sessions; QR moderation; abuse reports; analytics; audit and security events; settings, branding, appearance, e-mail, pricing, payment configuration; system status; MFA

2.8 Non-Functional Requirements Overview

- **Security:** authenticated access to private data; role checks on administrative functions; protection against CSRF, XSS, brute force and session fixation; secrets never sent to the browser.
- **Privacy:** no raw IP storage in scan analytics or audit logs; honour Do-Not-Track/GPC.
- **Reliability:** atomic quota checks and stock movements; idempotent webhook processing; startup configuration validation that refuses unsafe production configurations.
- **Usability:** responsive layout without horizontal scrolling at common widths; accessible names for controls; light and dark themes.
- **Maintainability:** typed code, shared validation schemas, automated tests and CI.
- **Portability:** SQLite and MySQL from one schema; runs anywhere Node.js 22 runs.
- **Performance:** no measured targets exist in the repository; Section 3.12 lists proposed targets, explicitly marked as unmeasured.

2.9 User Roles and Permissions

The roles are defined in `src/lib/roles.ts` and enforced in `src/lib/session.ts`, `src/lib/guard.ts` and `src/lib/admin-users.ts`.

Table 2.5: User roles

Role	How assigned	Permissions (summary)
Visitor (no account)	—	Open public pages, use the client-side generators and scanner, follow dynamic links, view hosted pages, submit abuse reports
`user`	Every sign-up (forced server-side)	Manage own QR codes, products, stock, uploads, analytics and subscription
`admin`	`npm run admin` CLI or a super administrator	Separate console sign-in; view statistics; suspend/reactivate users, revoke sessions, send reset links, change plans of ordinary users; moderate QR codes and reports; edit settings and branding
`super_admin`	CLI only (or promotion by another super admin)	Everything an admin can do, plus role changes, account deletion, pricing plans, payment credentials and subscription cancellation; must enable MFA before using the console

A workspace session never grants administrative access, even for administrators: the console requires a separate sign-in that creates a distinct `qs_admin` session.

2.10 Use Cases and User Interactions

The principal use cases are shown in Figure 4.2 (Chapter 4). Their textual descriptions are:

Table 2.6: Use-case descriptions

ID	Use case	Primary actor	Description
UC1	Sign up / sign in	User	Create an account or authenticate with e-mail/password or Google
UC2	Reset password	User	Request a one-hour, single-use reset link by e-mail
UC3	Generate static QR	User / visitor	Choose a type, fill the form, style and export; signed-in users can save
UC4	Create dynamic QR / hosted page	User	Create a QR whose destination can change; optionally a hosted landing page
UC5	Generate barcodes and labels	User / visitor	Validate and render a barcode, lay out labels, export
UC6	Scan a code	User / visitor	Decode from camera or image; see classification and warnings
UC7	Manage products	User	Create/edit/archive/delete products with barcodes
UC8	Record stock movement	User	Stock in, stock out or stock-take adjustment
UC9	Bulk generate / import	User	Generate many codes or import products from CSV/XLSX
UC10	View analytics	User	View scan statistics; export CSV where the plan allows
UC11	Subscribe to a paid plan	User	Checkout through Razorpay; plan upgraded after verification
UC12	Open dynamic QR / hosted page	Visitor	Be redirected or shown the hosted page; scan recorded
UC13	Report abusive QR	Visitor	Report phishing, scams, malware etc.
UC14	Moderate	Admin	Manage users, QR codes and abuse reports
UC15	Configure site	Admin	Edit settings and branding
UC16	Manage pricing	Super admin	Create, edit, publish, archive and reorder plans; sync provider plans
UC17	Manage privileged settings	Super admin	Change roles, delete accounts, manage payment credentials
UC18	Enrol MFA	Admin / super admin	Set up TOTP and recovery codes
UC19	Deliver payment events	Razorpay	Send signed webhooks that update subscriptions and payments

2.11 Business Rules

The following rules are enforced by the implementation:

- **BR1** Public sign-up always creates role `user`; any submitted `role` field is ignored (`src/app/api/auth/signup/route.ts`).
- **BR2** Passwords must have at least 8 characters including a letter and a digit; administrator passwords created by the CLI need at least 12 characters (`src/lib/auth-helpers.ts`, `scripts/admin.mts`).
- **BR3** A super administrator must enable MFA before using the console, cannot disable it, and the last super administrator cannot be demoted or deleted.
- **BR4** An SKU is unique per user; a barcode value is unique per user and format; equivalent GTIN forms (e.g. UPC-A and its EAN-13 form) are treated as duplicates through the `lookupKey`.

- **BR5** "Internal" EAN/UPC/ITF-14 numbers must use the restricted-circulation prefix 2 (20–29); numbers outside that range must be marked as GS1 "assigned".
- **BR6** Stock never becomes negative through a stock-out; scanning never changes stock; archived products cannot have stock changes and must be archived before permanent deletion.
- **BR7** Prices are stored as integers in minor units (paise); currencies are limited to INR, USD, EUR, GBP, AED and SGD.
- **BR8** Plan limits (saved QR codes, dynamic codes, hosted pages, products, barcodes, bulk rows, upload size, storage, analytics days and features) are checked on the server inside a transaction.
- **BR9** Dynamic QR codes require an http(s) destination; aliases are 3–40 characters and some words are reserved (`admin`, `api`, `login`, …).
- **BR10** A paid plan is granted only after server-side confirmation — a captured payment fetched from Razorpay or a signature-verified webhook — and duplicate webhook events have no effect.
- **BR11** Raw IP addresses are never stored; audit logs keep a salted hash of the IP and redact secrets.
- **BR12** A paid plan whose `planExpiresAt` has passed falls back to Free on the user's next authenticated request (`load()` in `src/lib/session.ts`).

2.12 Assumptions and Constraints

Assumptions: users access the system through a modern browser with JavaScript enabled; camera scanning requires a device camera and HTTPS (or localhost); production deployments set `NEXT_PUBLIC_APP_URL`, a strong `AUTH_SECRET` and HTTPS; an administrator installs and operates the server.

Constraints: Node.js 22.5 or later (declared in `package.json` `engines`); a single relational database; uploaded files on local disk (`UPLOAD_DIR`); Razorpay is the only payment provider implemented; Google is the only external identity provider; the in-memory rate limiter is per process unless `RATE_LIMIT_STORE=database`.

2.13 Risk Analysis

Table 2.7: Risk analysis

Risk	Likelihood	Impact	Mitigation in the system / recommendation
Account takeover by password guessing	Medium	High	Per-IP, per-account and global rate limits; generic errors; bcrypt cost 12; MFA for administrators
Dynamic QR used for phishing	Medium	High	Abuse reports, admin moderation (pause/disable); recommendation: URL reputation checks
Personal data leaked from repository history	Present	High	Database backup exists in Git history (finding S-01); history rewrite and credential rotation recommended
Loss of uploaded files on server failure	Medium	Medium	Local-disk storage; backup procedure documented; recommendation: object storage
Payment state inconsistency	Low	High	Signature verification, API re-fetch, idempotent ledger
Rate limits ineffective across instances	Medium	Medium	`RATE_LIMIT_STORE=database` option
Dependency vulnerabilities	Medium	Medium	`npm run audit:check` with time-limited allowlist
Schema drift between SQLite and MySQL	Low	Medium	Generated MySQL schema with CI check
Concurrent edits to the codebase without commits	Present	Medium	Work was uncommitted during the audit; commit and tag a release before submission

2.14 Comparative Analysis of Existing and Proposed Systems

The comparison below is between the *manual multi-tool approach* described in Section 2.2 and QRSathi as implemented. It is not a comparison with any specific commercial product, and no claim of superior performance is made because no benchmark was performed.

Table 2.8: Comparison of the existing approach and QRSathi

Criterion	Existing approach	QRSathi (verified)
Integration of QR, barcode, scanning and stock	Separate tools	One application and database
Editable printed QR codes	Not possible with static codes	Dynamic short links with edit/pause/expiry
Barcode validation	Depends on tool; often none	Format rules, check digits, GS1 ranges, uniqueness
Stock history	Usually none	Append-only ledger with balances
Access control	File sharing	Accounts, roles, admin console with MFA
Scan analytics	None, or third-party tracking	Built-in, privacy-preserving
Scam warnings while scanning	Rarely	Heuristic warnings for URLs and UPI
Bulk operations	Manual	Batch generation and import
Cost	Free tools; time-consuming	Open-source stack; hosting and payment fees apply
Offline use	Spreadsheets work offline	Requires network for the server; generation and scanning run client-side once loaded

CHAPTER 3
System Requirements

3.1 Introduction

This chapter specifies the requirements of QRSathi. Requirements that are *verified* (observed in the repository or in the audit environment) are distinguished from *recommended* values (judgements for a comfortable deployment that were not measured). Functional requirements are given identifiers (FR-xx) that are traced to source files and test evidence in Section 3.16, Appendix H and the accompanying spreadsheet `QRSathi_Requirement_Traceability_Matrix.xlsx`.

3.2 Hardware Requirements

Table 3.1: Hardware requirements

Component	Verified in audit environment	Recommended minimum (not measured)
Development machine	Windows 11 Pro (10.0.26200) workstation on which the build and all tests ran	4-core CPU, 8 GB RAM, 5 GB free disk (dependencies ~1 GB, builds, browser)
Production server	Not deployed during the audit	2 vCPU, 2–4 GB RAM for one Node.js instance; SSD storage sized for uploads and database
Database server	MySQL 8.4 in a local Docker container (for the migration test)	Managed or self-hosted MySQL 8 with automated backups
Client device	Desktop Edge/Chrome (automated browser tests)	Any device running a current browser; a camera for live scanning

3.3 Software Requirements

Official documentation for the main technologies is listed in the references: Next.js [1], [2], React [3], TypeScript [4], Prisma [5], Tailwind CSS [6], MySQL [7], SQLite [28], zod [24] and node-qrcode [27].

Table 3.2: Software requirements (versions verified from `package.json`, `npm ls` and the environment)

Software	Version verified	Purpose
Node.js	22.22.3 (engine requirement `>=22.5.0`; `.nvmrc` = 22)	JavaScript runtime for build, server and scripts
npm	10.9.8 (requirement `>=10`)	Package manager
Next.js	15.5.28	Full-stack React framework (App Router, route handlers, middleware)
React / React DOM	19.3.1	User-interface library
TypeScript	5.9.3	Static typing
Prisma CLI / @prisma/client	6.19.3	ORM, schema, migrations
SQLite	via Prisma's bundled engine	Development database (`prisma/dev.db`)
MySQL	8.x (CI uses 8.4; audit test used `mysql:8.4` image)	Production database target
Tailwind CSS	3.4.19 (+ PostCSS 8.5.31, Autoprefixer 10.6.1)	Utility-first styling
ESLint	9.39.5 with `eslint-config-next` 15.5.28	Linting
tsx	4.23.15	Running TypeScript test and admin scripts
Browser	Microsoft Edge / Google Chrome (Chromium)	Runtime for the UI; headless browser tests
Git / GitHub Actions	—	Version control and CI (`.github/workflows/ci.yml`)

3.4 Development Environment

Development uses `npm run dev` (Next.js development server on port 3000) with a SQLite database at `prisma/dev.db`. Before starting, a `predev` script copies the zxing WebAssembly decoder into `public/zxing/`. Visual

Studio Code was the editor in the audited environment. Code quality tools are `npm run typecheck` (TypeScript `--noEmit`), `npm run lint` (ESLint 9 flat config) and the test scripts listed in Chapter 5.

3.5 Runtime Environment

The production runtime is `next build` followed by `next start` on Node.js 22. At start-up, `src/instrumentation.ts` validates the configuration with `checkEnv()` and **refuses to start** a production server with unsafe settings (for example a short `AUTH_SECRET`, a non-HTTPS public URL, `DEMO_MODE=true`, or a secret exposed through a `NEXT_PUBLIC_` variable). The server is stateless apart from the database, uploaded files on disk and (by default) in-memory rate-limit counters.

3.6 Frontend Requirements

- React 19 components rendered by the Next.js App Router; client components for interactive tools (generator, scanner, editors).
- Tailwind CSS with CSS variables for brand colours, light/dark themes (`src/components/providers.tsx`).
- Icons from `lucide-react`; charts with `recharts`; toast notifications with `sonner`.
- Client-side libraries: `qrcode` (QR matrix), `bwip-js` (barcodes), `zxing-wasm` (decoding), `jspdf` and `jszip` (PDF/ZIP export), `jsqr` (verifying exported QR images), `papaparse`, `read-excel-file` and `write-excel-file` (CSV/XLSX).
- Browser APIs: `getUserMedia` for the camera (Permissions-Policy allows the camera only for the same origin), Canvas, Blob downloads, `localStorage` for theme and scan history.

3.7 Backend Requirements

- Next.js route handlers (`src/app/**/route.ts`) and server components; middleware for CSRF origin checks.
- Validation with `zod` 4.6.7 shared between client and server.
- Authentication: `bcryptjs` (cost 12) for passwords, `jose` for HS256-signed session tokens, Node `crypto` for TOTP, AES-256-GCM and HMAC.
- `nodemailer` for SMTP; `fetch` for Razorpay and Google APIs.
- Local file storage under `UPLOAD_DIR`.

3.8 Database Requirements

- A relational database supported by Prisma: SQLite (development) or MySQL 8 (production). The schema has 22 models (Section 4.9).
- MySQL must use a case-sensitive collation for identifier columns where required; the generated schema and migrations handle this (`scripts/db/mysql-schema.mjs`, verified by `npm run test:mysql`).
- Transactions are required for quota checks, stock movements and billing updates.
- Backups: documented in `docs/BACKUP-RECOVERY.md` (not automated by the application).

3.9 Network and Connectivity Requirements

- HTTPS in production (enforced by start-up validation unless the host is local). HSTS is sent automatically when the public URL uses HTTPS.
- Outbound HTTPS to `api.razorpay.com`, `accounts.google.com`, `oauth2.googleapis.com`, `openidconnect.googleapis.com` and the SMTP host — only if those optional integrations are enabled.
- Inbound HTTPS for Razorpay webhooks at `/api/billing/webhook`.
- Behind a reverse proxy, `TRUST_PROXY` must be set so that client IP addresses used for rate limiting are taken from trusted headers.

3.10 Browser Compatibility

The automated browser suite runs on Chromium (Edge or Chrome) using the DevTools protocol. Firefox and Safari were **not tested** in this audit. Camera scanning requires a browser that supports `getUserMedia` and WebAssembly; the scanner also accepts uploaded images when no camera is available.

3.11 Functional Requirements

Each requirement lists the actor, preconditions, normal flow, alternative flows and postconditions. "Implementation" names the main source location.

FR-01 User registration

- **Description:** A visitor creates an account with name, e-mail and password.
- **Actor:** Visitor.
- **Preconditions:** Sign-ups are enabled in site settings.
- **Normal flow:** (1) Visitor submits the sign-up form and accepts the terms. (2) The server rate-limits by IP, validates input, checks that the e-mail is not registered, hashes the password and creates a `user`-role account. (3) A session cookie is set and the user is redirected to the dashboard.
- **Alternative flows:** invalid input → 400 with the first validation message; e-mail exists → 409; sign-ups closed → 403; too many attempts → 429.
- **Postconditions:** `User` and `Session` rows exist; `auth.signup` is audited.
- **Implementation:** `src/app/api/auth/signup/route.ts`.

FR-02 User sign-in and sign-out

- **Description:** A user authenticates with e-mail and password, or Google; and can sign out.
- **Actor:** User.
- **Preconditions:** Account exists and is active.
- **Normal flow:** credentials verified (`verifyCredentials`), new `Session` row and `qs_session` cookie created, `lastLoginAt` updated.
- **Alternative flows:** wrong password, unknown e-mail or password-less account → identical 401 message; suspended → 403; rate limit → 429. Google: state mismatch, unverified e-mail or closed sign-ups → redirect to `/login?error=…`.
- **Postconditions:** session valid for 30 days or until sign-out, password change, suspension or revocation.
- **Implementation:** `src/app/api/auth/login`, `logout`, `google`, `google/callback`; `src/lib/auth-flow.ts`, `src/lib/session.ts`.

FR-03 Password reset

- **Description:** A user requests a reset link and sets a new password.
- **Actor:** User.
- **Normal flow:** e-mail submitted → same response whether or not the account exists → for active accounts a random token is generated, only its SHA-256 hash stored, valid for one hour → user opens link and submits a new password → token claimed atomically, password updated, all sessions revoked, new session created.
- **Alternative flows:** invalid/used/expired token → 400; more than 3 mails per account per hour are silently skipped.
- **Implementation:** `src/app/api/auth/forgot`, `reset`; `src/lib/password-reset.ts`.

FR-04 Account management

- **Description:** Change name or password, sign out other devices, delete account.
- **Actor:** User.

- **Normal flow:** password change requires the current password, increments `sessionVersion`, revokes other sessions and keeps the current one. Deletion requires typing the account e-mail.
- **Alternative flows:** wrong current password → 400; last super administrator cannot delete their own account → 400.
- **Implementation:** `src/app/api/account/route.ts`, `account/sessions/route.ts`.

FR-05 Static QR code generation

- **Description:** Generate a QR code for one of 27 content types and export it.
- **Actor:** Visitor or user.
- **Normal flow:** select type → fill the type-specific form (validated by the type's zod schema) → customise design → preview is rendered in the browser → export PNG, JPG, SVG or PDF (ZIP for batches).
- **Alternative flows:** invalid field → inline error; low contrast or logo too large → design warning.
- **Postconditions:** nothing stored unless the user saves (FR-06).
- **Implementation:** `src/lib/qr-types.ts`, `src/lib/qr-render.ts`, `src/lib/qr-export.ts`, `src/components/qr/*`.

FR-06 Save and manage QR codes

- **Description:** Signed-in users save QR codes and list, search, filter, favourite, edit, pause/resume, duplicate and delete them.
- **Actor:** User.
- **Preconditions:** Signed in; within plan quota.
- **Normal flow:** `POST /api/qrcodes` validates content again on the server, checks quotas in a transaction, stores the code.
- **Alternative flows:** quota exceeded → 403 naming the plan limit; QR disabled by an administrator cannot be re-activated by the owner → 403.
- **Implementation:** `src/lib/qr-service.ts`, `src/app/api/qrcodes/**`.

FR-07 Dynamic QR codes and hosted pages

- **Description:** Create a QR code whose printed short link redirects to an editable destination, or shows a hosted landing page.
- **Actor:** User (creates), visitor (scans).
- **Normal flow:** short code (random 7 characters or a custom alias where the plan allows) is allocated; visitors are redirected by `/r/{code}` with UTM parameters, or served `/p/{code}`.
- **Alternative flows:** destination not http(s) → 400; alias reserved or taken → 400/409; paused, disabled, expired or owner suspended → explanatory 410 page; unknown code → 404 page.
- **Implementation:** `src/lib/qr-service.ts`, `src/app/r/[code]/route.ts`, `src/app/p/[code]/page.tsx`.

FR-08 Scan analytics

- **Description:** Record scans of dynamic codes and display statistics.
- **Actor:** Visitor (generates), user (views).
- **Normal flow:** each non-bot scan, limited to 20 per client per code per 10 minutes, is stored with device, browser, OS and country and a salted daily visitor hash; DNT/GPC or tracking-off produce count-only rows. Users view charts limited to their plan's `analyticsDays`; CSV export requires the `analyticsExport` feature.
- **Implementation:** `src/lib/scans.ts`, `src/lib/analytics.ts`, `src/app/api/analytics/**`.

FR-09 Barcode generation and labels

- **Description:** Generate barcodes in 23 formats and printable labels.
- **Actor:** Visitor or user.

- **Normal flow:** select format → enter value → validation (length, characters, check digit; computed when omitted where applicable) → SVG rendered by bwip-js → optional label layout in one of six sizes → export.
- **Alternative flows:** invalid value → error explaining the rule.
- **Implementation:** `src/lib/barcodes.ts`, `src/components/barcode/*`, `src/lib/barcode-export.ts`.

FR-10 Universal scanner

- **Description:** Decode QR codes and barcodes from the camera or an image.
- **Actor:** Visitor or user.
- **Normal flow:** zxing-wasm decodes frames or an uploaded image (multiple codes per image supported) → content classified (URL, UPI, Wi-Fi, contact, product barcode …) → safety notes shown → entry saved to the on-device history.
- **Alternative flows:** no code found → hints; signed-in users can look up a product by the scanned code (read-only).
- **Implementation:** `src/app/(app)/scanner/scanner.tsx`, `src/lib/zxing-client.ts`, `src/lib/scan-classify.ts`, `src/lib/content-safety.ts`, `src/lib/scan-history.ts`.

FR-11 Product management

- **Description:** Create, view, edit, archive, restore and delete products, each with up to 10 barcodes.
- **Actor:** User.
- **Normal flow:** validate fields → validate and normalise barcodes → check SKU and barcode uniqueness → check product and barcode quotas → create product, barcodes and an initial `created` ledger entry.
- **Alternative flows:** duplicates → 409; invalid barcode or internal GTIN outside 20–29 → 400; delete of an active product → 400 ("archive first").
- **Implementation:** `src/lib/products.ts`, `src/app/api/products/**`.

FR-12 Stock movements and inventory

- **Description:** Record stock in, stock out and stock-take adjustments; show low stock and history.
- **Actor:** User.
- **Normal flow:** movement validated → applied atomically → ledger entry with signed quantity and balance → product flagged low when `stock <= lowStockAt`.
- **Alternative flows:** stock-out larger than stock → 400 with available quantity; quantity 0 for in/out → 400; archived product → 400.
- **Implementation:** `moveStock()` in `src/lib/products.ts`, `src/app/api/products/[id]/stock`, `src/app/api/inventory`.

FR-13 Bulk generation and import

- **Description:** Generate many QR codes or barcodes from a table and import products from CSV/XLSX.
- **Actor:** User.
- **Normal flow:** file parsed in the browser → rows validated → codes generated or `POST /api/products/import` with mode `skip` or `update` → per-row report (created/updated/skipped/error).
- **Alternative flows:** more rows than the plan's `bulkRows` → 403; duplicate SKU in the file or unknown barcode format → row error.
- **Implementation:** `src/app/(app)/bulk/*`, `src/lib/table-file.ts`, `src/app/api/products/import/route.ts`.

FR-14 File uploads for QR content

- **Description:** Upload files (PDF, images, Office documents, text) to be linked from "File" QR codes.
- **Actor:** User.

- **Normal flow:** size checked against the plan limit → extension allow-list → magic-byte check → stored under a random name → served at `/f/{name}` with restrictive headers.
- **Alternative flows:** unsupported type or content mismatch → 415; over limit → 413; storage quota exceeded → 403.
- **Known defect:** uploads above 10 MB fail with 400 even when the plan allows 25 or 50 MB (D-01, Chapter 5).
- **Implementation:** `src/app/api/uploads/route.ts`, `src/lib/storage.ts`, `src/app/f/[name]/route.ts`.

FR-15 Plans and entitlements

- **Description:** Enforce plan limits and features on the server.
- **Actor:** System.
- **Normal flow:** `planForUser()` loads the plan (database-backed catalogue with defaults Free, Pro and Business) → quota checks run inside `withQuota`, which first increments the user's `quotaLock` to serialise concurrent requests.
- **Implementation:** `src/lib/entitlements.ts`, `src/lib/plan-store.ts`, `src/lib/plans.ts`.

FR-16 Subscription billing

- **Description:** Users subscribe to a paid plan through Razorpay, see their billing status and cancel.
- **Actor:** User; Razorpay.
- **Preconditions:** checkout enabled by a super administrator with verified credentials; provider plan synchronised.
- **Normal flow:** see Figure 4.12 and Figure 4.24.
- **Alternative flows:** existing live subscription → 409; signature invalid → 400; amount mismatch → plan unchanged; webhook duplicate → no effect.
- **Implementation:** `src/lib/billing.ts`, `src/lib/razorpay.ts`, `src/app/api/billing/**`.

FR-17 Administrator authentication and MFA

- **Description:** Administrators sign in through `/admin/login`, with TOTP MFA (mandatory for super administrators) and recovery codes.
- **Actor:** Admin, super admin.
- **Normal flow:** password step → if MFA enabled, a 5-minute ticket → TOTP or recovery code → `qs_admin` session (12 h, 2 h idle, SameSite=Strict).
- **Alternative flows:** non-admin → same generic 401; replayed TOTP → 401; super admin without MFA → forced enrolment.
- **Implementation:** `src/app/api/admin/auth/**`, `src/app/api/admin/mfa`, `src/lib/mfa.ts`, `src/lib/admin-auth.ts`.

FR-18 User administration

- **Description:** Search users; suspend/reactivate; revoke sessions; send reset links; change plan; (super admin) change role and delete.
- **Actor:** Admin, super admin.
- **Alternative flows:** admin acting on an administrator, changing roles or deleting → 403; acting on oneself → 400; last super admin → 400.
- **Implementation:** `src/app/api/admin/users/**`, `src/lib/admin-users.ts`.

FR-19 Content moderation

- **Description:** View and pause/disable/delete dynamic QR codes; triage abuse reports submitted by visitors.
- **Actor:** Admin; visitor (report).

- **Implementation:** `src/app/api/admin/qrcodes/**`, `src/app/api/admin/reports/**`, `src/app/api/reports/route.ts`.

FR-20 Site configuration

- **Description:** Edit general settings (announcement, sign-ups), branding (names, colours, logos, favicon, social image; draft → publish), appearance, e-mail test, pricing plans and payment credentials.
- **Actor:** Admin (settings, branding); super admin (pricing, payments).
- **Implementation:** `src/app/api/admin/settings`, `branding`, `branding/assets`, `email`, `plans/**`, `payments/config`; `src/lib/branding.ts`, `src/lib/payment-config.ts`, `src/lib/secret-box.ts`.

FR-21 Audit, logs and system status

- **Description:** Record security events and administrator actions; show logs, security events and configuration status.
- **Actor:** System; admin.
- **Implementation:** `src/lib/audit.ts`, `src/lib/log.ts`, `src/lib/system-status.ts`, `src/app/api/admin/security`, `logs`, `system`; `/api/health`.

FR-22 UPI payment QR

- **Description:** Create UPI payment QR codes (`upi://pay`) for a VPA or bank account + IFSC, with optional amount and note, and printable standees.
- **Actor:** Visitor or user.
- **Alternative flows:** invalid VPA, IFSC or amount → validation error; amounts above ₹1,00,000 produce a warning and above ₹5,00,000 are rejected.
- **Implementation:** `src/lib/upi.ts`, `src/app/(app)/upi/*`, `src/lib/poster.ts`.

3.12 Non-Functional Requirements

Table 3.3: Non-functional requirements

ID	Category	Requirement	Status / evidence
NFR-01	Security	All private API routes require an authenticated session; admin routes require an admin-console session	Verified (test:api, test:admin)
NFR-02	Security	State-changing API requests from other origins are rejected	Verified ("cross-site POST blocked (403)")
NFR-03	Security	Passwords hashed with bcrypt (cost 12); reset tokens and recovery codes stored as SHA-256 hashes; MFA and payment secrets AES-256-GCM encrypted	Verified in source and tests
NFR-04	Security	Security headers (CSP, nosniff, frame options, referrer policy, permissions policy, COOP, HSTS on HTTPS)	Verified (header test); sandbox CSP on `/f` and `/brand` defective in the first audited build (D-02)
NFR-05	Privacy	No raw IP addresses stored; DNT/GPC honoured	Verified in source; scan cap verified by test
NFR-06	Reliability	Quotas, stock-outs, token claims and webhook effects are atomic or idempotent	Verified in source; duplicate-webhook test passed
NFR-07	Reliability	Production server refuses unsafe configuration	Verified (unit tests of `checkEnv`)
NFR-08	Usability	Pages fit 375/768/1280 px without horizontal scrolling	Verified for public and workspace pages by test:browser
NFR-09	Accessibility	Controls, links and images have accessible names	Verified automatically at 1280 px for tested pages; no manual WCAG audit
NFR-10	Maintainability	Type-checked, linted code; CI workflow	Verified (0 type errors, 0 lint errors, 5 warnings)
NFR-11	Portability	SQLite and MySQL from one schema	Verified (test:mysql 32/32 on MySQL 8.4)

ID	Category	Requirement	Status / evidence
NFR-12	Performance	**Target (not measured):** dynamic redirect < 200 ms server time; dashboard pages interactive < 3 s on 4G	Not measured
NFR-13	Scalability	Multiple instances supported with MySQL and `RATE_LIMIT_STORE=database`	Implemented; not load-tested

3.13 User Roles and Access-Control Matrix

Table 3.4: Access-control matrix (✓ allowed, ✗ denied, own = only the actor's own records)

Function	Visitor	User	Admin	Super admin
Public pages, client-side generators, scanner	✓	✓	✓	✓
Follow dynamic link / hosted page; report abuse	✓	✓	✓	✓
Save/edit QR codes, products, stock, uploads, analytics	✗	own	own (workspace session)	own (workspace session)
Subscribe / cancel subscription	✗	own	own	own
Admin console (requires separate admin sign-in)	✗	✗	✓	✓ (MFA required)
Suspend/reactivate, revoke sessions, reset link, change plan	✗	✗	ordinary users only	✓ (not self for status)
Change roles, delete accounts	✗	✗	✗	✓ (not self; not last super admin)
Moderate QR codes and reports	✗	✗	✓	✓
Settings, branding, appearance, e-mail test	✗	✗	✓	✓
Pricing plans (create/edit/reorder/sync)	✗	✗	view	✓
Payment credentials, enable checkout	✗	✗	view status	✓ (recent re-authentication)
Cancel a user's subscription	✗	✗	✗	✓

3.14 External Dependencies and Third-Party Services

Table 3.5: External services

Service	Use	Required?	Integration point
Razorpay	Subscription plans, checkout, payment fetch, cancellation, webhooks	Optional	`src/lib/razorpay.ts`, `src/lib/billing.ts`
Google OAuth 2.0 / OpenID Connect	"Continue with Google" sign-in	Optional	`src/app/api/auth/google/**`
SMTP server	Password-reset e-mails; admin test mail	Optional (required in practice for resets)	`src/lib/mail.ts`
GitHub Actions	CI	Development only	`.github/workflows/ci.yml`
Hosting platform headers (`x-vercel-ip-country`, `cf-ipcountry`)	Country of scans, if present	Optional	`src/lib/scans.ts`

3.15 Configuration and Environment Variables

Only variable **names and purposes** are listed; no values from any environment file were copied into this report.

Table 3.6: Environment variables (from `.env.example` and `src/lib/env-check.ts`)

Variable	Purpose	Required
`DATABASE_URL`	Prisma connection string (`file:` for SQLite, `mysql://` for MySQL)	Yes
`AUTH_SECRET`	Signs session tokens; ≥ 32 random characters	Yes
`NEXT_PUBLIC_APP_URL`	Public origin used for short links, reset links, OAuth callbacks and CSRF origin checks	Yes in production
`SETTINGS_ENCRYPTION_KEY`	Key for encrypting payment credentials stored in the database (≥ 32 chars, ≠ `AUTH_SECRET`)	Only to store credentials in DB
`MFA_ENCRYPTION_KEY`	Key for encrypting TOTP secrets (falls back to a key derived from `AUTH_SECRET`)	Recommended
`TRUST_PROXY`	`true` or `cloudflare` when behind a trusted proxy	Deployment-specific

Variable	Purpose	Required
RATE_LIMIT_STORE	memory (default) or database	Recommended database for multiple instances
AUDIT_IP_SALT	Salt for hashed IPs in the audit log	Optional
LOG_LEVEL, LOG_FORMAT	Logging verbosity and format	Optional
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Google sign-in	Optional (both or neither)
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, MAIL_FROM	Outgoing e-mail	Optional
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	Razorpay credentials via environment (alternative to encrypted DB storage)	Optional; webhook secret mandatory if keys set
RAZORPAY_API_BASE	Override of the Razorpay API address for automated tests only	Never in production
UPLOAD_DIR	Directory for uploaded files	Optional (default ./uploads)
DEMO_MODE	Enables demo data in development; rejected in production	Development only
NEXT_DIST_DIR	Alternative build directory (used for isolated test builds)	Optional
ADMIN_EMAILS	Obsolete; grants nothing and triggers a warning	Should be removed

The local `.env` file in the audited workspace still defines `ADMIN_EMAILS`; the application warns that this variable is obsolete (its value was not inspected).

3.16 Requirement Traceability Matrix

Table 3.7: Requirement traceability matrix (summary; full matrix in Appendix H and the XLSX file)

Req.	Module	Main source	Test evidence	Status
FR-01	Accounts	api/auth/signup/route.ts	TC-AUTH-01..04 (test:admin, test:api)	Verified
FR-02	Accounts	lib/auth-flow.ts, lib/session.ts	TC-AUTH-05..10	Verified (Google not exercised)
FR-03	Accounts	api/auth/forgot, reset	TC-AUTH-11..14	Verified (SMTP delivery not exercised)
FR-04	Accounts	api/account/**	TC-AUTH-15..17	Verified
FR-05	QR	lib/qr-types.ts, lib/qr-render.ts	TC-QR-01..03 (test:qr)	Verified
FR-06	QR	lib/qr-service.ts	TC-QR-04..06	Verified
FR-07	Dynamic QR	app/r/[code]/route.ts	TC-DYN-01..03	Verified (redirect by suite; pause/expiry/unknown by audit probe)
FR-08	Analytics	lib/scans.ts	TC-DYN-04	Partially verified
FR-09	Barcodes	lib/barcodes.ts	TC-BC-01..04 (test:barcodes)	Verified
FR-10	Scanner	scanner/scanner.tsx	TC-SCN-01..03 (test:browser)	Verified (Chromium)
FR-11	Products	lib/products.ts	TC-PRD-01..09	Verified
FR-12	Inventory	moveStock()	TC-INV-01..05	Verified
FR-13	Bulk/import	api/products/import	TC-PRD-10..11	Verified (import); bulk UI not individually asserted
FR-14	Uploads	api/uploads, lib/storage.ts	TC-UPL-01..03	Partially verified; defect D-01
FR-15	Entitlements	lib/entitlements.ts	TC-BIL-05 (test:billing)	Verified
FR-16	Billing	lib/billing.ts	TC-BIL-01..04 (mock Razorpay)	Verified against mock only

Req.	Module	Main source	Test evidence	Status
FR-17	Admin auth	`api/admin/auth/**`, `lib/mfa.ts`	TC-ADM-01..08	Verified
FR-18	Admin users	`api/admin/users/**`	TC-ADM-09..14	Verified
FR-19	Moderation	`api/admin/qrcodes`, `reports`	—	Implemented, not independently verified
FR-20	Configuration	`api/admin/settings`, `branding`, `plans`, `payments/config`	TC-ADM-15..17, TC-BIL-06	Verified
FR-21	Audit/logs	`lib/audit.ts`	TC-ADM-18	Verified
FR-22	UPI QR	`lib/upi.ts`	TC-QR-07..08	Verified

CHAPTER 4
System Design

4.1 Introduction to System Design

System design translates the requirements of Chapter 3 into a structure of components, data and interactions. This chapter presents QRSathi's design at several levels: the overall architecture (Sections 4.2–4.5), behavioural models — use cases, data-flow diagrams, flowcharts and sequence diagrams (Sections 4.6–4.8 and 4.12), the database design (Sections 4.9–4.10), structural UML views (Sections 4.11, 4.13, 4.14), and the user-interface, API and security design (Sections 4.15–4.17). All diagrams were drawn from the audited source code; their editable Mermaid [29] sources are in `docs/mini-project-report/diagrams/`, and PlantUML equivalents are provided for the use-case, deployment and login-sequence diagrams.

4.2 System Architecture

QRSathi is a **layered monolithic web application** built on Next.js. One deployable unit contains the user interface, the HTTP API and the business logic; data is held in one relational database and one directory of uploaded files.

Table 4.1: Architectural layers

Layer	Responsibility	Main locations
Presentation (client)	Interactive tools: QR workbench, barcode generator, scanner, product editor, admin views; SVG rendering; export; camera decoding	`src/components/**`, client components in `src/app/**`
Presentation (server)	Server-rendered pages and layouts, page guards, metadata	`src/app/**/page.tsx`, `layout.tsx`, `src/lib/guard.ts`
Edge of the API	CSRF origin check for state-changing API calls; request path header for admin redirects	`src/middleware.ts`
API	Route handlers: parse and validate input, authenticate, call domain services, map errors to HTTP	`src/app/api/**/route.ts`, `src/lib/api.ts`
Domain services	QR, products, barcodes, entitlements, billing, scans, branding, settings	`src/lib/*.ts`
Cross-cutting	Sessions/RBAC, rate limiting, audit, logging, redaction, encryption, configuration checks	`src/lib/session.ts`, `rate-limit.ts`, `audit.ts`, `logger.ts`, `redact.ts`, `mfa.ts`, `secret-box.ts`, `env-check.ts`
Data access	Prisma Client singleton	`src/lib/db.ts`
Persistence	SQLite or MySQL; local file storage	`prisma/`, `src/lib/storage.ts`

An important characteristic is that several modules are **isomorphic**: the QR type registry (`qr-types.ts`), the QR renderer (`qr-render.ts`), the barcode module (`barcodes.ts`), the plan schema (`plans.ts`) and branding helpers run in both the browser and the server. The same zod schema that shows an error message in the form is used again by the API to reject invalid data, so the client cannot bypass validation.

4.3 Architectural Style and Rationale

The architecture combines three styles, as supported by the Next.js App Router [1]:

1. **Client–server with server-side rendering.** Next.js renders pages on the server (React Server Components) and hydrates interactive parts in the browser.
2. **Layered architecture.** Route handlers depend on services, services depend on the data-access layer; there are no calls in the opposite direction. This keeps HTTP concerns (status codes, cookies) out of the domain logic and makes the logic reusable from scripts (the admin CLI and the tests import `src/lib` modules directly).
3. **Client-heavy processing for media.** Rendering QR/barcode images and decoding camera frames happen in the browser. This removes server load for the most frequent operations and keeps scanned images on the user's device.

The main alternative considered implicitly by this design — a separate single-page application and REST back-end, or microservices — would add deployment complexity without benefit at the project's scale. The trade-offs are discussed in Section 4.18.

4.4 High-Level System Architecture Diagram

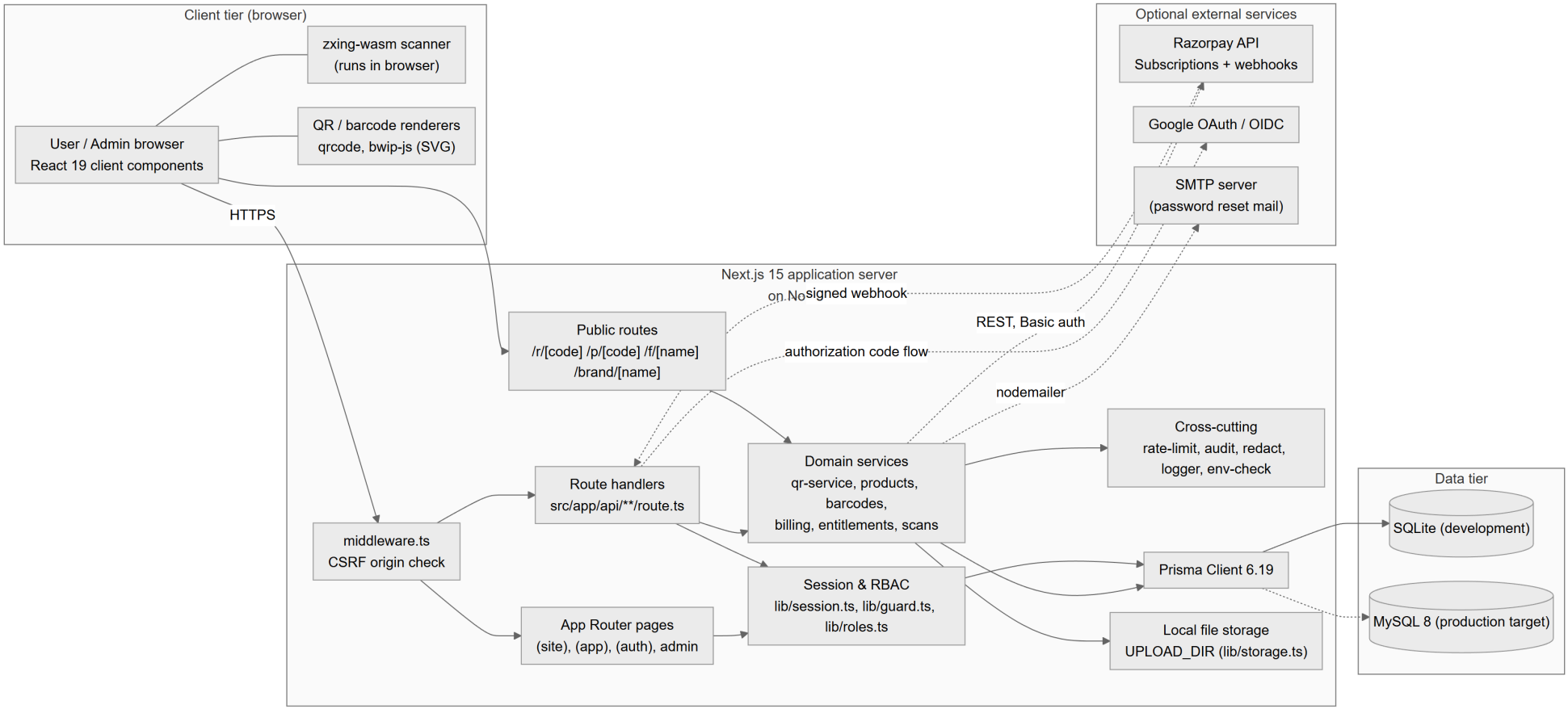


Figure 4.1: High-level system architecture of QRSathi

Figure 4.1 shows the verified components. The browser runs React client components, the zxing-wasm scanner and the QR/barcode renderers. All requests reach the Next.js server; for `/api` routes, the middleware first enforces the CSRF origin rule. App Router pages and API route handlers authenticate through the session/RBAC module and call domain services, which use cross-cutting services (rate limiting, audit, redaction, logging) and access the database through Prisma. SQLite is used in development; MySQL is the production target (dashed line, because no production deployment was verified). Public routes (`/r`, `/p`, `/f`, `/brand`) serve redirects, hosted pages, uploaded files and brand images. External services are optional: Razorpay (REST calls and signed webhooks), Google OAuth and SMTP.

4.5 Module Decomposition

Table 4.2: Module decomposition

Module	Responsibilities	Inputs	Outputs	Depends on
Authentication & sessions	Sign-up, sign-in, Google OAuth, reset, sessions, logout	Credentials, cookies	Session cookie, user DTO	Prisma, bcrypt, jose, rate limit, audit
Authorization (RBAC)	Roles, page and API guards, admin permission rules, recent-auth	Session, role	Allow / 401 / 403 / redirect	Sessions
Admin auth & MFA	Admin login, TOTP, recovery codes, re-authentication	Password, TOTP, ticket	`qs_admin` session	Node crypto, Prisma
QR types & rendering	27 type definitions, payload builders, SVG renderer, design warnings	Form data, design	Payload text, SVG	qrcode, zod
QR service	Persist, update, duplicate QR codes; short codes; UTM	Validated input	`QrCode` rows	Entitlements, Prisma
Dynamic resolution & scans	`/r/[code]`, `/p/[code]`, scan recording	Short code, request headers	302 / page / 410; `Scan` rows	Prisma, rate limit, UA parser
Analytics	Aggregate scans; CSV export	Date range	Chart data, CSV	Prisma, entitlements
Barcodes	23 formats: validation, check digits, GS1 parsing, rendering, labels, lookup keys	Format, value	SVG, normalised value	bwip-js
Scanner	Camera/image decoding, classification, safety notes, history	Video frames, images	Decoded results	zxing-wasm, content safety
Products & inventory	Product CRUD, barcodes, stock movements, lookup, import	Product data, movements	Products, ledger entries	Barcodes, entitlements, Prisma
Bulk	Batch generation, CSV/XLSX parsing and export	Spreadsheet rows	ZIP/PDF of codes, import report	papaparse, read/write-excel-file, jszip, jspdf
Uploads	Validate and store files; serve safely	Multipart file	`Upload` row, `/f/{name}`	Storage, entitlements
Plans & entitlements	Plan catalogue (DB with defaults), quota checks, feature flags	User plan	Allow / 403	Prisma
Billing	Provider plan sync, subscriptions, checkout verification, webhooks, refunds, cancellation	Plan id, Razorpay payloads	Subscription/Payment rows, plan changes	Razorpay client, payment config
Configuration & branding	Settings, branding draft/publish, assets, theme CSS	Admin input, images	`Setting`/`BrandAsset` rows	image-check, Prisma
Payment configuration	Store sealed credentials, verify, enable checkout	Keys, mode	Status (never secrets)	secret-box, Razorpay
Audit & logging	Audit events, system log, structured logs	Events	`AuditLog`, `Log`, stdout	redact
Admin console	Dashboards for users, codes, reports, products, analytics, subscriptions, payments, security, settings, system	Admin requests	Views	All of the above

4.6 Use-Case Diagram

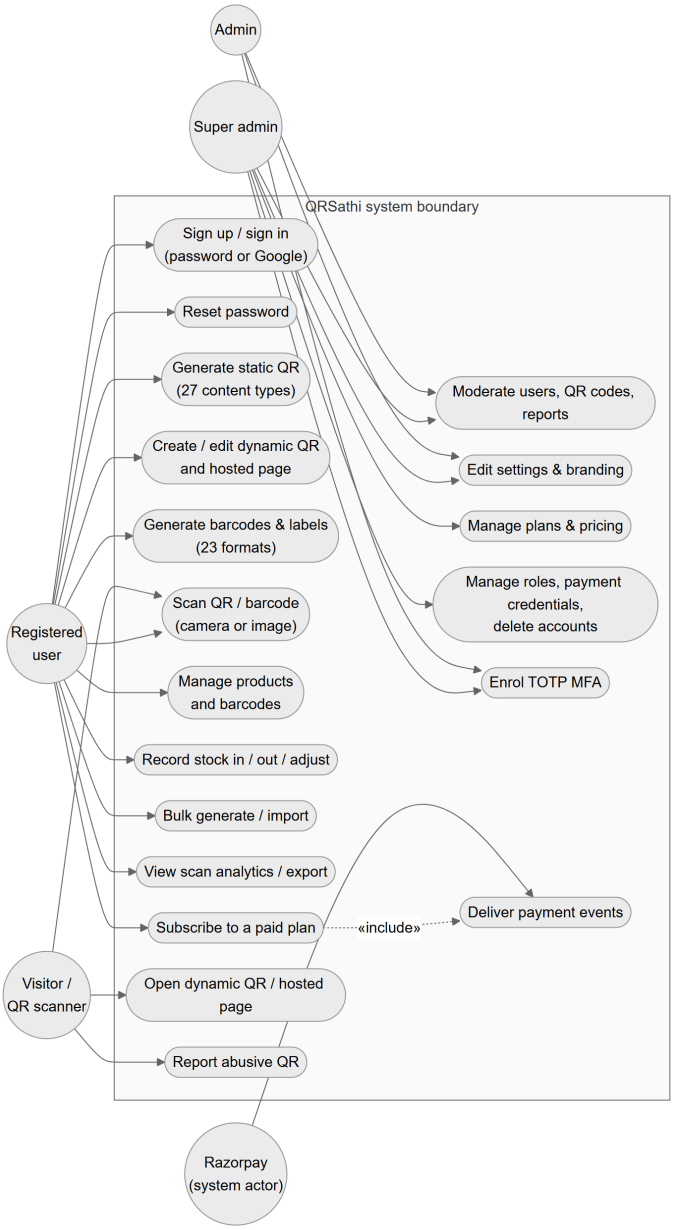


Figure 4.2: Use-case diagram (actors and principal use cases)

Figure 4.2 shows five actors. The *Visitor* (no account) opens dynamic links and hosted pages, reports abusive codes and can use the scanner. The *Registered user* performs the workspace use cases UC1–UC11. The *Admin* moderates and configures (UC14, UC15, UC18); the *Super admin* additionally manages pricing and privileged settings (UC16, UC17) — in UML terms the super admin specialises the admin (shown explicitly in the PlantUML source `use-case-diagram.puml`). *Razorpay* is a system actor that delivers payment events (UC19), which the subscription use case includes. The diagram is drawn with Mermaid flowchart shapes: circles are actors and rounded boxes are use cases inside the system boundary.

4.7 Data-Flow Diagrams

4.7.1 Context-Level DFD (Level 0)

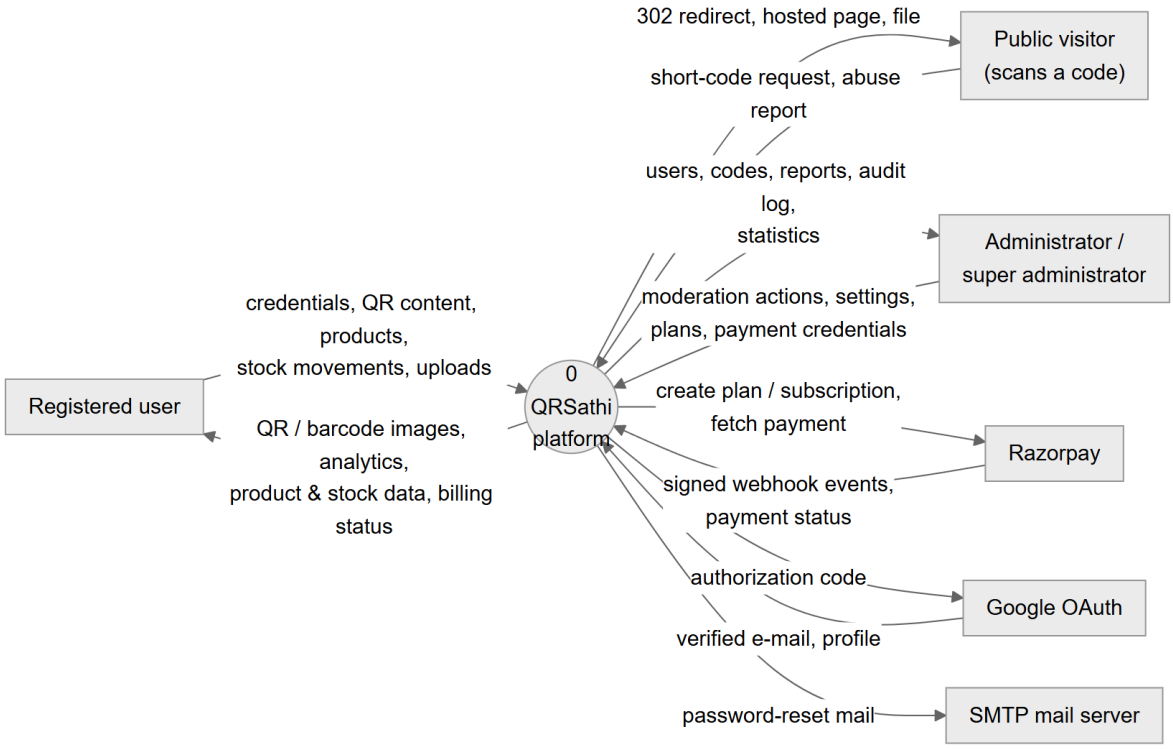


Figure 4.3: Context-level data-flow diagram (Level 0)

At the context level (Figure 4.3) QRSathi is a single process exchanging data with six external entities. Users send credentials, QR content, products, stock movements and uploads, and receive images, analytics and billing status. Visitors send short-code requests and abuse reports and receive redirects, hosted pages and files. Administrators send moderation and configuration actions and receive reports and audit data. Razorpay, Google and the SMTP server exchange the data shown on the arrows.

4.7.2 Level-1 DFD

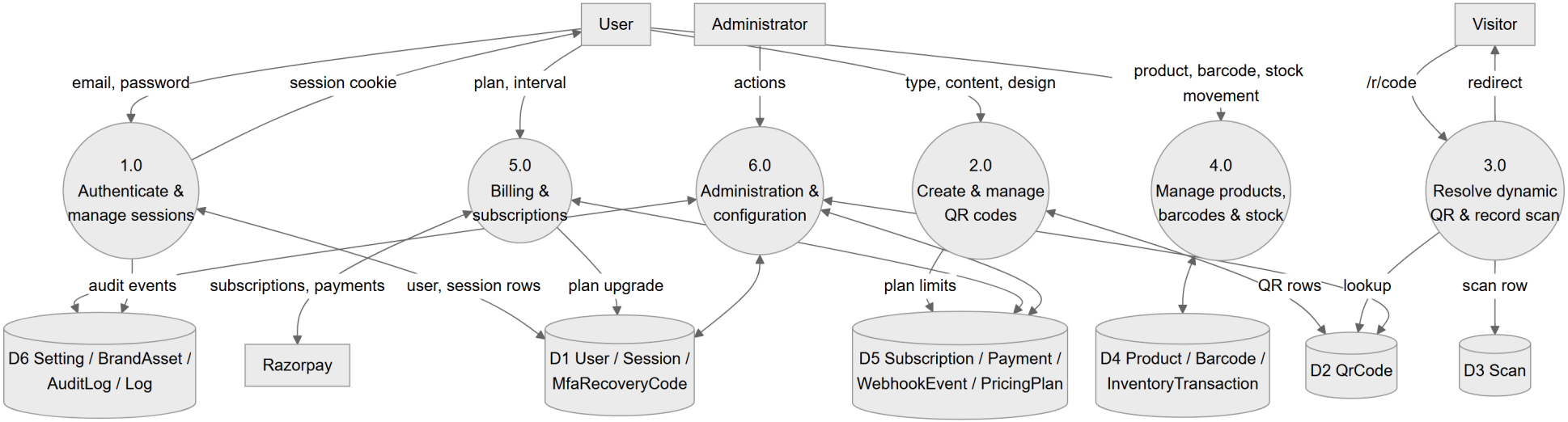


Figure 4.4: Level-1 data-flow diagram

Figure 4.4 decomposes the system into six processes and six data stores. Process 1.0 authenticates users against D1 (users, sessions, recovery codes) and writes audit events to D6. Process 2.0 creates QR codes in D2 after reading plan limits from D5. Process 3.0 serves visitors: it reads D2 and appends to D3 (scans). Process 4.0 maintains D4 (products, barcodes and the inventory ledger). Process 5.0 exchanges subscription and payment data with Razorpay and D5, and upgrades the user's plan in D1. Process 6.0, administration, reads and writes most stores.

4.7.3 Level-2 DFD: QR Creation and Dynamic Resolution

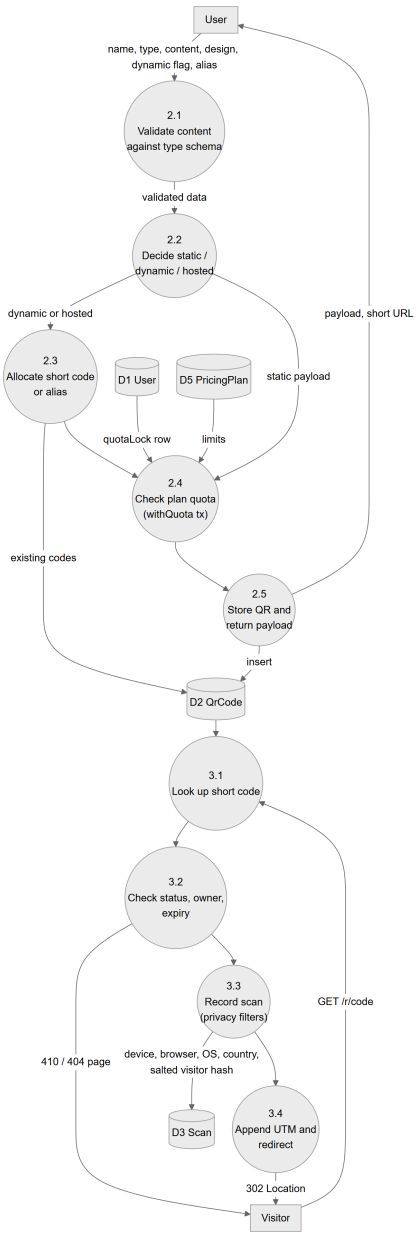


Figure 4.5: Level-2 DFD: QR code creation and dynamic resolution (processes 2.0 and 3.0)

Figure 4.5 refines processes 2.0 and 3.0. Creation (2.1–2.5) validates the content against the type schema, decides whether the code is static, dynamic or hosted, allocates a short code or alias for dynamic codes, checks quotas inside a transaction that locks the user's `quotaLock` row, and stores the code. Resolution (3.1–3.4) looks up the short code, checks status, owner and expiry, records the scan with privacy filters, appends UTM parameters and redirects.

4.8 Flowcharts

4.8.1 User Registration and Login

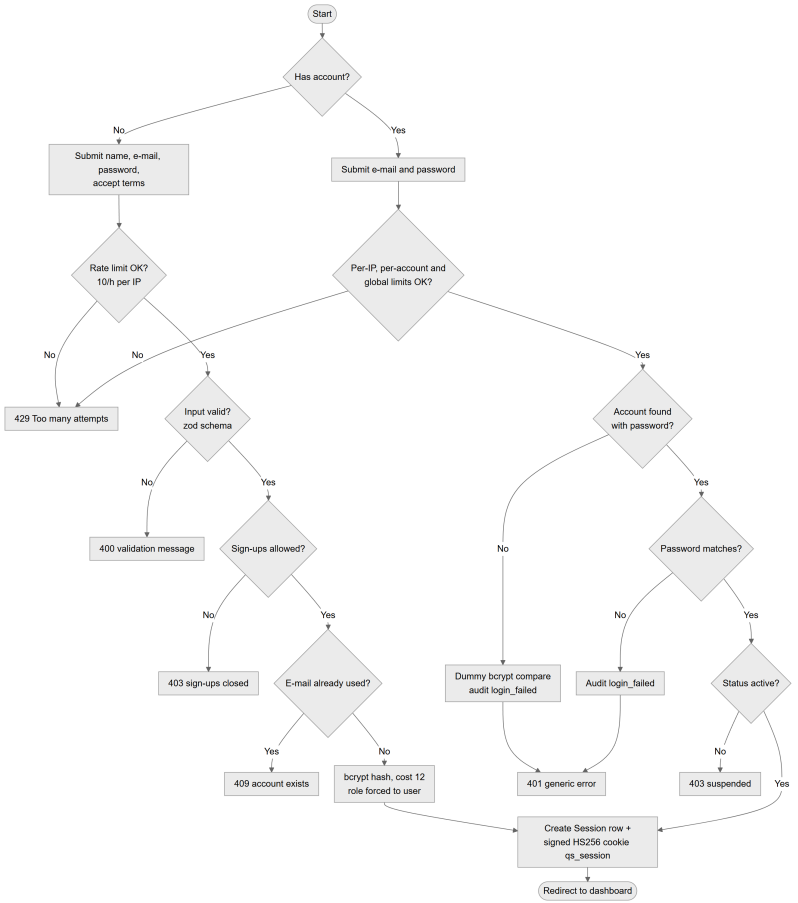


Figure 4.6: Flowchart: user registration and login

Figure 4.6 shows both paths. Registration is rate-limited, validated, checks whether sign-ups are open and whether the e-mail exists, then hashes the password and forces the role to `user`. Login applies three rate limits, performs a dummy bcrypt comparison for unknown accounts (so response time does not reveal whether an account exists), returns one generic error for all credential failures, and blocks suspended accounts. Both paths end by creating a server-side session.

4.8.2 Authentication and Authorization of a Request

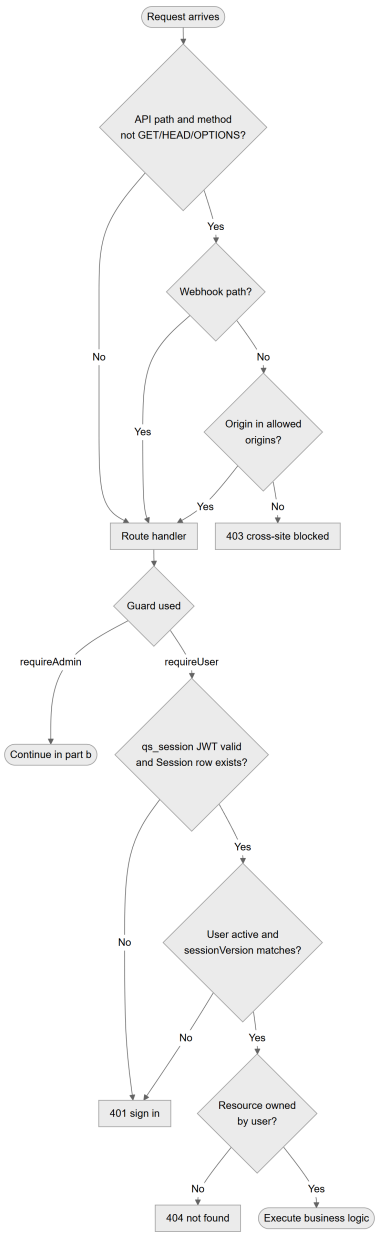


Figure 4.7(a): Request authorization — CSRF check and workspace guard

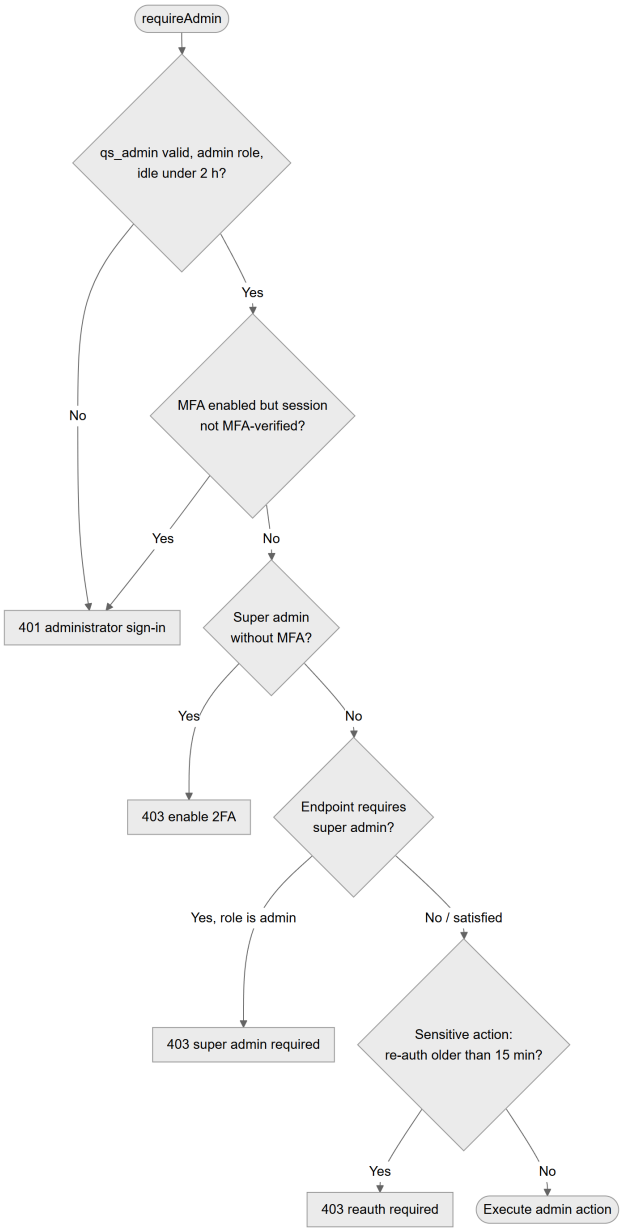


Figure 4.7(b): Request authorization — administrator guard

Every request passes the checks in Figure 4.7 (parts a and b). The middleware rejects cross-origin state-changing API calls (except the webhook, which is authenticated by signature). Workspace handlers call `requireUser()`, which verifies the signed cookie, the session row, the account status and the `sessionVersion`; resource ownership is then checked, and resources of other users produce 404 (not 403) to avoid revealing their existence. Admin handlers call `requireAdmin()`, which additionally enforces the admin role, a two-hour idle timeout, MFA verification of the session, mandatory MFA for super administrators, super-admin-only endpoints and — for payment credentials — re-authentication within 15 minutes.

4.8.3 QR Code Generation

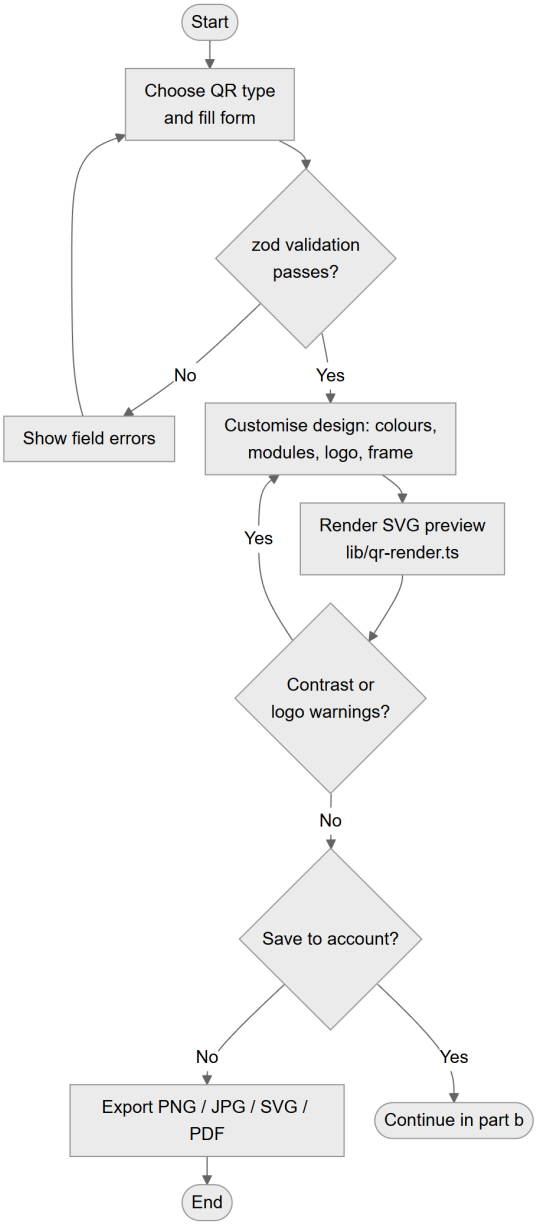


Figure 4.8(a): QR generation in the browser

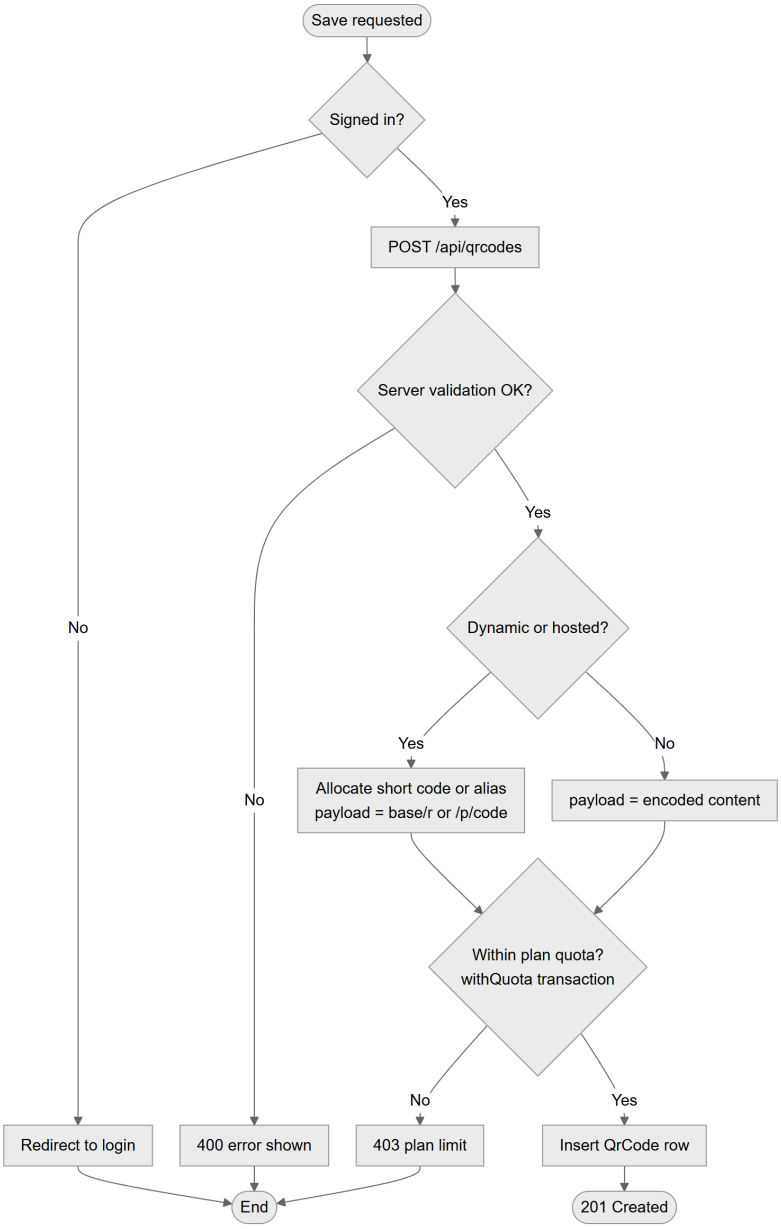


Figure 4.8(b): Saving a QR code on the server

Figure 4.8 shows that generation and export are fully client-side; the server is involved only when a signed-in user saves the code. Server-side validation is repeated before the quota check and insert.

4.8.4 Barcode Generation and Scanning

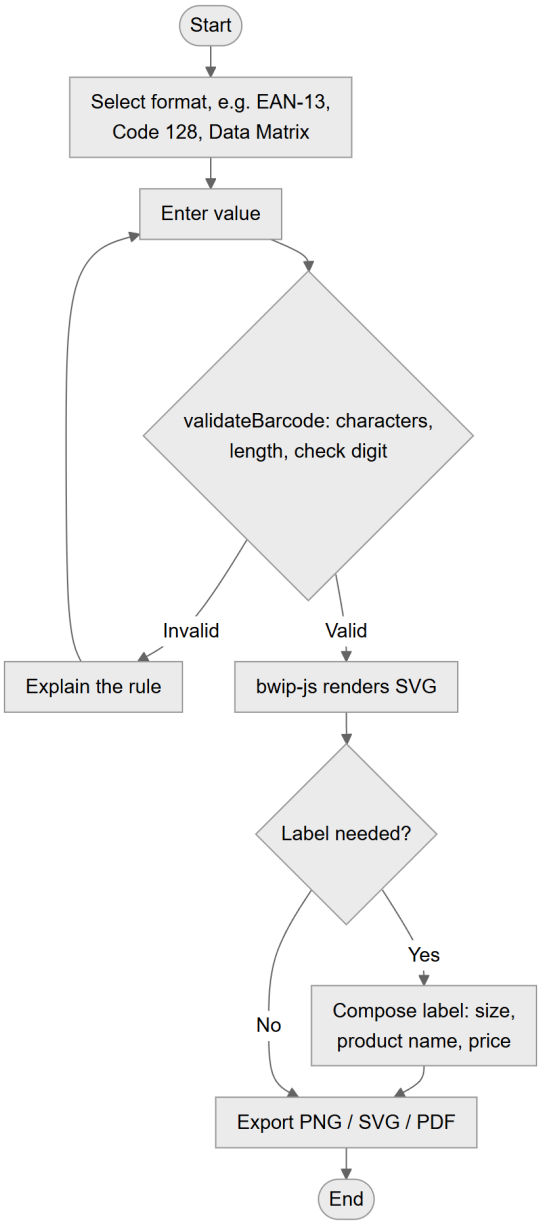


Figure 4.9(a): Barcode generation

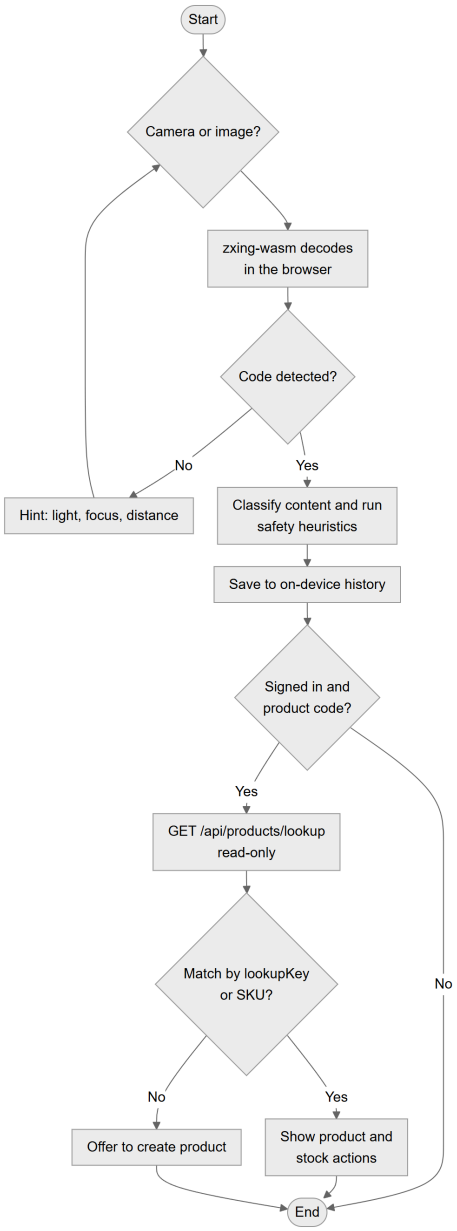


Figure 4.9(b): Barcode and QR scanning

Figure 4.9 has two branches. Generation validates the value for the chosen symbology before rendering with bwip-js and optionally composing a label. Scanning decodes in the browser, classifies the result, runs safety heuristics, stores the entry in the device's local history and — only for signed-in users — performs a read-only product lookup.

4.8.5 Product and Inventory Workflow

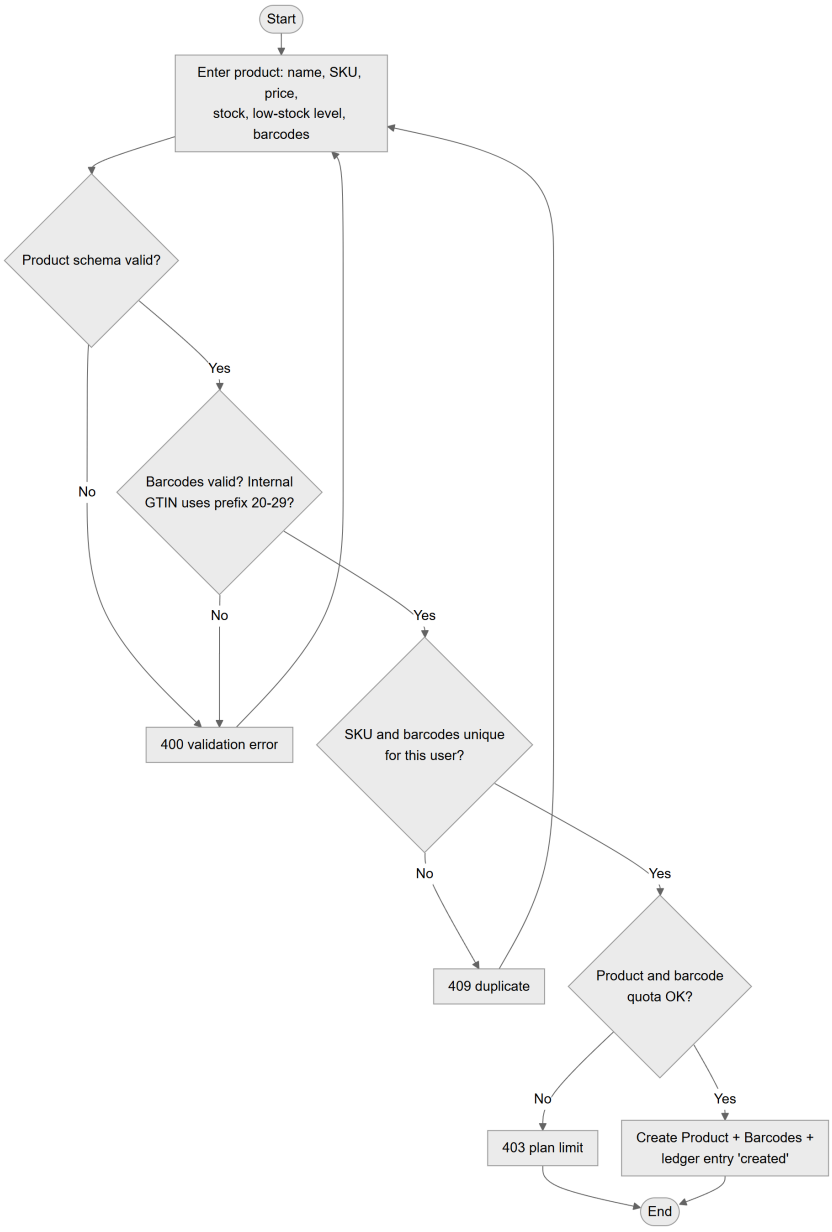


Figure 4.10(a): Product creation

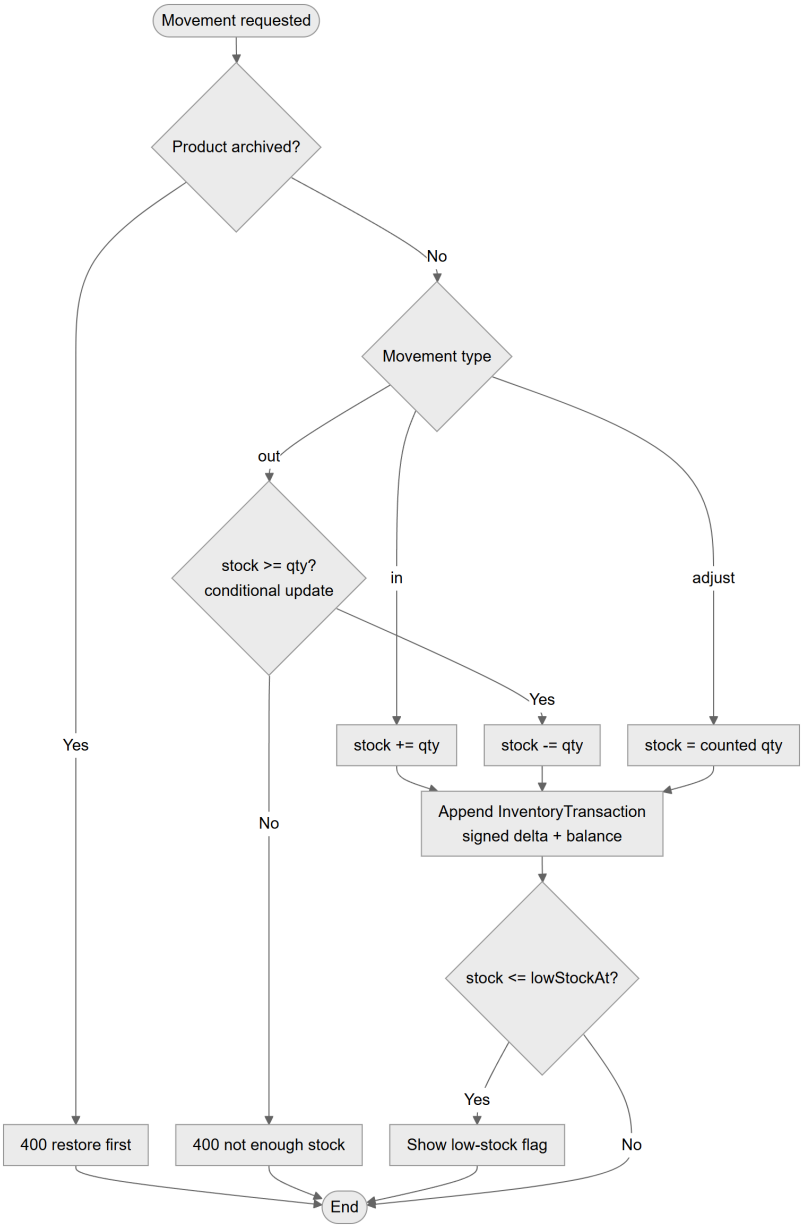


Figure 4.10(b): Stock movement

Figure 4.10 shows that a product is created only after field validation, barcode validation, uniqueness checks and quota checks. Stock then changes exclusively through movements; a stock-out uses a conditional update (`stock >= quantity`) so concurrent requests cannot drive stock negative. Every movement appends a ledger entry.

4.8.6 Dynamic QR Resolution

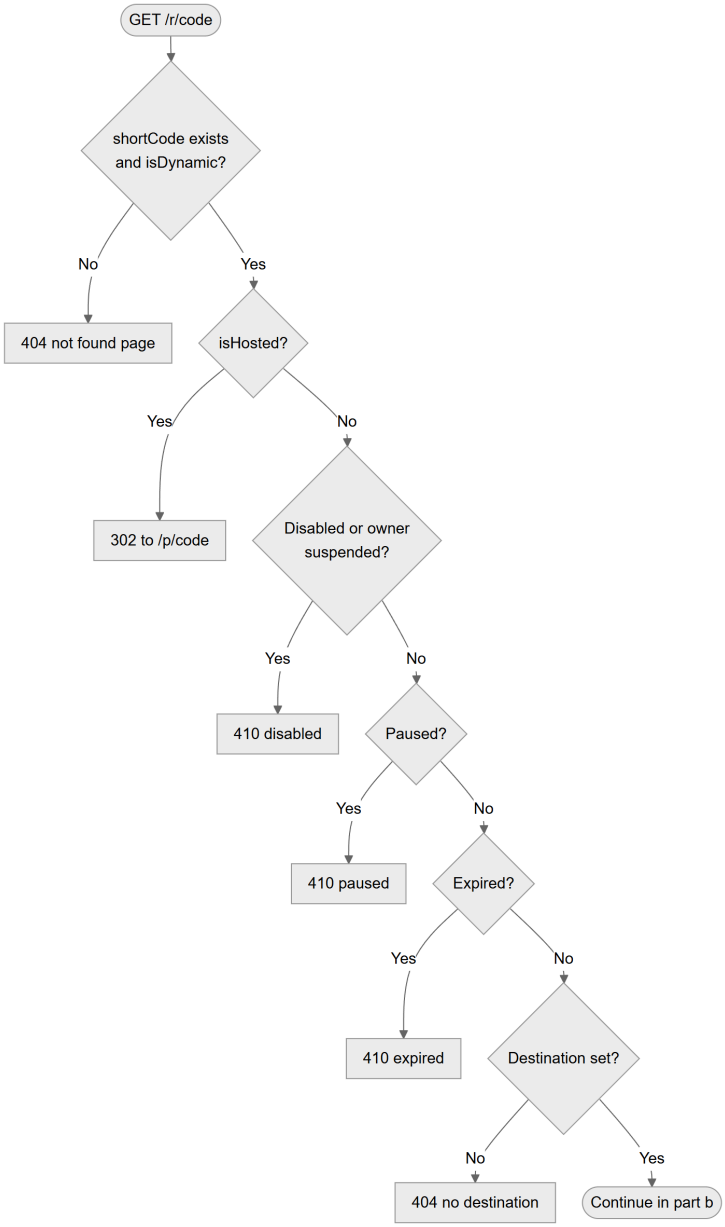


Figure 4.11(a): Dynamic QR resolution — status checks

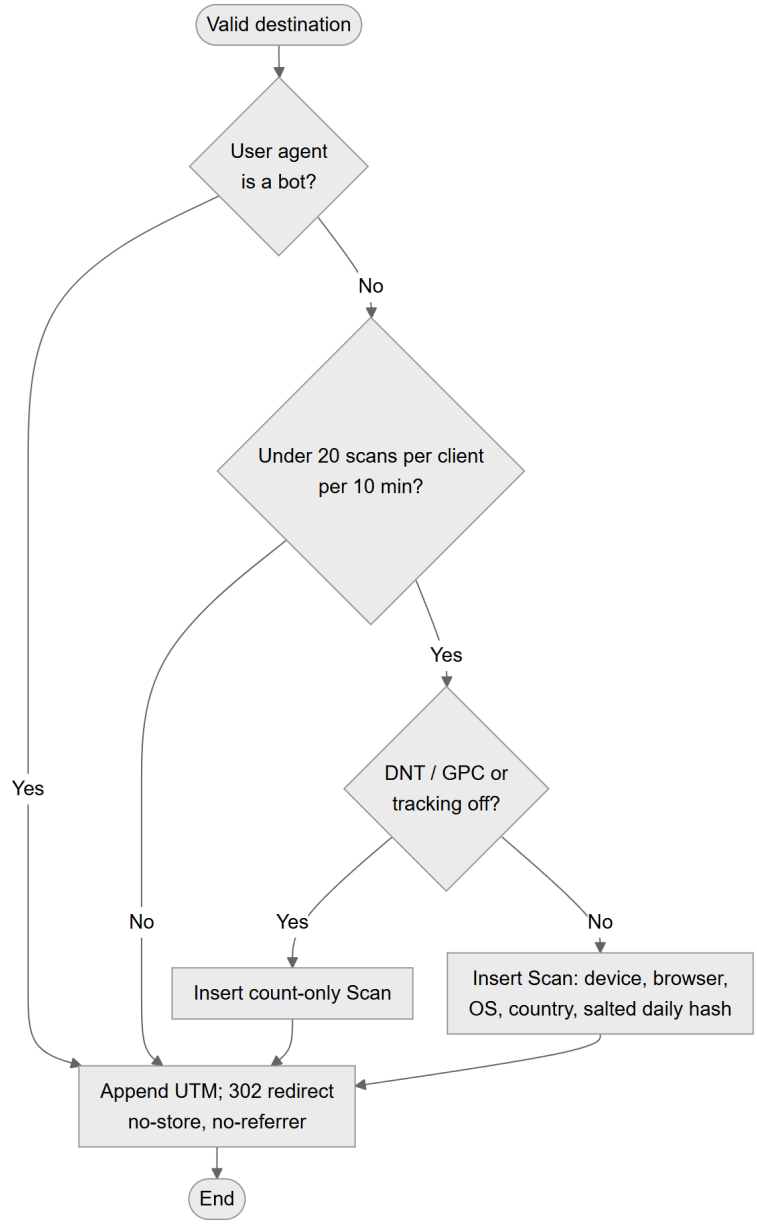


Figure 4.11(b): Dynamic QR resolution — scan recording and redirect

Figure 4.11 shows the order of checks in `src/app/r/[code]/route.ts`. Visitors are always served (redirected or shown an explanatory page); bots, rate-limited clients and privacy signals only affect what is *recorded*.

4.8.7 Subscription and Payment Workflow

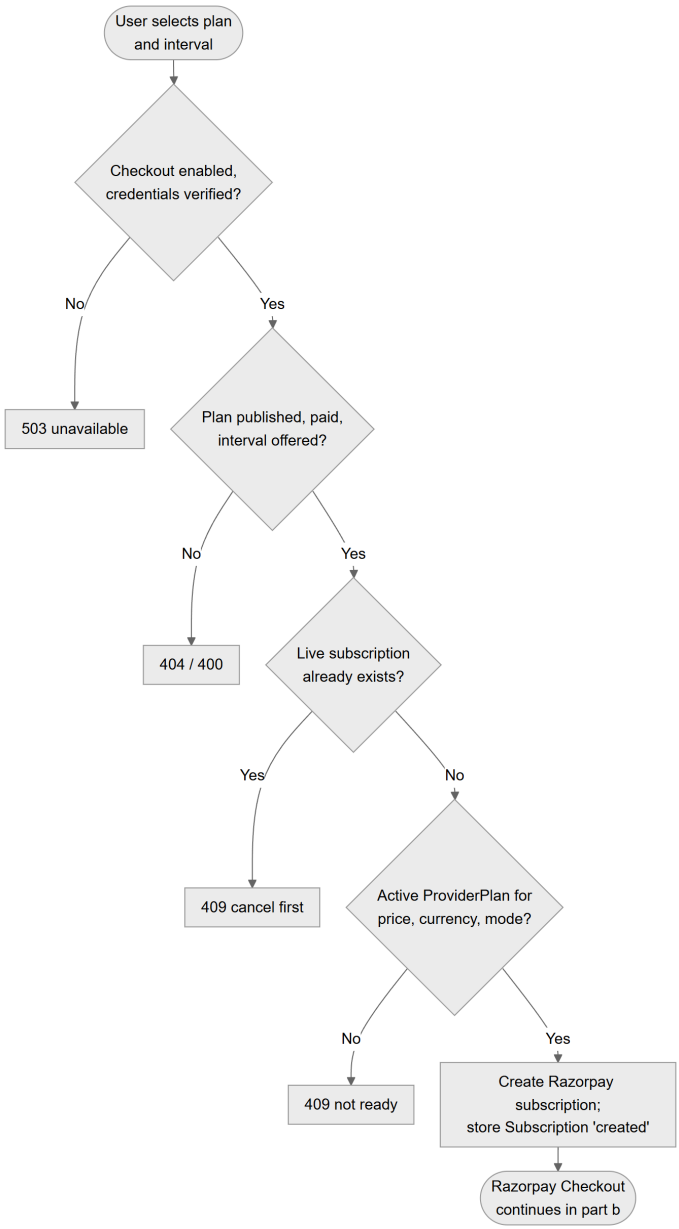


Figure 4.12(a): Subscription checkout — starting checkout

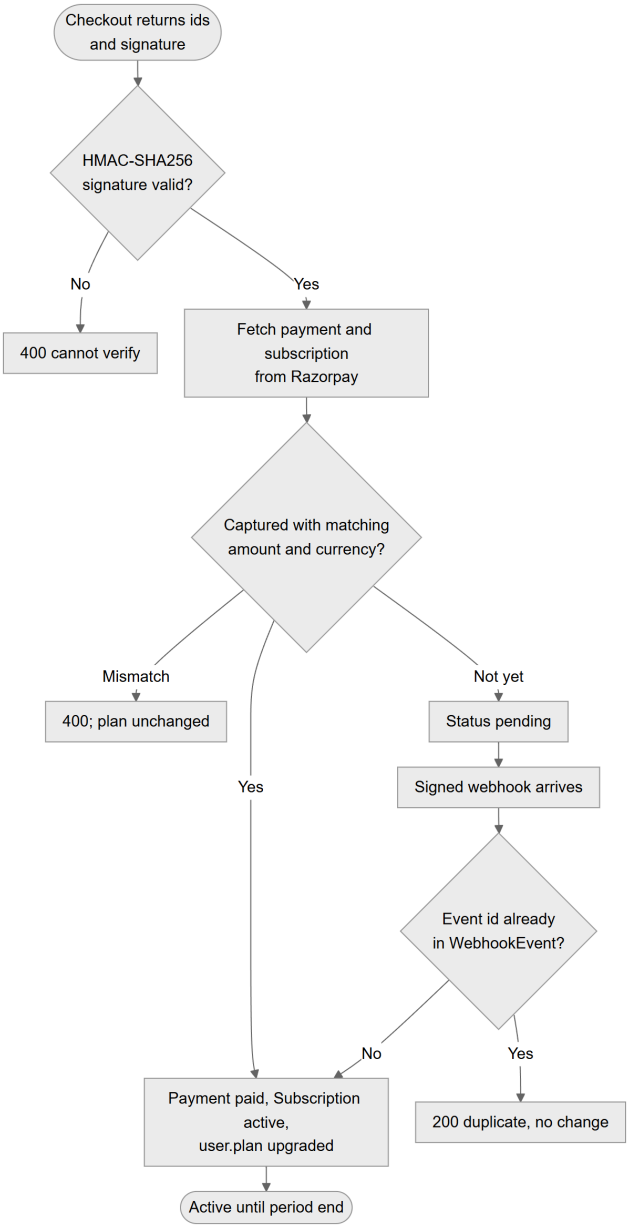


Figure 4.12(b): Subscription checkout — confirmation and webhook

Figure 4.12 shows that the client never supplies a price: it sends a plan identifier and interval, and the server looks up the amount and the active provider plan. The plan is upgraded only after verification of the checkout signature *and* an API fetch showing a captured payment of the right amount, or after a signed webhook. Duplicate webhook deliveries are detected through the `WebhookEvent` ledger.

4.8.8 Administrator Configuration Workflow

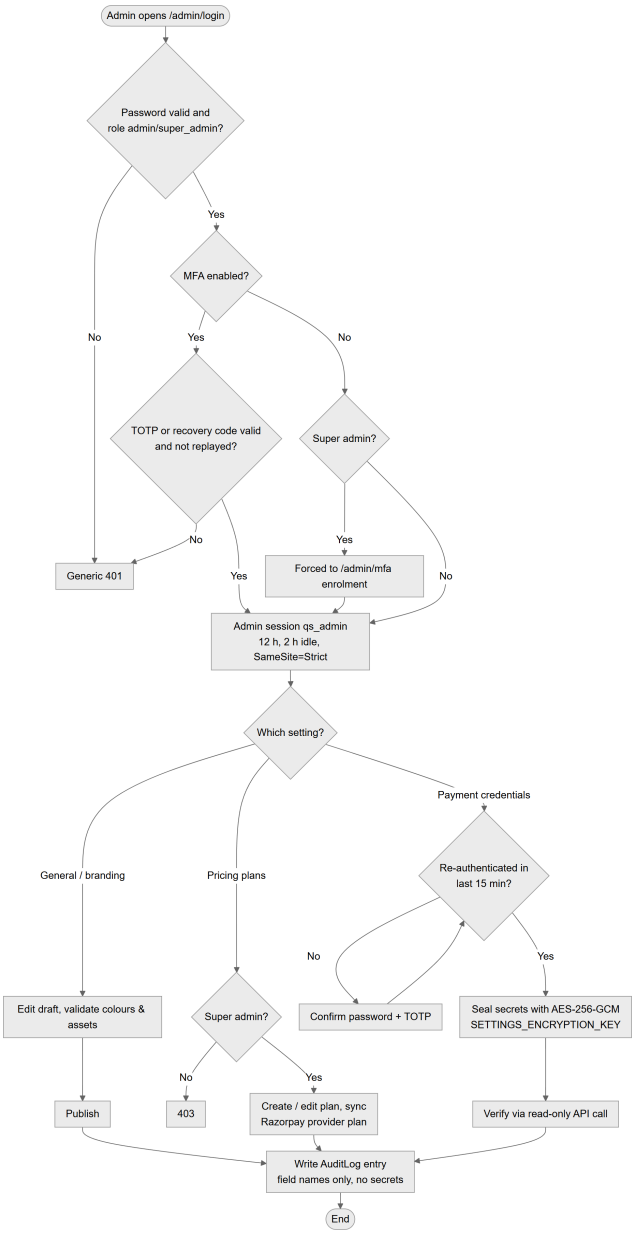


Figure 4.13: Flowchart: administrator configuration workflow

Figure 4.13 shows the admin sign-in (with MFA), the separation between settings any admin may change and super-admin-only settings, the re-authentication requirement for payment credentials, sealing of secrets with AES-256-GCM and the audit entry written for each change.

4.9 Database Design

4.9.1 Conceptual Data Model

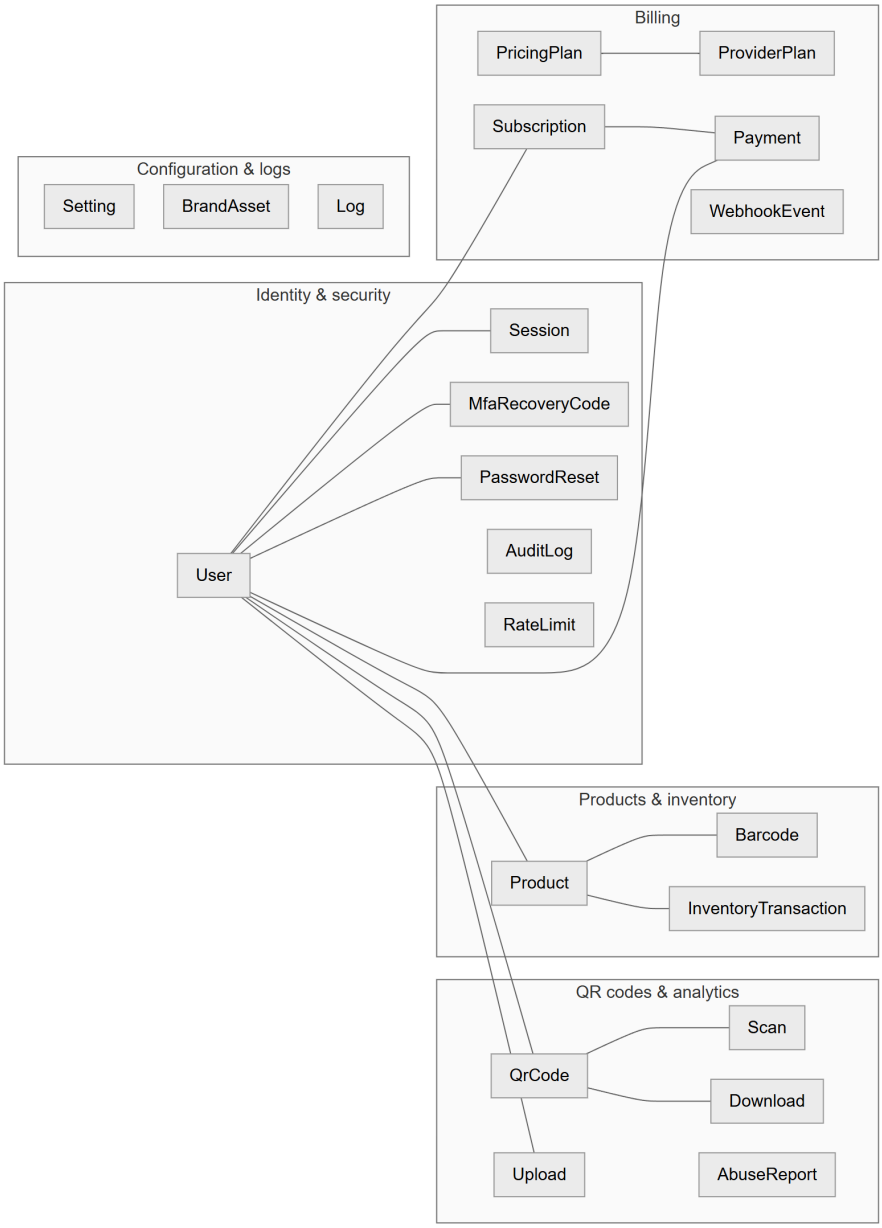


Figure 4.14: Conceptual data model (entity groups)

The conceptual model (Figure 4.14) groups the 22 entities into five areas: identity and security; QR codes and analytics; products and inventory; billing; and configuration and logs. The `User` entity is the hub: almost all business data is owned by a user, which implements multi-tenancy at the row level.

4.9.2 Entity-Relationship Diagrams

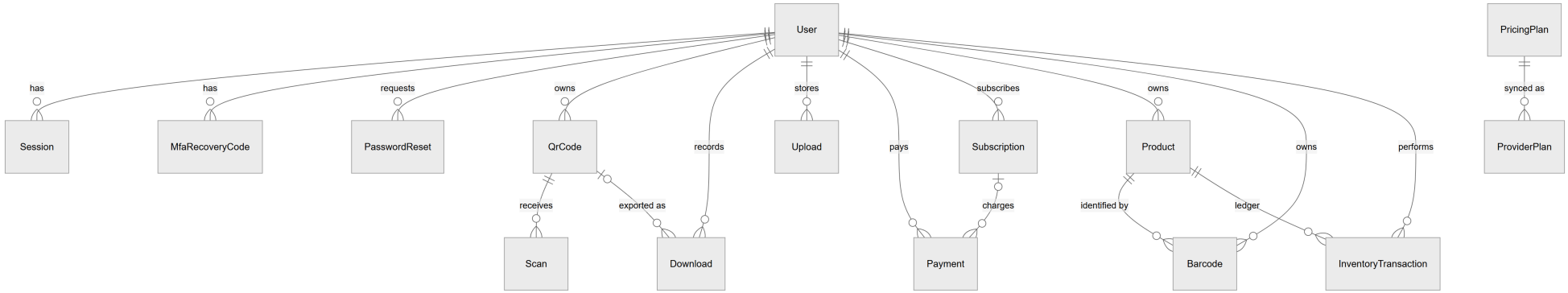


Figure 4.15: Entity-relationship overview (15 related models; 7 standalone models listed in Table 4.6)

Figure 4.15 shows all foreign-key relationships. Crow's-foot notation is used: `||--o{` means "exactly one to zero or more" and `|o--o{` "zero or one to zero or more" (an optional foreign key). The detailed ERDs below include the main attributes of each entity.

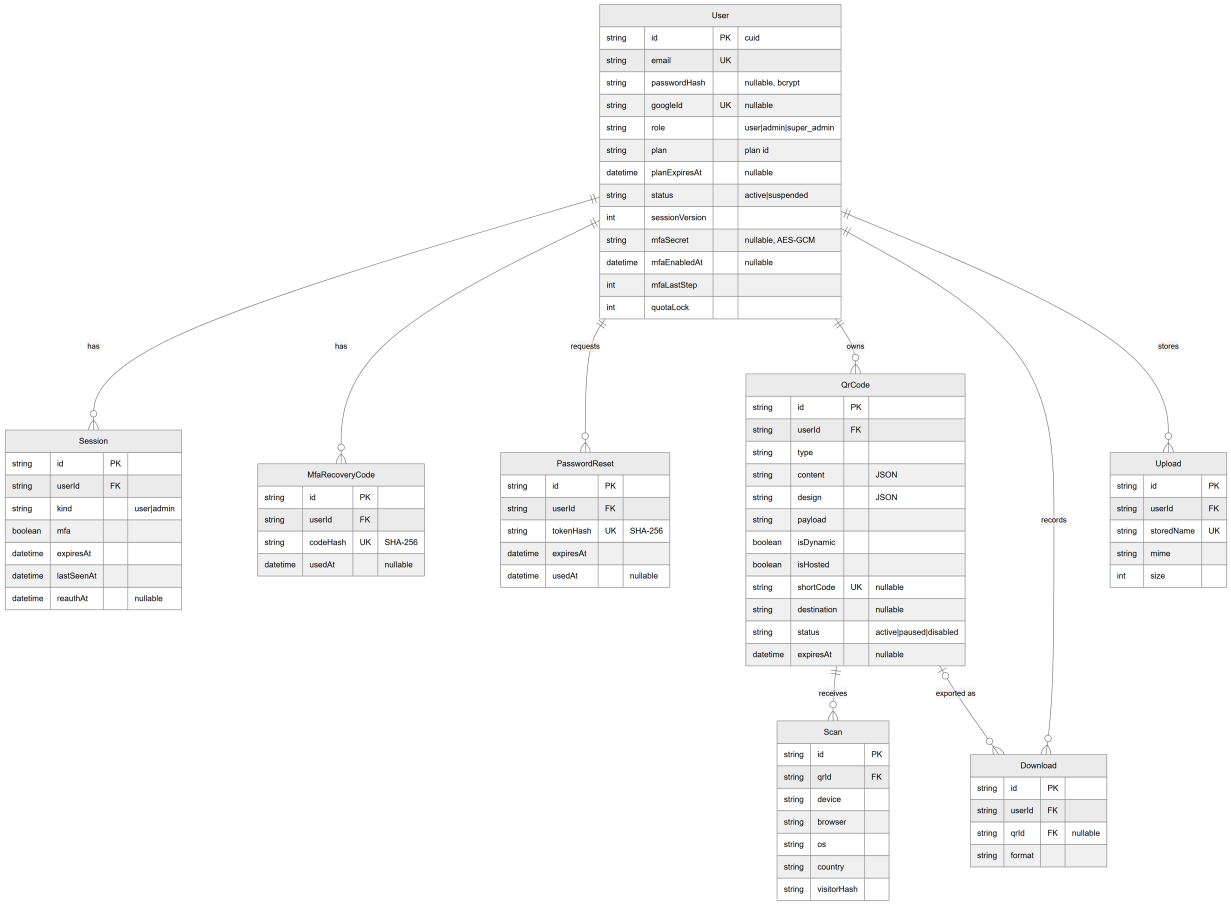


Figure 4.16: ERD detail: accounts, sessions, QR codes and analytics

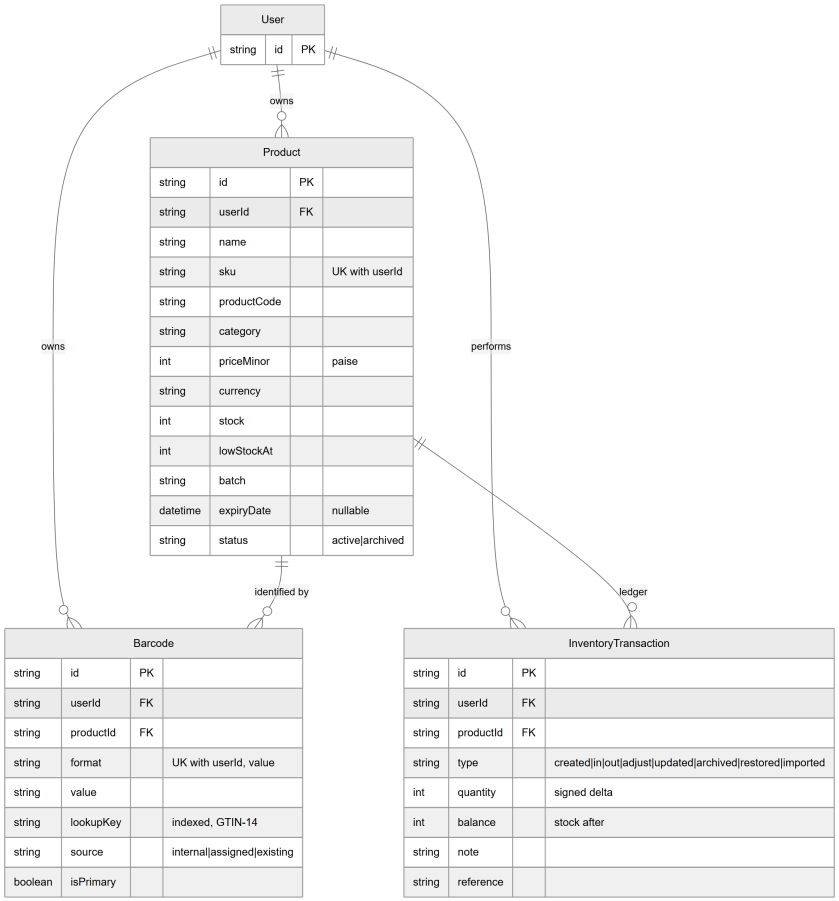


Figure 4.17: ERD detail: products, barcodes and inventory ledger

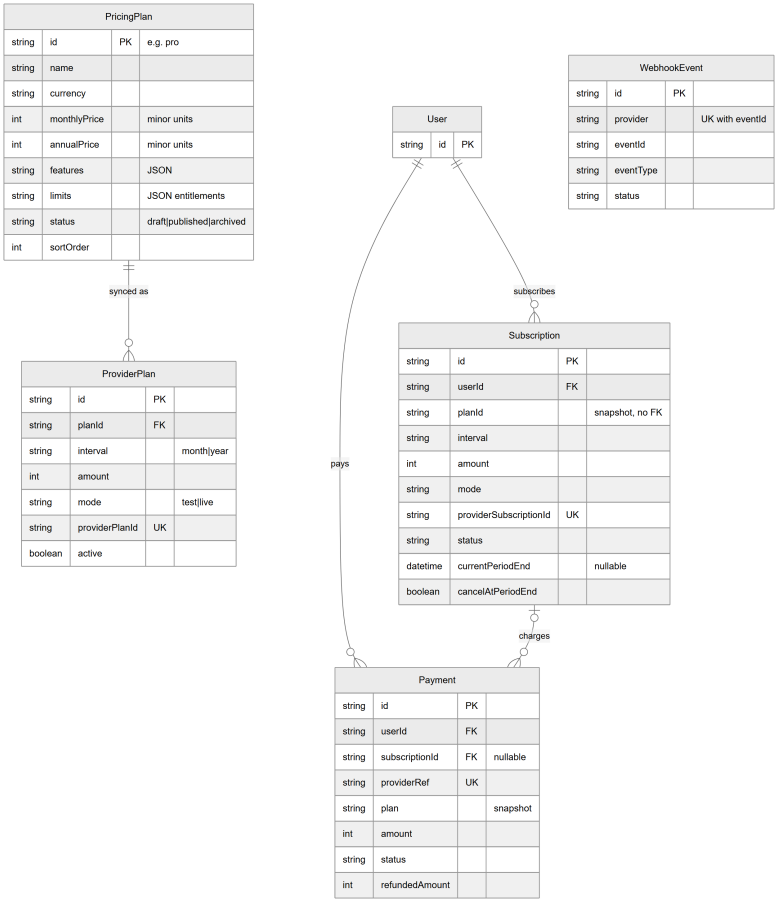


Figure 4.18: ERD detail: pricing plans, subscriptions and payments

Figures 4.16–4.18 were checked field by field against `prisma/schema.prisma`. Note that `Subscription.planId` and `Payment.plan` are deliberately *snapshots* without a foreign key to `PricingPlan`, so that later plan edits or archiving never rewrite payment history.

4.9.3 Logical Design: Entities and Relationships

Table 4.3: Relationships and cardinalities

Parent	Child	Cardinality	Foreign key	On delete
User	Session, MfaRecoveryCode, PasswordReset	1 : N	`userId`	Cascade
User	QrCode, Download, Upload	1 : N	`userId`	Cascade
User	Product, Barcode, InventoryTransaction	1 : N	`userId`	Cascade
User	Subscription, Payment	1 : N	`userId`	Cascade
QrCode	Scan	1 : N	`qrId`	Cascade
QrCode	Download	0..1 : N	`qrId` (nullable)	Set null
Product	Barcode, InventoryTransaction	1 : N	`productId`	Cascade
Subscription	Payment	0..1 : N	`subscriptionId` (nullable)	Set null
PricingPlan	ProviderPlan	1 : N	`planId`	Restrict

4.9.4 Physical Design

The canonical schema is `prisma/schema.prisma` (SQLite provider) managed with Prisma ORM [5]. The MySQL schema `prisma/mysql/schema.prisma` is *generated* from it by `scripts/db/mysql-schema.mjs`, which adds MySQL column types (`TEXT`, `MEDIUMTEXT`, `VARCHAR(n)`) for long strings and `relationMode = "foreignKeys"`. MySQL has two versioned migrations: `20261025000000_init` and `20261025180000_billing_branding`. SQLite development databases are synchronised with `prisma db push` and have no migration history. Primary keys are CUID strings generated by Prisma (`@default(cuid())`), except `PricingPlan.id` (a readable plan identifier such as `pro`), `Setting.key` and `RateLimit.key`. Money is stored as integers in minor units. JSON documents (QR content and design, UTM, plan features and limits, audit details) are stored as text and validated by zod in the application.

Table 4.4: Physical model summary (generated from `prisma/schema.prisma`)

Model	Scalar columns	Unique constraints	Secondary indexes	Foreign keys to
User	17	2	0	—
PasswordReset	6	1	0	User
QrCode	20	1	1	User
Scan	9	0	1	QrCode
Download	6	0	1	User, QrCode
Upload	7	1	0	User
AbuseReport	8	0	0	—
Payment	16	1	2	User, Subscription
PricingPlan	13	0	0	—
ProviderPlan	9	1	1	PricingPlan
Subscription	16	1	1	User
WebhookEvent	7	1	1	—
BrandAsset	9	1	1	—
Log	6	0	1	—
Setting	2	0	0	—
Product	20	1	2	User
Barcode	9	1	2	User, Product
InventoryTransaction	9	0	2	User, Product
Session	9	0	2	User

Model	Scalar columns	Unique constraints	Secondary indexes	Foreign keys to
MfaRecoveryCode	5	1	1	User
AuditLog	9	0	3	—
RateLimit	3	0	1	—

4.9.5 Indexes and Constraints

Table 4.5: Unique constraints and indexes, and their purpose

Model	Constraint / index	Purpose
User	`email` UNIQUE; `googleId` UNIQUE	One account per e-mail; link Google identity
QrCode	`shortCode` UNIQUE; index `userId`	Global short-link namespace; list a user's codes
Scan	index (`qrId`, `createdAt`)	Time-range analytics per code
Download	index (`userId`, `createdAt`)	Download history
Upload	`storedName` UNIQUE	Serve file by random name
Product	UNIQUE (`userId`, `sku`); index (`userId`, `status`); index (`userId`, `category`)	Per-user SKU uniqueness; filtered lists
Barcode	UNIQUE (`userId`, `format`, `value`); index (`userId`, `lookupKey`); index `productId`	Per-user barcode uniqueness; scan lookup
InventoryTransaction	index (`productId`, `createdAt`); index (`userId`, `createdAt`)	Product history; user activity
Payment	`providerRef` UNIQUE; index (`userId`, `createdAt`); index `subscriptionId`	Idempotent payment recording
Subscription	`providerSubscriptionId` UNIQUE; index (`userId`, `status`)	Map provider events; find live subscription
ProviderPlan	`providerPlanId` UNIQUE; index (`planId`, `interval`, `mode`, `active`)	Find active provider plan for checkout
WebhookEvent	UNIQUE (`provider`, `eventId`); index `receivedAt`	Duplicate delivery detection
Session	index `userId`; index `expiresAt`	Revoke all sessions; clean-up
MfaRecoveryCode	`codeHash` UNIQUE; index `userId`	One-time code lookup
PasswordReset	`tokenHash` UNIQUE	Token lookup by hash
AuditLog	index `createdAt`; `action`; `actorId`	Filter audit log
BrandAsset	`storedName` UNIQUE; index (`kind`, `createdAt`)	Serve and list assets
Log, RateLimit	index `createdAt` / `resetAt`	Pagination; expiry clean-up

4.9.6 Standalone Models

Table 4.6: Models without foreign-key relationships

Model	Role	Why standalone
AbuseReport	Reports from visitors about a short code or URL	Reporter is anonymous; reported code may be deleted later
WebhookEvent	Ledger of processed provider events	Exists before any matching subscription may be found
BrandAsset	Uploaded logos, favicon, social image	Site-level, not user-owned; `createdBy` stores the admin id as text
Log	System log shown in the console	Must survive user deletion; `userId` kept as plain text
Setting	Key–value site settings (including sealed credentials and branding JSON)	Global configuration
AuditLog	Security and admin events	Must survive deletion of the actor; `actorId` kept as plain text
RateLimit	Fixed-window counters (database store)	Keyed by arbitrary strings

4.9.7 Enumerated and Status Fields

Status fields are stored as strings (SQLite has no native enum type); permitted values are enforced by zod schemas and code.

Table 4.7: Enumerated fields

Field	Values
`User.role`	`user`, `admin`, `super_admin`
`User.status`	`active`, `suspended`
`User.plan`	Plan identifier, default `free` (defaults: `free`, `pro`, `business`)
`Session.kind`	`user`, `admin`
`QrCode.status`	`active`, `paused`, `disabled` (admin only)
`Scan.device`	`mobile`, `tablet`, `desktop`, `bot`, `unknown`
`Product.status`	`active`, `archived`
`Barcode.source`	`internal`, `assigned`, `existing`
`InventoryTransaction.type`	`created`, `in`, `out`, `adjust`, `updated`, `archived`, `restored`, `imported`
`AbuseReport.status`	`open`, `resolved`, `dismissed`
`Payment.status`	`created`, `pending`, `paid`, `failed`, `expired`, `refunded`, `partially_refunded`
`Subscription.status`	`created`, `authenticated`, `active`, `pending`, `halted`, `cancelled`, `completed`, `expired`
`PricingPlan.status`	`draft`, `published`, `archived`
`ProviderPlan.mode` / `Subscription.mode`	`test`, `live`
`WebhookEvent.status`	`processed`, `ignored`, `rejected`

4.9.8 Audit-Related Fields

Most tables carry `createdAt` (and mutable ones `updatedAt`, maintained by Prisma). Security-relevant history is kept in `AuditLog` (actor, action, target, redacted JSON details, hashed IP), `Log`, `InventoryTransaction` (business audit of stock), `Session.lastSeenAt`/`reauthAt`, `User.lastLoginAt`, and `WebhookEvent`. The full table-by-table data dictionary is in Appendix C.

4.10 Database Normalization

The schema was assessed against the normal forms using the actual columns:

Table 4.8: Normalization assessment

Normal form	Assessment
First normal form (1NF)	Each column holds a single value **at the database level**, and each table has a primary key. However, several columns store JSON documents (`QrCode.content`, `QrCode.design`, `QrCode.utm`, `PricingPlan.features`, `PricingPlan.limits`, `AuditLog.details`, branding in `Setting.value`). The database treats them as opaque strings; they are never queried by their internal fields. Strictly, these are non-atomic attributes; the design accepts this to store type-specific, schema-less data that is always read and written as a whole.
Second normal form (2NF)	All primary keys are single-column surrogate keys (or a single natural key), so no partial dependency on part of a composite key can exist. 2NF holds for every table.
Third normal form (3NF)	Non-key attributes depend on the key. Deliberate exceptions are **snapshots** for history: `Payment.plan`, `planName`, `interval`, `amount` and `Subscription.planName`, `amount` copy plan data at the time of payment so history does not change when a plan is edited. `InventoryTransaction.balance` is a derived value stored for an immutable ledger. `Product.stock` is redundant with the sum of ledger quantities, kept for fast reads and protected by atomic updates. `Barcode.userId` duplicates `Product.userId` so that uniqueness can be enforced per user by a database constraint.
BCNF	Candidate keys in tables with composite unique constraints (`Product(userId, sku)`, `Barcode(userId, format, value)`, `WebhookEvent(provider, eventId)`) determine the remaining attributes, and no other non-trivial functional dependencies were found; apart from the intentional redundancies listed above, the design is consistent with BCNF.

In summary, the schema is normalised to 3NF for its relational data, with documented, intentional denormalisation for history snapshots, ledger balances and per-tenant constraints, and with JSON columns for flexible documents.

4.11 Class Diagram

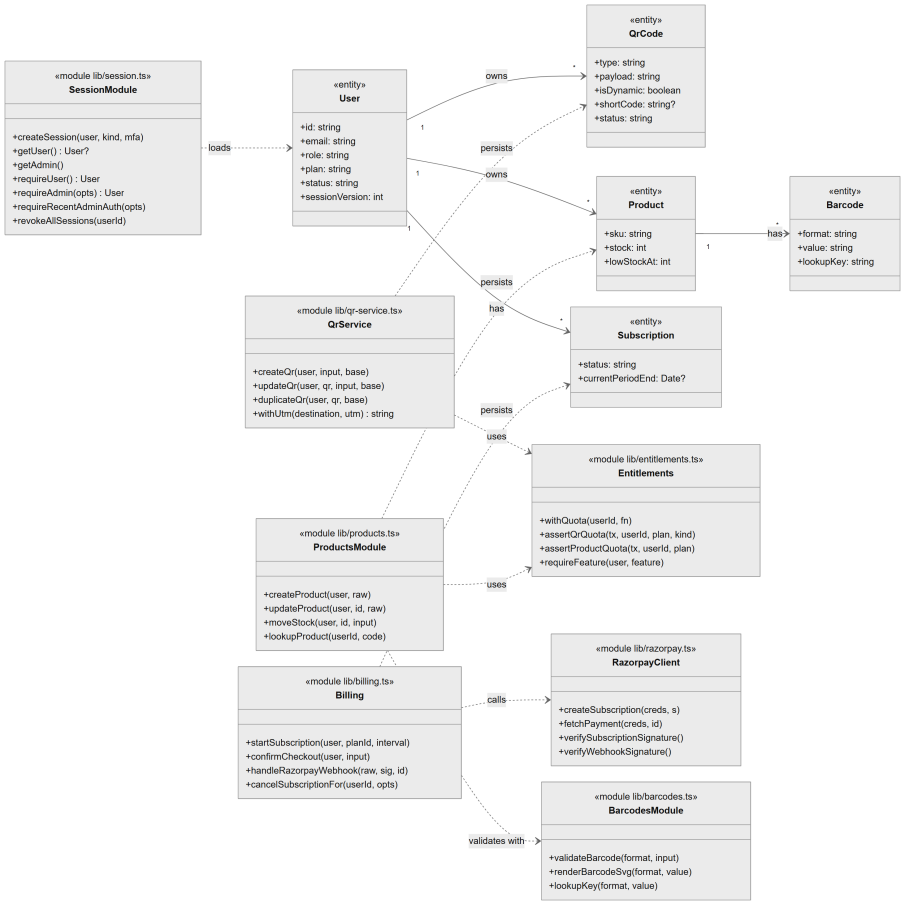


Figure 4.19: Class diagram: domain entities and server-side service modules

QRSathi is written in a functional TypeScript style: the server-side "classes" are ES modules exporting functions, and database entities are Prisma-generated types. Figure 4.19 therefore uses two stereotypes. `<<entity>>` classes are Prisma models (only key attributes are shown). `<<module ...>>` classes are server modules with their main exported functions. Solid arrows with multiplicities are associations between entities; dashed arrows are "uses" dependencies of modules. The only true TypeScript classes in the server code are small error types (`ApiError`, `RazorpayError`), which are omitted.

4.12 Sequence Diagrams

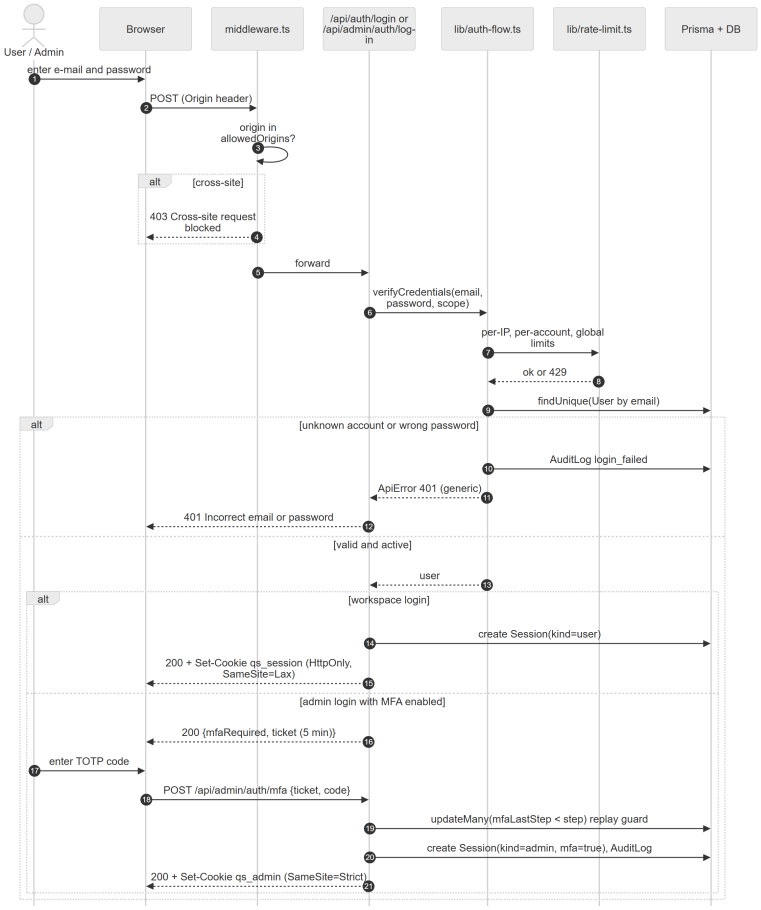


Figure 4.20: Sequence diagram: workspace login and administrator MFA login

Figure 4.20 traces a login through the middleware, the login route, the shared credential check and the database. The `alt` fragments show the generic failure path and the two success paths: a workspace session, or — for an MFA-enabled administrator — a short-lived ticket followed by TOTP verification with an atomic replay guard.

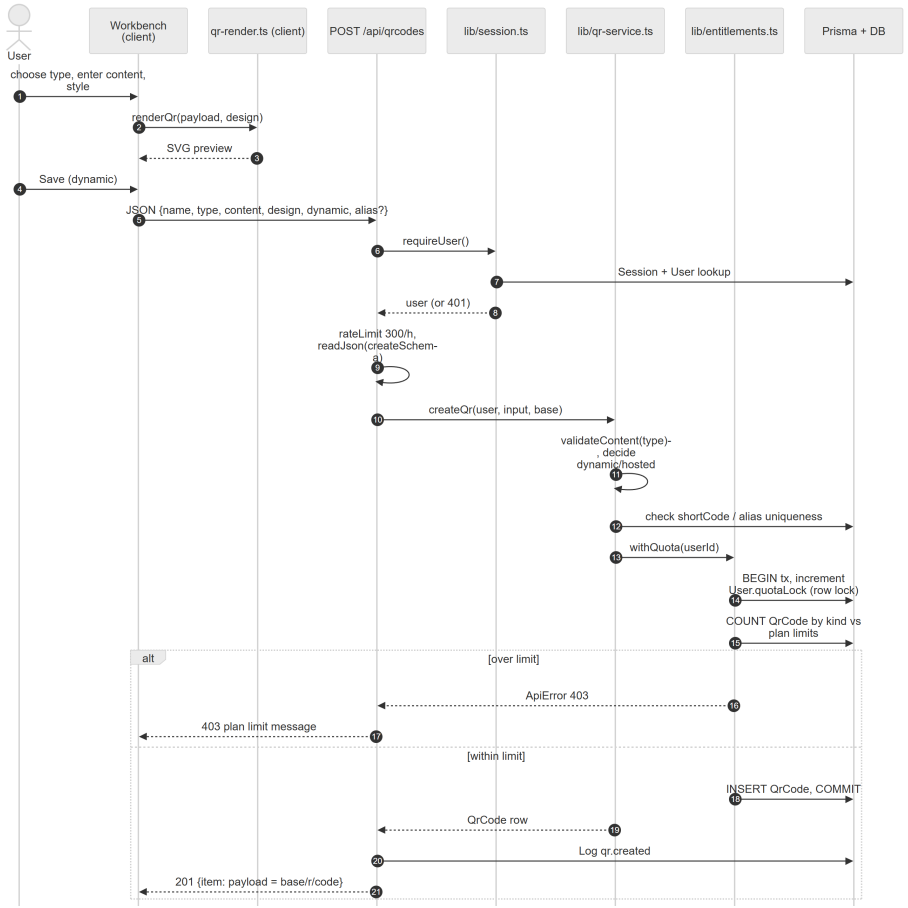


Figure 4.21: Sequence diagram: saving a dynamic QR code

Figure 4.21 shows client-side rendering followed by the save request. Note the transaction in `withQuota`: the first statement increments `User.quotaLock`, which takes a row lock in MySQL (or the write lock in SQLite) so that two concurrent saves by the same user are serialised and cannot both pass the limit check.

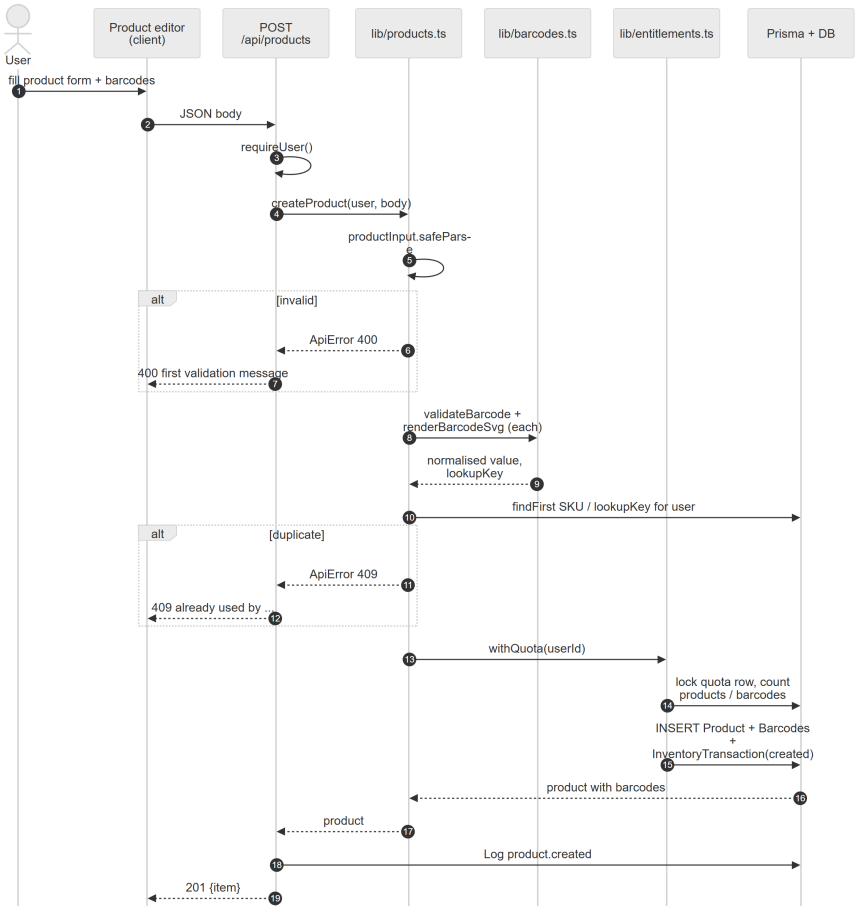


Figure 4.22: Sequence diagram: database-backed product creation

Figure 4.22 shows validation, barcode normalisation, uniqueness checks and the nested insert of a product with its barcodes and first ledger entry in one transaction.

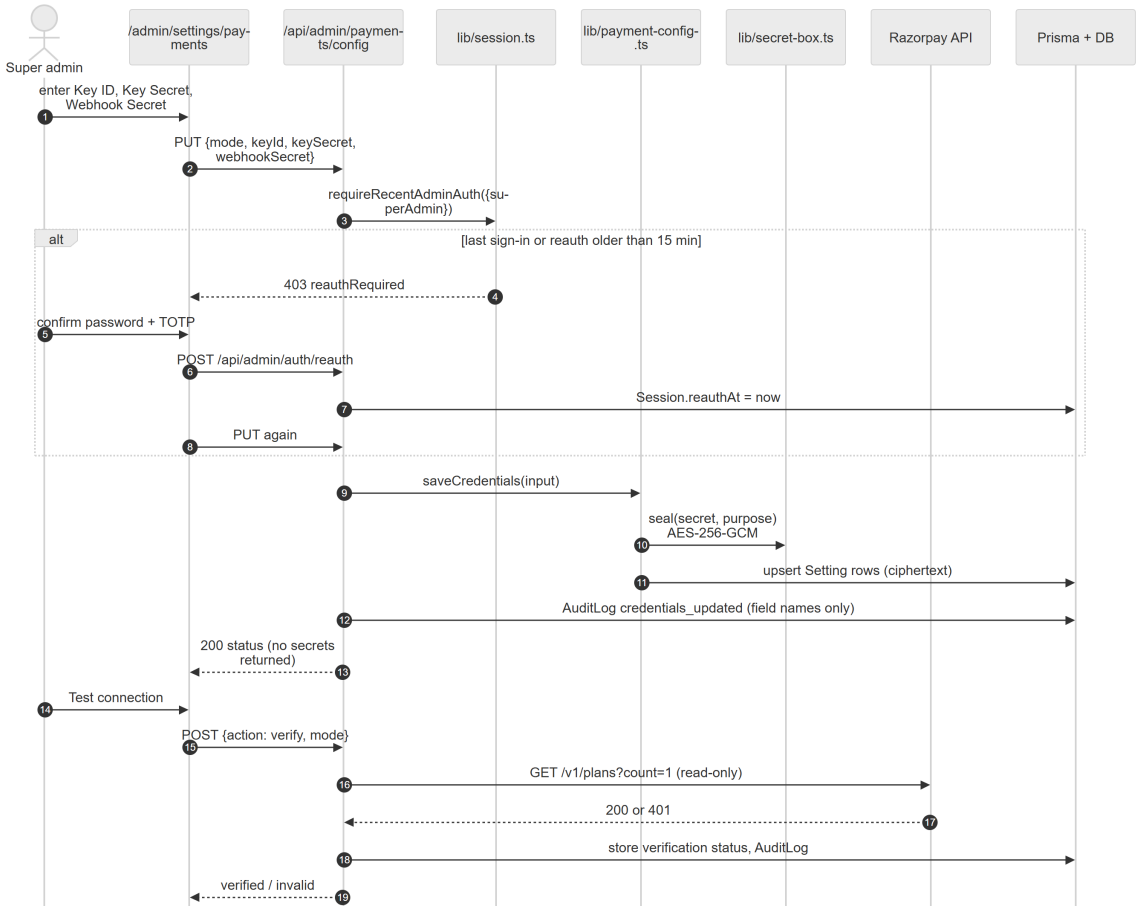


Figure 4.23: Sequence diagram: super administrator saves payment-gateway credentials

Figure 4.23 shows the re-authentication loop, sealing of secrets, audit logging of field *names* only, and a read-only verification call to Razorpay that creates nothing.

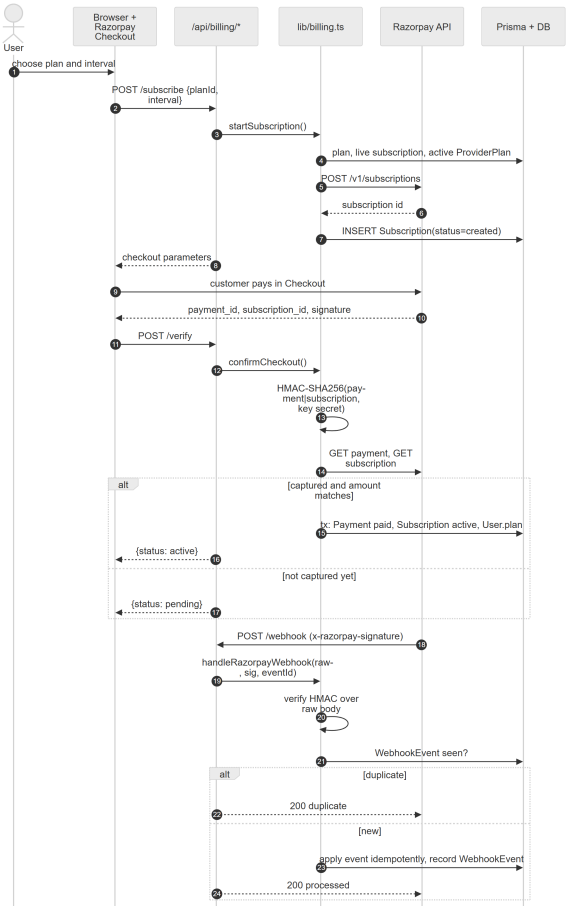


Figure 4.24: Sequence diagram: subscription checkout, verification and webhook

Figure 4.24 shows the two independent confirmation paths (client callback with server verification, and the signed webhook) and how idempotency prevents a payment from being applied twice.

4.13 Component Diagram

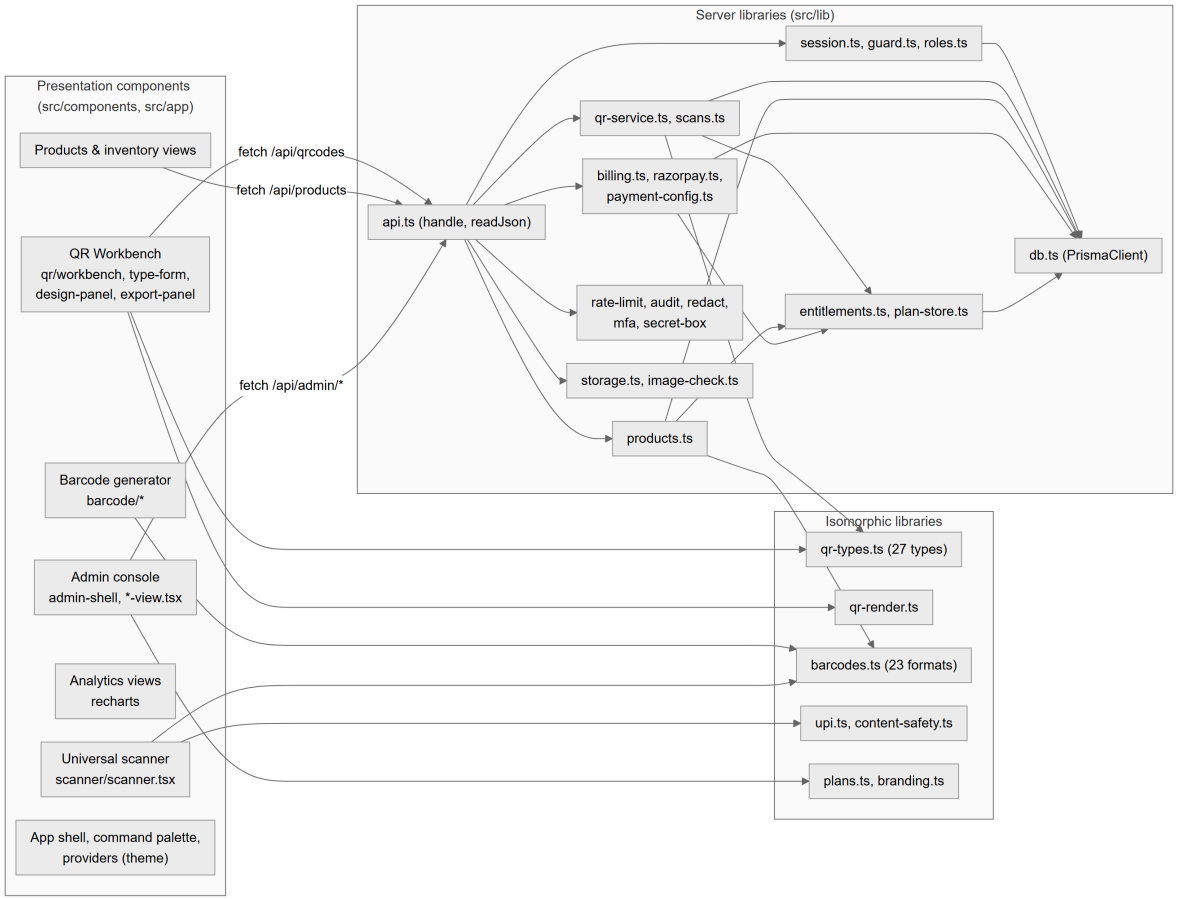


Figure 4.25: Component diagram

Figure 4.25 groups the code into presentation components, isomorphic libraries shared by client and server, and server-only libraries. Presentation components call the server only through `fetch` to API routes; they never import server modules. Isomorphic libraries contain no React or database imports, which allows the same validation and rendering code to run on both sides.

4.14 Deployment Diagram

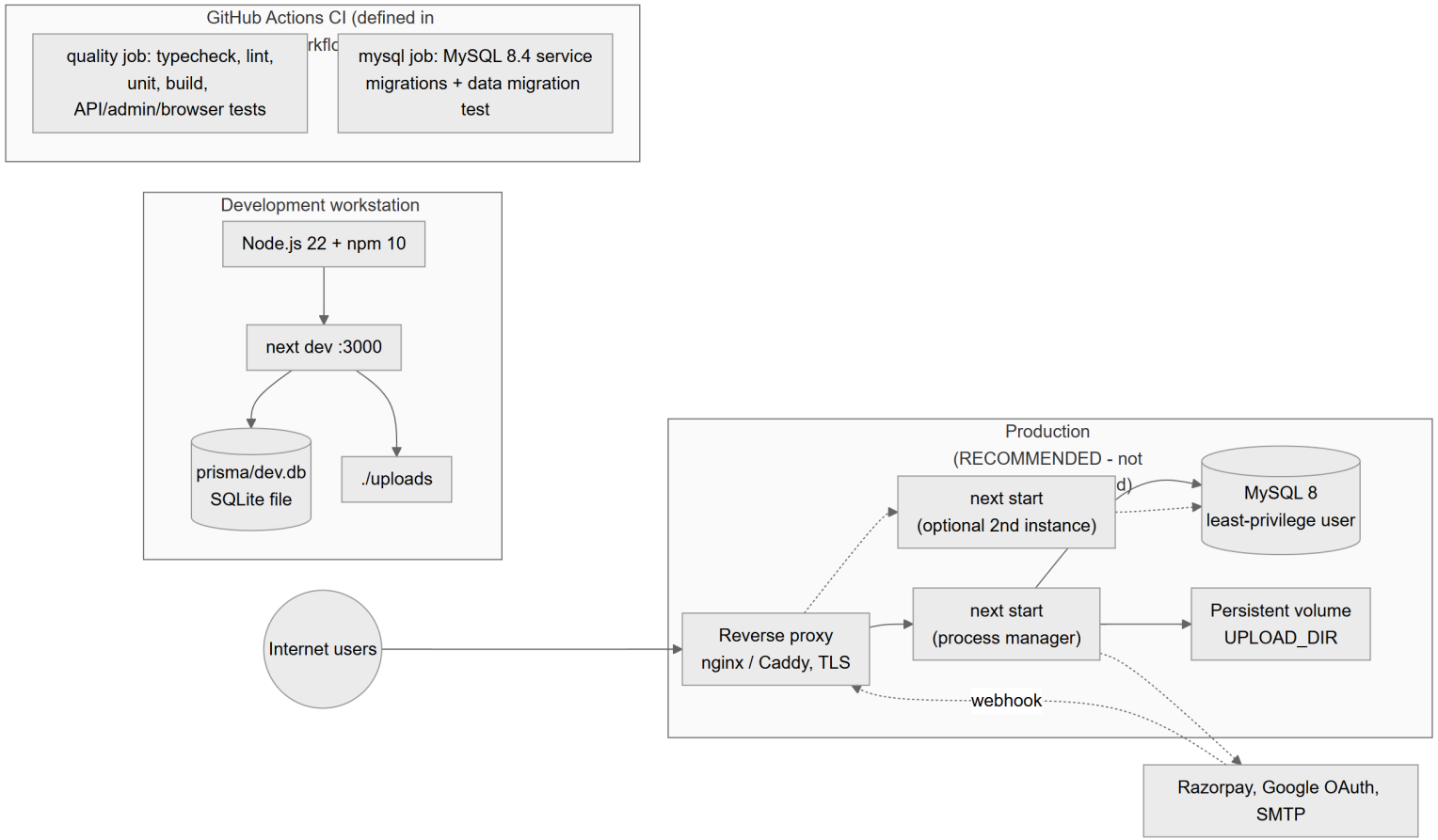


Figure 4.26: Deployment diagram: verified development set-up and recommended production topology

Figure 4.26 distinguishes what was verified from what is recommended. **Verified:** the development set-up (Node.js 22, `next dev`, SQLite file, local uploads) and the isolated production build used for testing (`next start` on port 3100 with a throw-away SQLite database), and the CI workflow definition. **Recommended (not deployed or verified in this audit):** a reverse proxy terminating TLS in front of one or more `next start` processes, a MySQL 8 server with a least-privilege user, and a persistent volume for uploads, as described in `docs/DEPLOYMENT.md`.

4.15 UI/UX Design

Navigation. The workspace uses a sidebar with four groups defined in `src/components/nav-config.ts`: *Workspace* (Overview, QR Generator, Barcode Generator, Universal Scanner, Scan History), *Business* (Products, Inventory, Bulk Generator, Analytics), *More tools* (My QR Codes, Dynamic QR, UPI Payment QR, Design Studio, Plans & Pricing; collapsed by default) and *Account* (Settings, Help & Support). A command palette offers keyword search across these destinations. The public site has marketing pages (home, features, tools, pricing, about, contact, help, privacy, terms, report). The admin console has its own shell and navigation (`src/components/admin/admin-nav.ts`) and never shows workspace navigation (verified by the admin test suite).

Table 4.9: Principal pages

Area	Routes
Public site	/, /features, /tools, /pricing, /about, /contact, /help, /privacy, /terms, /report
Authentication	/login, /signup, /forgot-password, /reset-password
Workspace	/dashboard, /generator, /barcodes, /scanner, /scan-history, /codes, /dynamic, /upi, /studio, /bulk, /analytics, /analytics/[id], /products, /products/new, /products/[id], /inventory, /billing/status, /dashboard/settings, /dashboard/history
Public QR endpoints	/r/[code] (redirect), /p/[code] (hosted page), /f/[name] (file), /brand/[name] (brand image)
Admin console	/admin (overview), /admin/users, /admin/qr-codes, /admin/reports, /admin/products, /admin/barcodes, /admin/inventory, /admin/analytics, /admin/subscriptions, /admin/payments, /admin/security (+ /admins, /events), /admin/settings (+ /branding, /appearance, /email, /pricing, /payments), /admin/system, /admin/mfa, /admin/login, /admin/forgot-password

Design principles. The interface uses a consistent component library (`src/components/ui.tsx`), clear empty states, inline validation messages from the shared zod schemas, warnings before risky actions (for example, "archive before delete"), and toast notifications for outcomes.

Responsiveness. Layouts use Tailwind responsive utilities. The automated browser suite loads every public and workspace page at 375, 768 and 1280 px and fails if the page scrolls horizontally; all such checks passed for public and workspace pages in the final audit run.

Accessibility. At 1280 px the browser suite checks that every control, link and image has an accessible name. A "Skip to content" link is present in the admin shell. No manual audit against WCAG 2.2 [20] was performed.

Themes and branding. Users can choose light, dark or system themes (stored in `localStorage`). Administrators can change brand colours, which are converted into CSS variables (`themeCss()` in `src/lib/branding.ts`), and the editor checks colour contrast.

4.16 API Design

The API follows a resource-oriented JSON-over-HTTP design implemented with Next.js route handlers. There are 64 API route modules with 89 method handlers (full inventory in Appendix B and `QRSathi_API_Documentation.md`).

Conventions.

- Requests and responses are JSON (`Content-Type: application/json`), except file uploads (multipart), the CSV export, the webhook (raw text) and public HTML routes.
- Every handler is wrapped in `handle()` (`src/lib/api.ts`), which converts an `ApiError(status, message)` into `{ "error": message, "details"?: … }` with that status, and any other exception into a generic 500 message while

logging the details server-side. Responses carry `Cache-Control: no-store`.

- Input is parsed by `readJson(req, schema, maxBytes)`, which enforces a body-size limit, rejects malformed JSON with 400 and returns the first zod validation message.
- Resources of other users return 404.

Table 4.10: HTTP status codes used by the API

Status	Meaning in QRSathi
200 / 201	Success / resource created
400	Invalid input or business-rule violation (e.g. not enough stock)
401	Not signed in, or credentials/MFA code invalid
403	Signed in but not allowed (role, plan limit, MFA required, re-authentication required, cross-site request)
404	Not found or not owned
409	Conflict (duplicate e-mail, SKU, barcode, alias; existing subscription)
410	Dynamic QR paused, disabled or expired (HTML page)
413 / 415	Body too large / unsupported or mismatched file type
429	Rate limit exceeded
500 / 502 / 503	Unexpected error / payment provider error / payments or webhook not configured

Example. Creating a product:

```
POST /api/products
Cookie: qs_session=<signed token>
Origin: https://app.example.com
Content-Type: application/json

{"name":"Masala Chai 250g","sku":"CHAI-250","price":"149.00","stock":10,
 "barcodes":[{"format":"ean13","value":"2001234567893","source":"internal"}]}

201 Created
{"item":{"id":"…","sku":"CHAI-250","price":"149.00","priceMinor":14900,"stock":10,
 "lowStock":false,"barcodes":[{"format":"ean13","value":"2001234567893","isPrimary":true,…}],…}}
```

4.17 Security Design

The controls below map to common guidance such as the OWASP Cheat Sheet Series [16], OWASP Top 10 [17] and NIST SP 800-63B [18].

Table 4.11: Security controls present in the implementation

Concern	Control	Location
Password storage	bcrypt, cost 12; dummy hash for unknown accounts	`src/lib/auth-helpers.ts`
Sessions	Server-side `Session` rows referenced by HS256-signed JWT [15], HttpOnly cookies; new id on every login; `sessionVersion` invalidates all tokens on password change, suspension or role change	`src/lib/session.ts`
Admin separation	Separate `qs_admin` cookie (SameSite=Strict, 12 h, 2 h idle); workspace session never grants admin access	`src/lib/session.ts`
MFA	RFC 6238 TOTP [12], [13], ±1 step, replay guard via `mfaLastStep`; 10 single-use recovery codes stored as SHA-256 hashes; secrets AES-256-GCM encrypted	`src/lib/mfa.ts`
Re-authentication	Payment credentials require password (+TOTP) within 15 minutes	`requireRecentAdminAuth()`
Authorization	Deny-by-default guards; ownership checks; admin permission rules; last-super-admin protection	`session.ts`, `guard.ts`, `admin-users.ts`
CSRF	Origin check for non-GET API requests; SameSite cookies	`src/middleware.ts`
Brute force / abuse	Fixed-window rate limits per IP, per account and global; scan recording cap	`src/lib/rate-limit.ts`
Input validation	zod schemas; body-size limits; type-specific validation for QR content and barcodes	`src/lib/api.ts`, `qr-types.ts`, `products.ts`
SQL injection	All queries through Prisma's parameterised API; no raw SQL in application code	`src/lib/**`

Concern	Control	Location
XSS	React escaping; branding/settings validation rejects markup; JSON-LD escaping; CSP	`branding.ts`, `next.config.mjs`
Uploads	Extension allow-list, magic-byte check, random names, stored outside `public/`, served with `nosniff`, attachment disposition for Office files and a sandboxing CSP	`storage.ts`, `app/f/[name]/route.ts`
Branding images	Header parsing for PNG/JPEG/WebP dimensions; SVG reject-list for scripts and external references	`src/lib/image-check.ts`
Open redirects	`safeNextPath()` allows only same-origin relative paths	`src/lib/safe-redirect.ts`
Secrets	Server-only environment variables; startup check rejects secrets in `NEXT_PUBLIC_*`; payment credentials sealed with a key not stored in the database; status APIs never return secrets	`env-check.ts`, `secret-box.ts`, `payment-config.ts`
Payments	HMAC-SHA256 checkout and webhook signatures with constant-time comparison; amounts from the database; API re-fetch; idempotent ledger	`razorpay.ts`, `billing.ts`
Privacy	Salted, truncated IP hashes; no raw IPs; DNT/GPC respected; log redaction of e-mails and tokens	`audit.ts`, `scans.ts`, `redact.ts`
Headers	CSP [19], X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, COOP, HSTS (HTTPS)	`next.config.mjs`
Fail-closed configuration	Production server exits on unsafe configuration	`src/instrumentation.ts`

Table 4.12: Security gaps identified by the audit

ID	Gap	Severity
S-01	A SQLite backup containing user records and bcrypt hashes is present in Git history (commit `21277e0`)	High (if the repository is or becomes public)
S-02	Sandboxing CSP for `/f/*` and `/brand/*` was overridden by the global CSP in the first audited build; a fix exists in the working tree and was re-tested (Chapter 5)	Medium
S-03	E-mail addresses are not verified at sign-up	Low–Medium
S-04	MFA mandatory only for super administrators	Medium
S-05	CSP allows `'unsafe-inline'` scripts	Low–Medium
S-06	No breached-password check	Low
S-07	In-memory rate limiting by default; IP-based limits are spoofable without `TRUST_PROXY`	Medium (deployment-dependent)
S-08	Dynamic QR destinations are not checked against URL reputation services	Medium
S-09	Uploaded files are publicly readable by anyone who knows the random 128-bit name (by design, for File QR codes)	Low
S-10	Dependency advisories accepted until 2027-01-31 (postcss, deepmerge-ts)	Low–Medium

4.18 Design Decisions and Trade-offs

Table 4.13: Design decisions

Decision	Rationale	Trade-off / alternative
Next.js full-stack monolith	One language, one deployment, server rendering and API together	Less independent scaling than separate services
TypeScript + zod shared schemas	Same validation on client and server; compile-time safety	Build step; schema code duplication avoided but schemas grow large (`qr-types.ts` ≈ 800 lines)
Prisma ORM	Type-safe queries, migrations, multi-database support	Some database features (native enums, JSON queries) unused to stay portable
SQLite for development, MySQL for production	Zero-setup local development; robust production database	Two providers must be kept compatible; solved by a generated schema and a CI check
Server-side sessions instead of stateless JWT	Immediate revocation on logout, password change, suspension	One database read per request (mitigated by per-request caching)

Decision	Rationale	Trade-off / alternative
Client-side rendering and decoding of codes	No server cost; privacy for scanned images; works with any data	Depends on browser capabilities (WebAssembly, camera)
JSON text columns for QR content/design and plan limits	Each QR type has different fields; plans are editable without migrations	Not queryable by SQL; validated in the application
Snapshots in payments/subscriptions	History remains correct after plan edits	Denormalised data
Local-disk uploads	Simple, no external dependency	Not suitable for multiple servers without shared storage
Fixed-window rate limiter with memory/database stores	Simple and dependency-free	Bursts at window edges; memory store per process
Razorpay Subscriptions	Indian payment methods and recurring billing	Provider lock-in; only one provider implemented

CHAPTER 5
System Testing

5.1 Introduction

This chapter reports how QRSathi was tested for this report. A strict distinction is kept between **executed** tests — commands that were actually run during the audit, with their real output — and **proposed** tests, which are documented procedures marked "Not executed". No result in this chapter is estimated or assumed.

The repository contains its own automated suites (`scripts/test-*.mts`). The audit executed all of them; added a small number of probe checks for scenarios the suites do not assert; and recorded every command, environment detail, pass/fail count and failure explanation.

5.2 Testing Objectives

1. Confirm that the code compiles, type-checks and satisfies the lint rules.
2. Confirm that core logic (QR payloads, barcode validation, security utilities) behaves correctly in isolation.
3. Confirm that the HTTP API enforces authentication, authorization, validation and business rules.
4. Confirm security properties: session handling, RBAC, MFA, CSRF, rate limiting, secret handling, webhook verification.
5. Confirm database portability between SQLite and MySQL.
6. Confirm that the user interface renders without layout overflow, CSP violations or missing accessible names, and that the scanner decodes real codes.
7. Identify and record defects.

5.3 Testing Strategy

The strategy follows the test pyramid:

- **Static analysis** — TypeScript (`tsc --noEmit`) and ESLint.
- **Unit tests** — pure functions tested without a server (`npm test`).
- **Integration/API tests** — real HTTP requests against a production build with a throw-away database (`test:api`, `test:admin`, `test:billing`).
- **Database tests** — migrations and data migration on a real MySQL server (`test:mysql`).
- **End-to-end browser tests** — headless Chromium driven through the DevTools protocol (`test:browser`).
- **Audit probes** — short scripts written by the auditor for gaps (Section 5.16), run against the same isolated server.

Black-box techniques used by the suites include equivalence partitioning (valid/invalid inputs), boundary-value analysis (stock exactly zero, sixth login attempt, 25 visits vs a 20-scan cap, upload just over a limit) and negative/unauthorized-access testing. White-box knowledge was used to target transactions and idempotency (concurrent creates, duplicate webhooks).

5.4 Testing Environment

Table 5.1: Test environment

Item	Value
Operating system	Windows 11 Pro 10.0.26200
Node.js / npm	22.22.3 / 10.9.8
Code under test	Git HEAD `9b5dae7` **plus uncommitted working-tree changes** (115 entries in `git status` at the final run)
Build	`next build` into an isolated directory `.next-audit` (`NEXT_DIST_DIR`), production mode

Item	Value
Server	`next start -p 3100`, `NODE_ENV=production`, freshly generated random secrets, no SMTP, no real payment keys
Database (integration)	Throw-away SQLite file in a temporary folder (never `prisma/dev.db`)
Database (MySQL test)	`mysql:8.4` Docker container on 127.0.0.1:33306, removed after the run
Browser	Microsoft Edge (Chromium), headless, driven by `scripts/test-browser.mts`
Payment provider	Deterministic mock Razorpay server started by `test:billing` (no real payments)
Final run window	2026-10-25 18:41:35 to 18:46:32 IST (integration suites); unit checks re-run afterwards

Important note on the code snapshot. The working tree was being modified while the audit was in progress (for example `next.config.mjs` changed at 18:27:25 and `scripts/test-browser.mts` at 18:34). An earlier run therefore differed from the final run; both are reported in Section 5.16 so that the effect is transparent. All headline numbers in this chapter come from the **final run** on the tree as it stood at 18:41 IST.

5.5 Unit Testing

`npm test` runs three suites without a server:

- `scripts/test-security.mts` — open-redirect guard, log redaction, JSON-LD escaping, start-up configuration validation, TOTP against RFC 6238 test vectors, MFA secret encryption, recovery codes, rate limiting, IP-spoofing rules, CSRF origin rules, RBAC rules, and the MySQL schema generator. **Result: 134 passed, 0 failed.**
- `scripts/test-qr.mts` — every QR content type and design style is rendered and decoded again. **Result: "All QR checks passed"** (this suite prints no assertion count).
- `scripts/test-barcodes.mts` — check digits, invalid-input rejection, generate → render → decode with zxing-wasm for every format, label scannability, multi-code images, lookup-key normalisation and PDF export. **Result: 166 passed, 0 failed.**

5.6 Integration Testing

`test:api` exercises authentication and CSRF, products, barcode lookup, stock movements, isolation between users, edit/archive, import and the existing QR/UPI/dynamic features over HTTP. **Final result: 73 passed, 0 failed.**

`test:billing` starts a mock Razorpay server and tests the payment gateway configuration, re-authentication, pricing administration, server-side pricing at checkout, webhooks (signature, ledger, duplicates, mismatches), renewal, failure, halt, cancellation and refunds, price changes, server-side entitlements, branding and secret leakage. **Final result: 128 passed, 0 failed.** (First run: 126 passed, 2 failed — see D-02.)

5.7 System Testing

System testing exercised the complete production build through its public interfaces: HTTP APIs, public routes (`/r`, `/p`, `/f`, `/brand`) and pages, together with the start-up configuration checks. The smoke checks in `test:api` confirm that the main pages return HTTP 200, that private pages redirect guests to the login page, and that the WebAssembly decoder is served.

5.8 End-to-End Testing

`test:browser` drives a real browser:

1. **Camera scanning** — a generated EAN-13, Code 128, QR code and Data Matrix are fed to the page as a fake webcam video; each must be decoded and classified.
2. **Image upload** — one image containing four codes must produce four results, and a UPI QR must show a payment warning.

3. **Responsive layout** — public and workspace pages at 375, 768 and 1280 px must not scroll horizontally, must produce no CSP violations or console errors, and (at 1280 px) all controls must have accessible names.
4. **Admin console** — an administrator created with the CLI signs in through the real form; every console page is checked the same way.

Final result: 445 passed, 7 failed. All camera, upload, public-page and workspace-page checks passed. The seven failures are all in the admin-console section: six are heading checks that look for the text "Paid accounts" and "Payment records", which no longer exists anywhere in `src/` after the admin console was redesigned (stale test expectations, D-04), and one is a genuine horizontal overflow of `/admin/settings/pricing` at 768 px (scroll width 825 px, D-03).

5.9 Database Testing

- `npx prisma validate` — schema valid.
- `npm run db:mysql:schema -- --check` — the committed MySQL schema matches the canonical schema.
- `npm run test:mysql` against MySQL 8.4 — applies both MySQL migrations to a new database, migrates an edge-case SQLite database with the real migration script, and checks exact round-trips of IDs, foreign keys, millisecond timestamps, NULLs, booleans, Unicode, 1,500-character barcodes and money values; case-sensitive short codes; case-insensitive search; and that the migration refuses non-empty targets and writes nothing when pre-flight checks fail. **Result: 32 passed, 0 failed (74 s).**
- Business-level database behaviour (unique constraints, transactions, cascade rules) is exercised through the API suites (duplicate SKU/barcode → 409, concurrent quota checks, idempotent payments).

5.10 Authentication and Authorization Testing

`test:admin` focuses on identity and access control: public endpoint headers; sign-up never creating administrators; admin APIs denying by default; separate admin sign-in; mandatory MFA enrolment for super administrators; logout and MFA sign-in including replayed and forged codes; user management and RBAC rules; audit logging; settings validation and stored XSS; sessions, password changes and reset tokens; IDOR and uploads; webhook signature and idempotency; rate limiting; and a scan of the client bundle for secrets. **Final result: 99 passed, 0 failed.**

5.11 Security Testing

Security testing in this audit consisted of (a) the automated security assertions above, (b) source-code review of authentication, authorization, validation, upload handling, payment verification and secret handling (Section 4.17 and `QRSathi_Project_Audit.md`), (c) dependency checks (`npm run audit:check`: passed, with five advisories explicitly accepted until 2027-01-31) and (d) the probe scripts in Section 5.16. **No formal penetration test was performed**, and the real Google, SMTP and Razorpay integrations were not exercised.

5.12 Usability and Responsive Testing

Responsive behaviour was tested automatically at three widths (Section 5.8). Visual inspection of the captured screenshots (Appendix G) was performed by the auditor. No usability study with real users, no task-completion measurements and no manual screen-reader testing were performed.

5.13 Performance Testing

Not executed. No load-testing tool or production-like environment was available, and the repository contains no performance tests. The only timing data collected are incidental: the production build took 30–67 seconds and the server started in under 0.5 seconds on the audit machine. These are not performance measurements of the application under load. A proposed procedure is listed as TC-SYS-08.

5.14 Regression Testing

Re-running the suites after the working tree changed acted as regression testing. Two failures in the first billing run (D-02) disappeared in the final run after a configuration change made concurrently by the developer, while defect D-01 was confirmed to persist on the final build. The CI workflow (`.github/workflows/ci.yml`) is designed to run these suites on every push and pull request; CI execution on GitHub was **not** observed in this audit.

5.15 Compatibility Testing

Only Chromium (Edge) was tested automatically. Firefox, Safari and real mobile devices were not tested (TC-SYS-09, not executed). Database compatibility (SQLite ↔ MySQL 8.4) was tested (Section 5.9).

5.16 Test Cases and Test Results

Table 5.2: Commands executed and results (final run unless stated)

#	Command	Result	Notes
1	`npm run typecheck`	Pass (0 errors)	9 s
2	`npm run lint`	Pass (0 errors, 5 warnings)	warnings: unused eslint-disable directives, a missing hook dependency in `scanner.tsx`, an unused expression in `products-list.tsx`, anonymous default export in `postcss.config.mjs`
3	`npm test`	Pass: 134 + QR suite + 166	security, QR, barcodes
4	`npx prisma validate`	Pass	—
5	`npm run db:mysql:schema -- --check`	Pass	schemas in sync
6	`npm run audit:check`	Pass	5 advisories accepted until 2027-01-31
7	`npm run license:check`	Pass	licence report produced
8	`next build` (isolated)	Pass	30 s final; 67 s first build
9	`npm run test:api`	73 passed, 0 failed	—
10	`npm run test:admin`	99 passed, 0 failed	first attempt aborted by a broken server log pipe in the audit harness; re-run passed
11	`npm run test:billing`	128 passed, 0 failed	first run 126/2 (D-02)
12	`npm run test:browser`	445 passed, 7 failed	D-03, D-04; earlier runs also had environment-related connection failures
13	`npm run test:mysql`	32 passed, 0 failed	MySQL 8.4 container
14	Audit probe: upload sizes	5 MB ✓, 9.5 MB ✓, 12 MB ✗ (Pro plan)	D-01
15	Audit probe: gap cases	7 of 7 passed	TC-AUTH-03/04, TC-QR-06, TC-DYN-03, TC-UPL-04

In total, **745 integration and end-to-end assertions passed** in the final runs (73 + 99 + 128 + 445), plus 300 counted unit assertions (134 + 166), the uncounted QR suite and 32 MySQL assertions, against **7 failures**, all in the browser suite.

Table 5.3: Test-case summary (full detail in Appendix D)

ID	Module	Scenario	Type	Status
TC-AUTH-01	Auth	Sign-up creates an ordinary user even if a role is injected	positive / security	Pass
TC-AUTH-02	Auth	Sign-up response and session cookie are safe	security	Pass
TC-AUTH-03	Auth	Duplicate e-mail rejected at sign-up	negative / duplicate	Pass
TC-AUTH-04	Auth	Weak password rejected	negative / boundary	Pass
TC-AUTH-05	Auth	Private API requires sign-in	unauthorized	Pass

ID	Module	Scenario	Type	Status
TC-AUTH-06	Auth	Cross-site state-changing request blocked	security / CSRF	Pass
TC-AUTH-07	Auth	Generic error for wrong password, unknown e-mail and non-admin	negative / enumeration	Pass
TC-AUTH-08	Auth	Session fixation prevented	security	Pass
TC-AUTH-09	Auth	Logout invalidates token server-side	expired / revoked session	Pass
TC-AUTH-10	Auth	Suspended user cannot sign in; sessions end immediately	role restriction	Pass
TC-AUTH-11	Auth	Forgot-password does not reveal accounts	security	Pass
TC-AUTH-12	Auth	Reset token works once and is stored hashed	positive / negative	Pass
TC-AUTH-13	Auth	Expired reset token rejected	boundary / expired	Pass
TC-AUTH-14	Auth	Reset signs out existing sessions	security	Pass
TC-AUTH-15	Account	Password change signs out other devices but keeps current	positive	Pass
TC-AUTH-16	Auth	Brute-force limit on admin sign-in	boundary / abuse	Pass
TC-AUTH-17	Auth	Google sign-in (OAuth) end-to-end	integration	Not executed
TC-QR-01	QR	Every QR type and style renders and decodes	positive	Pass
TC-QR-02	QR	Generator page loads	smoke	Pass
TC-QR-03	QR	Generator page fits mobile/tablet/desktop with no CSP errors	usability	Pass
TC-QR-04	QR	User B cannot read or delete A's QR code	unauthorized / IDOR	Pass
TC-QR-05	QR	Dynamic-QR quota enforced under concurrency	boundary / concurrency	Pass
TC-QR-06	QR	Custom alias collision / reserved word	negative	Pass
TC-QR-07	UPI	UPI QR saved with correct URI	positive	Pass
TC-QR-08	UPI	Invalid UPI ID rejected	negative	Pass
TC-DYN-01	Dynamic QR	Dynamic QR created	positive	Pass
TC-DYN-02	Dynamic QR	Dynamic QR redirects	positive	Pass
TC-DYN-03	Dynamic QR	Paused / expired code shows 410 page	negative / boundary	Pass
TC-DYN-04	Analytics	Scan recording capped per client	boundary / abuse	Pass
TC-BC-01	Barcodes	All 23 formats render and decode with the app's scanner	positive	Pass
TC-BC-02	Barcodes	Wrong EAN-13 check digit rejected	negative	Pass
TC-BC-03	Barcodes	Internal GTIN outside restricted range rejected	negative / business rule	Pass
TC-BC-04	Barcodes	Internal EAN-13 generated in restricted range	positive	Pass
TC-SCN-01	Scanner	Camera scan of EAN-13, Code 128, QR and Data Matrix	positive	Pass
TC-SCN-02	Scanner	Image with several codes decoded	positive	Pass
TC-SCN-03	Scanner	UPI QR shows payment warning	security / usability	Pass
TC-PRD-01	Products	Create product with barcodes; price in paise	positive	Pass
TC-PRD-02	Products	Duplicate SKU rejected	duplicate	Pass
TC-PRD-03	Products	Duplicate barcode rejected (incl. UPC-A/EAN-13 equivalence)	duplicate	Pass
TC-PRD-04	Products	Missing name / invalid price rejected	negative	Pass
TC-PRD-05	Products	Other user cannot read, edit or move stock	unauthorized / IDOR	Pass
TC-PRD-06	Products	Same SKU allowed for a different user	positive / tenancy	Pass
TC-PRD-07	Products	Partial edit keeps untouched fields	positive	Pass

ID	Module	Scenario	Type	Status
TC-PRD-08	Products	Delete requires archive first	business rule	Pass
TC-PRD-09	Products	Barcode lookup is read-only and finds equivalents	positive	Pass
TC-PRD-10	Import	CSV/XLSX import with per-row report	positive / negative	Pass
TC-PRD-11	Import	Import update mode logs stock as adjustment	positive	Pass
TC-INV-01	Inventory	Stock in increases stock	positive	Pass
TC-INV-02	Inventory	Stock out decreases stock	positive	Pass
TC-INV-03	Inventory	Stock out beyond available rejected	boundary	Pass
TC-INV-04	Inventory	Zero quantity rejected; adjust sets value and flags low stock	boundary	Pass
TC-INV-05	Inventory	History records every movement with balances; archived blocks movement	positive / negative	Pass
TC-UPL-01	Uploads	Oversized upload rejected before parsing (Free plan)	boundary	Pass
TC-UPL-02	Uploads	Paid-plan upload between 10 MB and plan limit	boundary / defect	Fail
TC-UPL-03	Uploads	Uploaded files served with sandboxing CSP	security / regression	Pass
TC-UPL-04	Uploads	File whose content does not match its extension	negative	Pass
TC-BIL-01	Billing	Forged webhook signature rejected	security / signature	Pass
TC-BIL-02	Billing	Underpaid payment does not upgrade	negative	Pass
TC-BIL-03	Billing	Duplicate webhook deliveries applied once	idempotency	Pass
TC-BIL-04	Billing	Subscription checkout lifecycle against mock Razorpay	integration (mock)	Pass
TC-BIL-05	Entitlements	Plan features enforced on server (alias, analytics, export)	role / plan restriction	Pass
TC-BIL-06	Billing	Live Razorpay payment in test mode	integration	Not executed
TC-ADM-01	Admin	All admin APIs deny without admin session	unauthorized	Pass
TC-ADM-02	Admin	User token replayed as admin cookie rejected	security	Pass
TC-ADM-03	Admin	Admin cookie flags	security	Pass
TC-ADM-04	Admin	Super admin forced to enrol MFA	role restriction	Pass
TC-ADM-05	Admin	MFA enrolment stores encrypted secret and hashed recovery codes	security	Pass
TC-ADM-06	Admin	Wrong, replayed or forged MFA input rejected	negative	Pass
TC-ADM-07	Admin	Recovery code single use	boundary	Pass
TC-ADM-08	Admin	Admin CLI refuses weak or command-line passwords	negative	Pass
TC-ADM-09	Admin	Admin cannot change roles, touch super admin or delete	role restriction	Pass
TC-ADM-10	Admin	Admin changes ordinary user's plan; unknown fields ignored	positive	Pass
TC-ADM-11	Admin	Super admin promotes/demotes; cannot change own role	role restriction	Pass
TC-ADM-12	Admin	Bulk action skips own account	business rule	Pass
TC-ADM-13	Admin	Admin triggers reset e-mail without seeing token	security	Pass
TC-ADM-14	Admin	User search returns no secrets; pagination	security	Pass
TC-ADM-15	Settings	Settings validation rejects invalid e-mail/colour	negative	Pass
TC-ADM-16	Settings	Stored XSS via brand name / announcement rejected	security / XSS	Pass
TC-ADM-17	Branding	Branding publish, assets and SVG sandboxing	security	Pass
TC-ADM-18	Audit	Audit log records admin actions without secrets	security	Pass
TC-ADM-19	Admin UI	Admin sign-in through real form and console pages render	usability / regression	Fail
TC-SYS-01	Platform	Health endpoint reveals only status	security	Pass
TC-SYS-02	Platform	Security headers present; no X-Powered-By	security	Pass
TC-SYS-03	Platform	No secrets in client bundle	security	Pass
TC-SYS-04	Platform	Unsafe production configuration refused	security	Pass
TC-SYS-05	Database	MySQL migrations and SQLite→MySQL data migration	database	Pass
TC-SYS-06	Database	MySQL schema in sync with canonical schema	database	Pass
TC-SYS-07	Responsive	Public and workspace pages fit 375/768/1280 px without CSP or console errors	usability / accessibility	Pass

ID	Module	Scenario	Type	Status
TC-SYS-08	Performance	Redirect latency and page load under load	performance	Not executed
TC-SYS-09	Compatibility	Firefox and Safari	compatibility	Not executed
TC-SYS-10	Database	Database unavailable	negative / database error	Not executed

5.17 Defect Log

Table 5.4: Defect log

ID	Severity	Module	Description	Evidence	Status
D-01	Medium	Uploads	Uploads larger than 10 MB fail with "No file received." even when the user's plan allows 25 MB (Pro) or 50 MB (Business). The middleware matches `/api/:path*`, and Next.js buffers at most 10 MB of a request body that passes through middleware, so the multipart body is truncated.	Probe: 12 MB PDF as Pro user → 400, on first and final builds; server log "Request body exceeded 10MB for /api/uploads"	Open
D-02	Medium (security)	Headers	The sandboxing CSP set by `/f/[name]` and `/brand/[name]` was replaced by the global CSP from `next.config.mjs`, so uploaded SVG/PDF files did not get the `sandbox` directive.	test:billing first run: 2 failures	Fixed in working tree (new header rules in `next.config.mjs`); final run passed. Not yet committed
D-03	Low	Admin UI	`/admin/settings/pricing` scrolls horizontally at 768 px (scroll width 825 px).	test:browser final run	Open
D-04	Low (test)	Test suite	Browser test expects headings "Paid accounts" and "Payment records" that no longer exist in the redesigned admin pages.	test:browser final run (6 failures); `grep` finds the strings only in the test	Open
D-05	Low	Admin API	`PATCH /api/admin/users/[id]` accepts only the plan ids `free`, `pro`, `business`, although plans are now administrator-defined in the database; custom plans cannot be assigned manually.	Source review (`z.enum(["free","pro","business"])`)	Open
D-06	Low	Code quality	5 ESLint warnings, including a missing `useEffect` dependency in the scanner.	`npm run lint`	Open
D-07	High (repository)	Security	A SQLite backup with user records and bcrypt hashes is present in Git history (commit `21277e0`).	`git log --all --name-only`	Open (documented by the developer in `docs/OPEN-SOURCE-READINESS.md`)
D-08	Info	Configuration	The local `.env` still defines the obsolete `ADMIN_EMAILS`, which produces a start-up warning.	Variable names in `.env`; `env-check.ts`	Open

5.18 Test Coverage and Limitations

- No code-coverage tool is configured, so **line or branch coverage is unknown**. Coverage is described functionally by the requirement traceability matrix: of 22 functional requirements, 17 are verified by executed tests, 4 are partially verified and 1 (moderation, FR-19) is implemented but not independently verified.
- External integrations (Google OAuth, SMTP delivery, live Razorpay) were not tested.
- Only Chromium was tested; no real mobile devices.
- No performance, load, soak or penetration testing.
- The code under test included uncommitted changes that were still being edited; the results describe the snapshot at 18:41 IST on 2026-10-25.
- Browser runs before the final one were affected by intermittent `ERR_CONNECTION_REFUSED` errors whose cause was not determined (the final run had none); those runs are not counted.

5.19 Testing Summary

The project has an unusually complete automated test set for a mini project, and it was executed end-to-end in an isolated environment. All type, lint, unit, API, administration/security, billing and MySQL tests passed in the final run. The browser suite passed 445 checks, with seven admin-console failures traced to one minor layout defect and stale test expectations. Independent probes confirmed seven additional behaviours and uncovered one functional defect (D-01, large uploads on paid plans). Of the 91 documented test cases, 84 passed, 2 failed and 5 were not executed because they require external services, other browsers or a load-testing environment.

CHAPTER 6
Implementation and Evaluation

6.1 Introduction

This chapter explains how the design of Chapter 4 is realised in code. It describes the development set-up, the folder structure and the implementation of each module, with short excerpts from the real source files (file paths are given for each excerpt; excerpts are abridged only where marked). It then gives the installation, execution and deployment procedures, presents screenshots captured from the running application, and evaluates the implementation against the objectives.

The audited code base contains about 20,200 lines of TypeScript/TSX in `src/` (57 pages, 67 route handlers — 64 under `/api` — and 31 component files), 22 database models, and about 2,700 lines of test and tooling scripts.

6.2 Development Environment Setup

1. Install Node.js 22 (the repository's `.nvmrc` pins major version 22; `package.json` requires `>=22.5.0`) and npm 10.
2. Clone the repository and run `npm install`. The `postinstall` script generates the Prisma client (`scripts/db/prisma.mjs generate`) and copies the zxing WebAssembly file into `public/zxing/`.
3. Copy `.env.example` to `.env` and set at least `DATABASE_URL="file:./dev.db"`, a random `AUTH_SECRET` of 32+ characters and `NEXT_PUBLIC_APP_URL="http://localhost:3000"`.
4. Create the development database with `npm run db:push`.
5. Create the first administrator with `npm run admin -- create --email you@example.com --super` (the password is read from standard input, never from the command line).
6. Start the development server with `npm run dev` and open `http://localhost:3000`.

Optional: `npm run seed:demo` loads demo data when `DEMO_MODE=true` (development only).

6.3 Installation and Configuration

Configuration is entirely through environment variables (Table 3.6). The application validates them at start-up (`src/lib/env-check.ts`); in production any error stops the server, and warnings are logged. Secrets are never committed: `.gitignore` excludes `.env*` (except `.env.example`), databases, backups, uploads and logs. Optional integrations are enabled simply by setting their variables: Google sign-in (`GOOGLE_CLIENT_ID/SECRET`), SMTP (`SMTP_*`, `MAIL_FROM`) and Razorpay (environment variables, or encrypted database storage configured from the admin console when `SETTINGS_ENCRYPTION_KEY` is set).

6.4 Project Folder Structure

```
qrsathi/
├── .github/            CI workflow, Dependabot, issue/PR templates
├── docs/               ADMIN, BACKUP-RECOVERY, BARCODES, DATABASE, DEPLOYMENT,
│                       LICENSING, OPEN-SOURCE-READINESS, SECURITY-AUDIT
├── prisma/
│   ├── schema.prisma   canonical schema (SQLite provider)
│   ├── mysql/          generated MySQL schema + migrations
│   └── seed-demo.mjs   development demo data
├── public/             static assets (zxing WASM copied at install)
├── scripts/            admin CLI, DB tools, test suites, audit/licence checks
├── security/           dependency-advisory allowlist
├── src/
│   ├── app/
│   │   ├── (site)/     public marketing pages
│   │   ├── (auth)/     login, signup, forgot/reset password
│   │   ├── (app)/      signed-in workspace (generator, products, …)
│   │   ├── admin/      admin console: (auth) sign-in, (console) pages
│   │   ├── api/        64 API route modules
│   │   ├── r/ p/ f/ brand/   public redirect, hosted page, files, brand images
│   │   └── layout.tsx, globals.css, robots.ts, sitemap.ts
```

```
│   ├── components/     UI components (qr/, barcode/, admin/, pricing/, …)
│   ├── lib/            domain and infrastructure modules (58 files)
│   ├── middleware.ts   CSRF origin check
│   └── instrumentation.ts  start-up configuration validation
├── package.json, next.config.mjs, tailwind.config.ts, tsconfig.json
└── eslint.config.mjs, postcss.config.mjs
```

The full tree is in Appendix A. Route groups in parentheses (for example `(app)`) organise layouts without appearing in URLs.

6.5 Frontend Implementation

The frontend uses the Next.js App Router. Layouts provide three shells: the public site (`src/app/(site)/layout.tsx`), the workspace with sidebar and command palette (`src/components/app-shell.tsx`), and the admin console (`src/components/admin/admin-shell.tsx`). Pages that need data on first render are server components; interactive tools are client components (`"use client"`).

The QR workbench (`src/components/qr/workbench.tsx`) composes four parts: the type selector, a **generic form renderer** (`type-form.tsx`) that draws fields from the type definition in `qr-types.ts`, the design panel, and the preview/export panel. Because each QR type declares its own fields and zod schema, adding a type requires no new form code. Styling uses Tailwind utility classes; brand colours are CSS variables so that administrators can re-brand the site without rebuilding. Charts use Recharts and notifications use Sonner.

6.6 Backend Implementation

Every API endpoint is a route handler wrapped in `handle()`, which centralises error handling. A typical handler authenticates, rate-limits, validates and delegates:

```
// src/app/api/qrcodes/route.ts (POST handler)
export const POST = handle(async (req) => {
  const user = await requireUser();
  await rateLimit(`qr-create:${user.id}`, 300, 60 * 60 * 1000);
  const input = await readJson(req, createSchema);
  const qr = await createQr(user, input, baseUrl(req));
  await logEvent("info", "qr.created", `${qr.type}${qr.isDynamic ? " (dynamic)" : ""} ${qr.shortCode ?? ""}`,
user.id);
  return json({ item: serializeQr(qr) }, 201);
});
```

`handle()` converts domain errors to HTTP responses and hides unexpected errors from clients:

```
// src/lib/api.ts
export function handle<C = unknown>(fn: Handler<C>): Handler<C> {
  return async (req, ctx) => {
    try {
      return await fn(req, ctx);
    } catch (e) {
      if (e instanceof ApiError) return json({ error: e.message, details: e.details }, e.status);
      const msg = e instanceof Error ? e.message : String(e);
      await logEvent("error", "api.unhandled", `${req.method} ${new URL(req.url).pathname}: ${msg}`.slice(0, 1000));
      return json({ error: "Something went wrong. Please try again." }, 500);
    }
  };
}
```

6.7 Database Implementation

Prisma Client is created once per process (`src/lib/db.ts`) and reused (in development it is cached on `globalThis` to survive hot reloads). The canonical schema `prisma/schema.prisma` targets SQLite. `npm run db:mysql:schema` generates `prisma/mysql/schema.prisma`, adding MySQL column types; migrations for MySQL live in `prisma/mysql/migrations` and are applied with `npm run db:mysql:deploy`. `npm run db:migrate-data` copies an existing SQLite database into an empty MySQL database with pre-flight checks (orphaned foreign keys, case-only duplicate e-mails) and verification. Both paths were exercised by `test:mysql` (32/32 passed).

Transactions are used wherever several rows must change together: quota checks with inserts (`withQuota`), stock movements, product creation with barcodes and ledger entry, payment application and token claims.

6.8 Authentication Implementation

Sessions are stored in the database and referenced by a signed cookie. A request is authenticated only if the token is valid **and** the session row and user still match:

```
// src/lib/session.ts — load() (abridged)
const session = await db.session.findUnique({ where: { id: p.sid }, include: { user: true } });
const now = new Date();
if (!session || session.userId !== p.sub || session.kind !== kind || session.expiresAt < now) return null;
const user = session.user;
if (user.status !== "active" || user.sessionVersion !== p.sv) return null;
if (kind === "admin") {
  if (!isAdminRole(user.role)) return null;
  if (now.getTime() - session.lastSeenAt.getTime() > ADMIN_IDLE * 1000) { /* delete and */ return null; }
  // MFA-enabled administrators must have verified MFA in this session.
  if (user.mfaEnabledAt && !session.mfa) return null;
}
```

Incrementing `sessionVersion` (on password change, reset, suspension or role change) therefore invalidates every token at once. Login is protected by three rate limits and a dummy bcrypt comparison for unknown accounts (`src/lib/auth-flow.ts`). The TOTP verifier accepts one step of clock drift and returns the matched step so the caller can reject replays:

```
// src/lib/mfa.ts
export function verifyTotp(secret: string, code: string, lastStep = 0, now = Date.now()): number | null {
  const c = code.replace(/\s/g, "");
  if (!/^\d{6}$/.test(c)) return null;
  const s = currentStep(now);
  for (const step of [s - 1, s, s + 1]) {
    if (step <= lastStep) continue;
    const expected = Buffer.from(totpAt(secret, step));
    if (timingSafeEqual(expected, Buffer.from(c))) return step;
  }
  return null;
}
```

The admin MFA route then advances `mfaLastStep` with a conditional `updateMany(... mfaLastStep: { lt: step })`, so two concurrent requests cannot both use the same code.

CSRF protection is implemented in the middleware by comparing the `Origin` header of state-changing API requests with the configured application origin:

```
// src/middleware.ts (abridged)
if (["GET", "HEAD", "OPTIONS"].includes(method)) return NextResponse.next();
if (req.nextUrl.pathname.startsWith("/api/billing/webhook")) return NextResponse.next();
const origin = req.headers.get("origin");
if (!origin) return NextResponse.json({ error: "Missing origin." }, { status: 403 });
if (!allowedOrigins(req).has(new URL(origin).origin))
  return NextResponse.json({ error: "Cross-site request blocked." }, { status: 403 });
```

6.9 QR Code Module Implementation

`src/lib/qr-types.ts` is a registry of 27 types. Each entry declares fields, defaults, a zod schema and a `build()` function that produces the encoded text:

```
// src/lib/qr-types.ts (first entry)
{
  id: "url", label: "Website URL", description: "Open any web page", category: "basic", icon: "Globe",
  fields: [{ name: "url", label: "Website URL", kind: "url", placeholder: "https://yourwebsite.com", required: true
}],
  defaults: { url: "https://example.com" },
  schema: z.object({ url: url("Website URL") }),
  build: (d) => d.url,
  dynamicCapable: true, logo: "link", caption: "SCAN ME",
},
```

The 27 types are: Website URL, Dynamic URL, Text, vCard, Phone, SMS, E-mail, Wi-Fi, WhatsApp, Instagram, Facebook, YouTube, LinkedIn, Maps, UPI, Bank (account + IFSC as UPI), Product, File, Event, App, Links (link-in-bio), Menu, Business, Feedback, Coupon and Landing page. Some types (for example menus and link lists) are **hosted**: the QR code points to `/p/{code}`, where the content is rendered as a mobile page.

`src/lib/qr-render.ts` turns a payload and a design into SVG using the `qrcode` library's module matrix. The design model supports foreground/background colours, transparency, linear/radial gradients, module styles, finder-eye shapes (square, rounded, circle, leaf), a centre logo, six frame styles with captions, background patterns, error-correction level and quiet-zone margin. When a logo is present, error correction is raised to H automatically:

```
// src/lib/qr-render.ts
export function effectiveEcc(design: QrDesign): Ecc {
  if (design.logo && ECC_ORDER.indexOf(design.ecc) < 3) return "H";
  return design.ecc;
}
```

`designWarnings()` warns about low contrast or oversized logos, and the export panel performs a **scan test**: the exported image is decoded again with `jsqr` and compared with the expected payload ("Scan test passed — decodes to your exact content", visible in Figure 6.3). Exports are produced in the browser: PNG/JPG via canvas, SVG directly, PDF via jsPDF and ZIP via JSZip (`src/lib/qr-export.ts`).

6.10 Barcode Module Implementation

`src/lib/barcodes.ts` defines 23 formats in four groups: retail (EAN-13, EAN-8, UPC-A, UPC-E, ISBN, ISSN), general (Code 128, Code 39, Code 93, Codabar), logistics (GS1-128, ITF, ITF-14, Standard 2 of 5, MSI) and 2D (Data Matrix, GS1 DataMatrix, PDF417, Aztec, Micro QR, GS1 QR). Each format has a validator; rendering uses bwip-js. GS1 check digits use the standard weighted modulo-10 algorithm [9], [10]; rendering uses bwip-js [25] and decoding zxing-wasm [26]:

```
// src/lib/barcodes.ts
export function gs1CheckDigit(body: string): number {
  let sum = 0;
  for (let i = body.length - 1, w = 3; i >= 0; i--, w = w === 3 ? 1 : 3) sum += Number(body[i]) * w;
  return (10 - (sum % 10)) % 10;
}
```

To match scanned codes to products, every barcode also stores a **lookup key**. GTIN-family numbers are normalised to 14 digits, so a UPC-A label scanned as its 13-digit EAN form still matches:

```
// src/lib/barcodes.ts
export function lookupKey(formatId: string, value: string): string {
  const v = value.trim();
  if (formatId === "upce" && /^[01]\d{7}$/.test(v)) return upceToUpca(v).padStart(14, "0");
  if (["ean13", "ean8", "upca", "itf14", "isbn", "issn"].includes(formatId) && /^\d{8,14}$/.test(v)) return
v.padStart(14, "0");
  const gs1 = parseGs1(v);
  const gtin = gs1?.find((p) => p.ai === "01")?.value;
  if (gtin) return gtin;
  return v;
}
```

Internal numbers generated by the system always use the GS1 restricted-circulation prefix 2 (`generateInternalNumber`), and `isRestrictedCirculation` rejects "internal" numbers outside it. Labels are composed as SVG in six sizes from 38 × 25 mm to 100 × 150 mm and four layouts (barcode only, product, shelf, compact).

The scanner (`src/app/(app)/scanner/scanner.tsx`) loads zxing-wasm lazily (`src/lib/zxing-client.ts`), decodes camera frames or uploaded images (several codes per image), classifies the text (`scan-classify.ts`), merges duplicate detections (`scan-merge.ts`) and shows safety notes from `content-safety.ts`. History is kept only in the browser's `localStorage` (`scan-history.ts`) and can be exported to CSV.

6.11 Product and Inventory Implementation

`src/lib/products.ts` contains the product logic. Prices are parsed from strings such as "₹1,499.50" and stored in paise; integer fields reject non-integers and out-of-range values; barcodes are validated, rendered once (to apply encoder rules) and normalised. The stock-out movement uses a conditional update so that stock cannot become negative even under concurrent requests:

```
// src/lib/products.ts — moveStock() (abridged)
} else if (input.type === "out") {
  delta = -input.quantity;
  const res = await tx.product.updateMany({ where: { id, stock: { gte: input.quantity } }, data: { stock: {
decrement: input.quantity } } });
  if (res.count === 0) {
    const cur = await tx.product.findUnique({ where: { id }, select: { stock: true } });
    throw new ApiError(400, `Not enough stock: ${cur?.stock ?? 0} available, ${input.quantity} requested.`);
  }
}
// … every movement then appends an InventoryTransaction with the signed delta and the resulting balance
```

Plan limits are enforced in the same transaction as the insert. `withQuota` first increments the user's `quotaLock` column, which serialises concurrent requests by the same user:

```
// src/lib/entitlements.ts
export async function withQuota<T>(userId: string, fn: (tx: Tx) => Promise<T>): Promise<T> {
  return db.$transaction(async (tx) => {
    await tx.user.update({ where: { id: userId }, data: { quotaLock: { increment: 1 } } });
    return fn(tx);
  }, { maxWait: 10_000, timeout: 20_000 });
}
```

The import endpoint accepts rows parsed in the browser from CSV or XLSX (snake_case headers), supports `skip` and `update` modes, rejects duplicates within the file, and records stock changes from imports as logged adjustments.

6.12 Administrative Functionality

The admin console (`src/app/admin/(console)`) has pages for the platform overview, users, QR codes, abuse reports, products and barcodes, inventory, analytics, subscriptions, payments, security (audit log, administrators, security events), settings (general, branding, appearance, e-mail, pricing, payment gateway), system health and the administrator's own MFA. Permission rules for actions on accounts are centralised in `src/lib/admin-users.ts`: administrators may act only on ordinary users; only super administrators may change roles or delete accounts; nobody may change their own status or role; and the last active super administrator can never be demoted, suspended or deleted. The command-line tool `scripts/admin.mts` creates, promotes, demotes and lists administrators and resets a lost MFA enrolment (`reset-mfa`), and refuses weak passwords or passwords passed as arguments.

6.13 Dynamic QR Implementation

When a dynamic code is created, `createQr()` allocates a short code (seven random characters, retried up to six times, or a validated alias), stores the destination separately from the payload, and sets the payload to `{base}/r/{code}`. The redirect handler (`src/app/r/[code]/route.ts`) checks existence, hosted status, administrator/owner status, pause, expiry and destination, records the scan and returns:

```
// src/app/r/[code]/route.ts
await recordScan(qr, req.headers, clientIp(req));
return new Response(null, {
  status: 302,
  headers: {
    Location: withUtm(qr.destination, qr.utm),
    "Cache-Control": "no-store, max-age=0",
    "Referrer-Policy": "no-referrer",
    "X-Robots-Tag": "noindex",
  },
});
```

`no-store` prevents browsers and proxies from caching the redirect, so an edited destination takes effect immediately. `recordScan()` ignores bots, caps recording at 20 scans per client per code per 10 minutes, and stores only coarse device information and a salted per-day visitor hash — or only a count if the visitor sends Do-Not-Track/GPC or the owner disabled detailed tracking.

6.14 Subscription and Payment Implementation

Billing is implemented with Razorpay Subscriptions (`src/lib/billing.ts`, `src/lib/razorpay.ts`). Super administrators edit plans (`/admin/settings/pricing`); a plan's price is synchronised to an immutable Razorpay plan (`ProviderPlan`). At checkout the client sends only `planId` and `interval`; the server looks up the price. Signatures are verified with HMAC-SHA256 [14] and a constant-time comparison, following Razorpay's documented scheme [21], [22]:

```
// src/lib/razorpay.ts
function safeEqualHex(expected: string, given: string) {
  const a = Buffer.from(expected, "utf8"), b = Buffer.from(given || "", "utf8");
  return a.length === b.length && timingSafeEqual(a, b);
}
/** Webhooks: HMAC_SHA256(raw request body, webhook_secret). */
export function verifyWebhookSignature(rawBody: string, signature: string, webhookSecret: string) {
  return safeEqualHex(createHmac("sha256", webhookSecret).update(rawBody).digest("hex"), signature);
}
```

A plan is upgraded only after the payment is fetched from Razorpay as captured with the expected amount, or after a signed webhook; each webhook event id is recorded in `WebhookEvent` so duplicates are ignored, and payments are unique by provider reference. Renewal, failure, halt, cancellation and refunds update subscription and payment status. Payment credentials can be stored in the database only in sealed form (AES-256-GCM, key from `SETTINGS_ENCRYPTION_KEY`, bound to the field by HKDF "purpose"), require recent re-authentication to change, and are never returned by any API. This module was verified against a mock Razorpay server only.

6.15 Branding and Configuration

Branding (`src/lib/branding.ts`, `src/lib/settings.ts`) supports a draft that can be previewed and then published, a reset to defaults, and assets (logo, dark logo, favicon, social image). Uploaded images are checked by `src/lib/image-check.ts` (magic bytes and dimensions for PNG, JPEG and WebP; a strict reject-list for SVG elements and attributes that could execute script or load external content). Colours are validated as hex values, converted to HSL CSS variables, and checked for contrast; text fields reject markup. General settings include an announcement banner and a switch to close sign-ups.

6.16 Error Handling and Validation

- **Validation:** zod schemas on every input; `readJson()` rejects oversized or malformed bodies; type-specific validators for QR content, UPI identifiers, IFSC codes, barcodes, prices and dates.
- **Errors:** domain code throws `ApiError(status, message)`; `handle()` returns a JSON error; unexpected exceptions return a generic message and are logged with redaction.
- **Logging:** `logger` writes JSON lines in production; `logEvent()` also stores entries in the `Log` table shown in the admin console; `audit()` records security events. Neither throws if the database write fails.
- **User-facing pages:** dynamic-link errors render friendly HTML pages with appropriate status codes (404/410) and a link to report abuse; a custom `not-found.tsx` exists.

6.17 Important Code Walkthroughs

Table 6.1: Code walkthrough index

#	Excerpt	File	What it demonstrates
1	`POST /api/qrcodes`	`src/app/api/qrcodes/route.ts`	Layered handler: authenticate → rate-limit → validate → service → response

#	Excerpt	File	What it demonstrates
2	`handle()`	`src/lib/api.ts`	Central error mapping without leaking internals
3	`load()`	`src/lib/session.ts`	Server-side session validation, idle timeout, MFA flag
4	`verifyTotp()`	`src/lib/mfa.ts`	RFC 6238 verification with drift window and replay protection
5	Origin check	`src/middleware.ts`	CSRF defence
6	`effectiveEcc()`	`src/lib/qr-render.ts`	Automatic error-correction increase for logos
7	`gs1CheckDigit()`, `lookupKey()`	`src/lib/barcodes.ts`	Standards-based validation and GTIN normalisation
8	`moveStock()`	`src/lib/products.ts`	Concurrency-safe stock decrement and ledger
9	`withQuota()`	`src/lib/entitlements.ts`	Serialising quota checks per user
10	Redirect response	`src/app/r/[code]/route.ts`	Non-cacheable redirect with UTM
11	Signature verification	`src/lib/razorpay.ts`	HMAC with constant-time comparison

6.18 Database Initialization and Migration

- **Development (SQLite):** `npm run db:push` creates or updates `prisma/dev.db` from the schema. `npm run db:studio` opens Prisma Studio.
- **Production (MySQL):** create a database and a least-privilege user; set `DATABASE_URL`; run `npm run db:mysql:deploy` (`prisma migrate deploy` with the MySQL schema); then `npm run build`.
- **Schema changes:** edit `prisma/schema.prisma`, run `npm run db:mysql:schema` to regenerate the MySQL schema, create a MySQL migration, and run the CI check `npm run db:mysql:schema -- --check`.
- **Data migration:** stop writes, back up, run `npm run db:migrate-data` (dry run first), switch `DATABASE_URL`, smoke-test (procedure in `docs/DATABASE.md`).
- **Plans:** on first use the default plans (Free, Pro, Business) are inserted only if the `PricingPlan` table is empty (`src/lib/plan-store.ts`).

6.19 Application Execution Procedure

Table 6.3: Execution commands

Purpose	Commands
Development	`npm run dev` → http://localhost:3000
Production build	`npm run build` (copies WASM, generates Prisma client, builds)
Production start	`npm start` (or `next start -p <port>`) behind HTTPS
Administrator	`npm run admin -- create --email <e-mail> --super` (password via stdin)
Checks	`npm run typecheck`, `npm run lint`, `npm test`
Integration tests	build, start against a throw-away DB, then `test:api`, `test:admin`, `test:browser`, `test:billing` (see README)

6.20 Deployment Procedure

Verified during this audit: building and running the production server locally (`next build`, `next start`) against a throw-away SQLite database, and applying the MySQL migrations to a MySQL 8.4 server in Docker.

Recommended (from `docs/DEPLOYMENT.md`; not performed in this audit):

1. Provision a Linux server with Node.js 22 and a MySQL 8 database; create a least-privilege database user.
2. Set production environment variables (strong random `AUTH_SECRET`, `MFA_ENCRYPTION_KEY`, `SETTINGS_ENCRYPTION_KEY`, `NEXT_PUBLIC_APP_URL` with https, `TRUST_PROXY`, `RATE_LIMIT_STORE=database`, SMTP).
3. `npm ci`, `npm run db:mysql:deploy`, `npm run build`.
4. Run `npm start` under a process manager (for example systemd or PM2).
5. Put a reverse proxy (nginx or Caddy) in front for TLS and forward `X-Forwarded-*` headers.

6. Point an uptime monitor at `/api/health`; configure log collection and database/upload backups (`docs/BACKUP-RECOVERY.md`).
7. Create the first super administrator and enrol MFA; configure Razorpay webhooks to `{APP_URL}/api/billing/webhook`.
8. **Before deployment, fix D-01** (upload size) and commit the working tree.

6.21 User Interface Screenshots

All screenshots below were captured automatically from the **actual running production build** by the project's browser test (`SHOT_DIR`) during the final audit run, using throw-away accounts on an empty database. Workspace pages therefore show an empty account, and the admin pages show a test administrator (e-mail `ui-admin-…@example.com`) with the "2FA not enabled" banner that an ordinary admin without MFA sees. No screenshot was edited. The complete set (47 desktop and 47 mobile images) is in `screenshots/` and indexed in Appendix G.

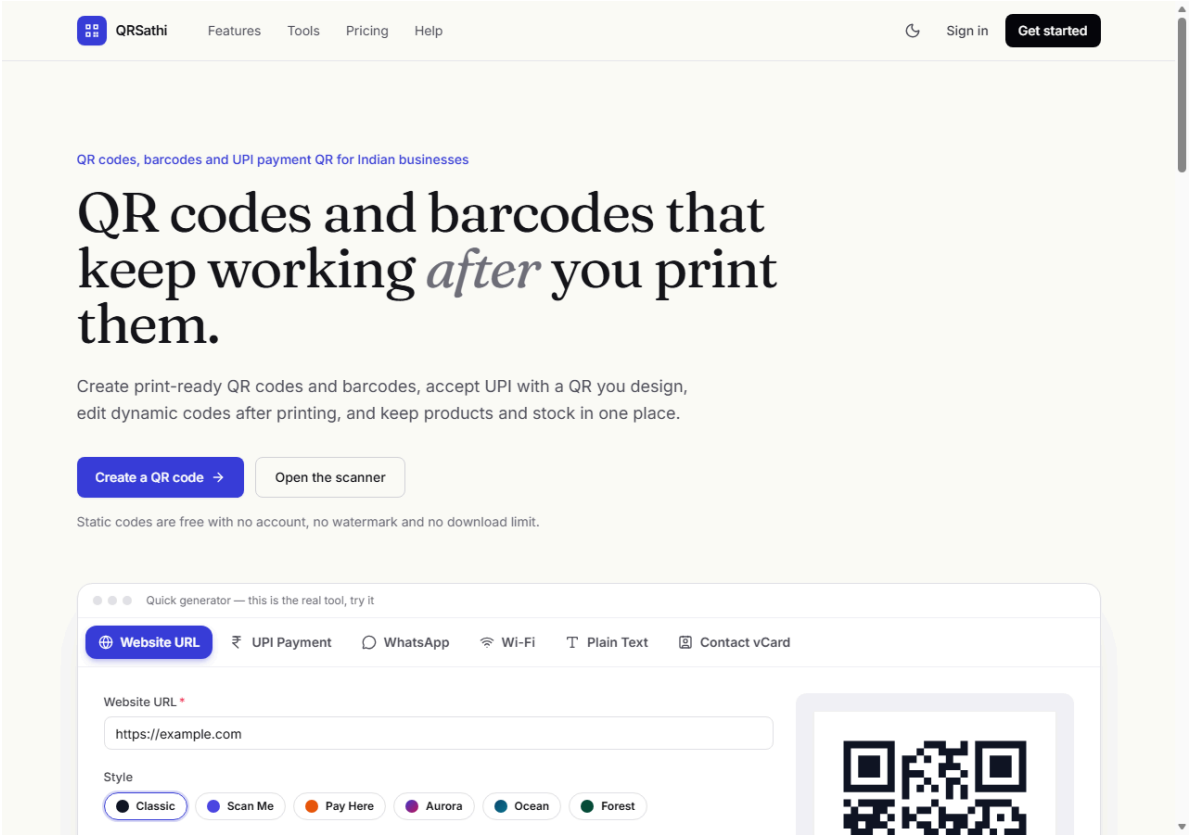


Figure 6.1: Landing page (desktop, 1280 px)

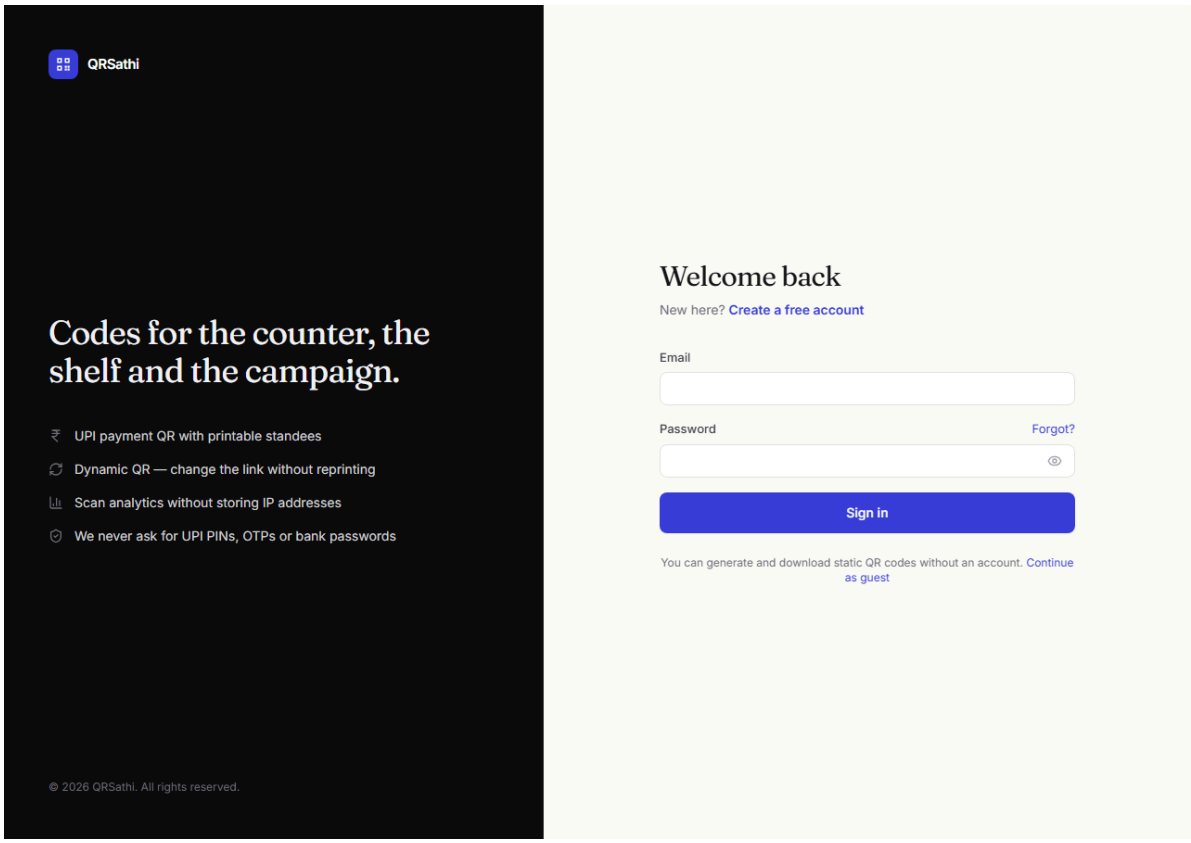


Figure 6.2: Sign-in page

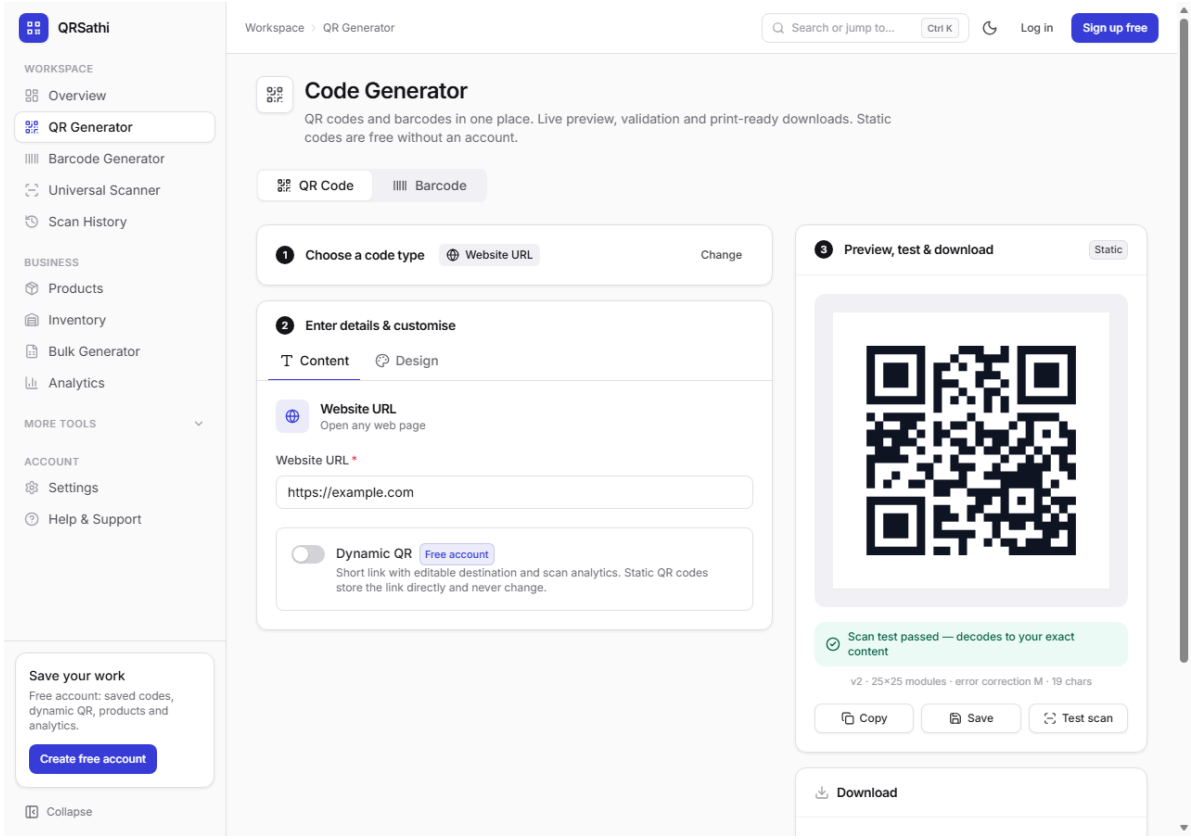


Figure 6.3: QR code generator with live preview and scan test

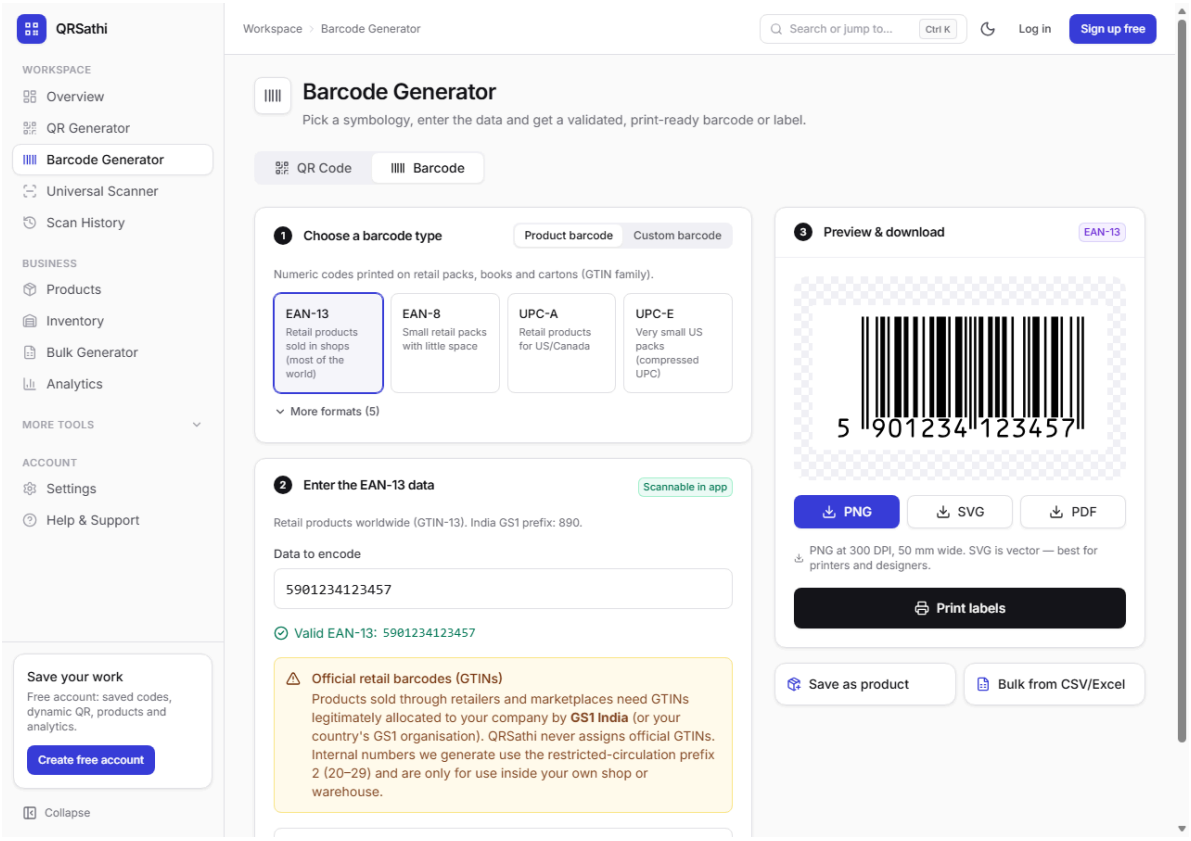


Figure 6.4: Barcode generator

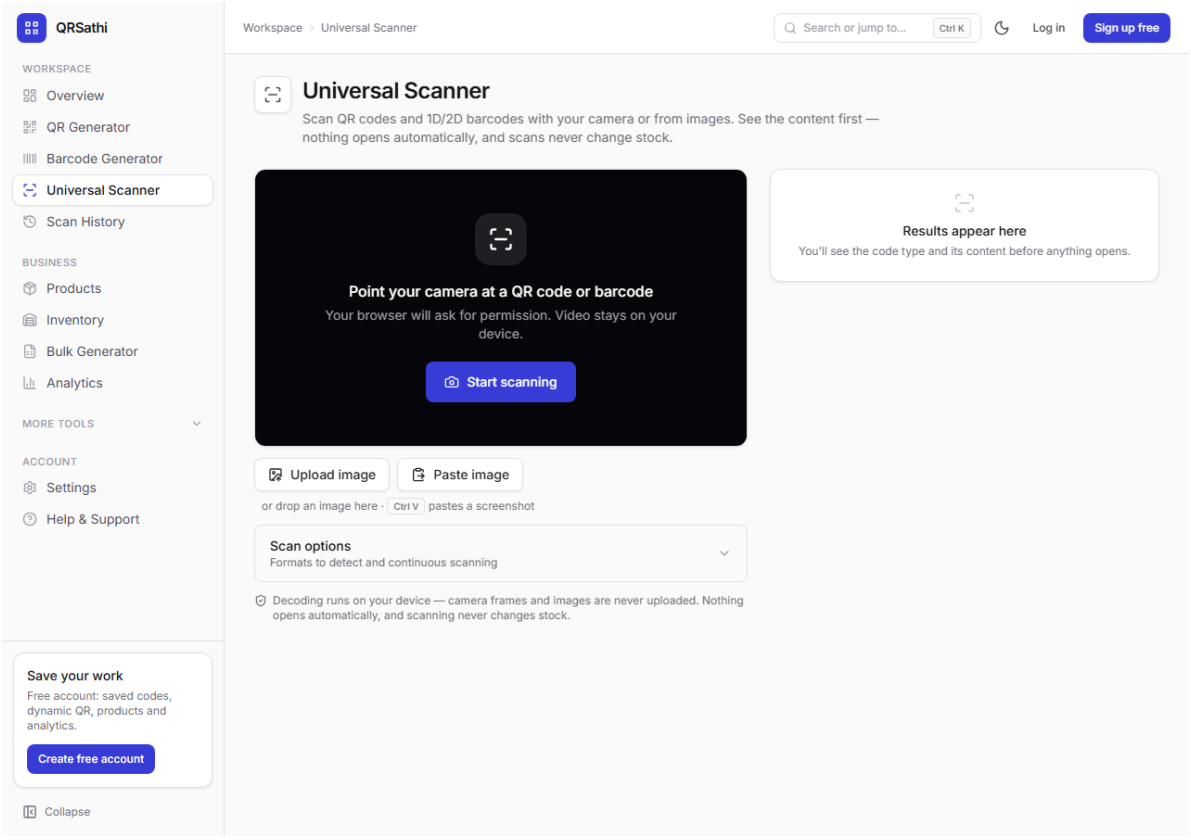


Figure 6.5: Universal scanner

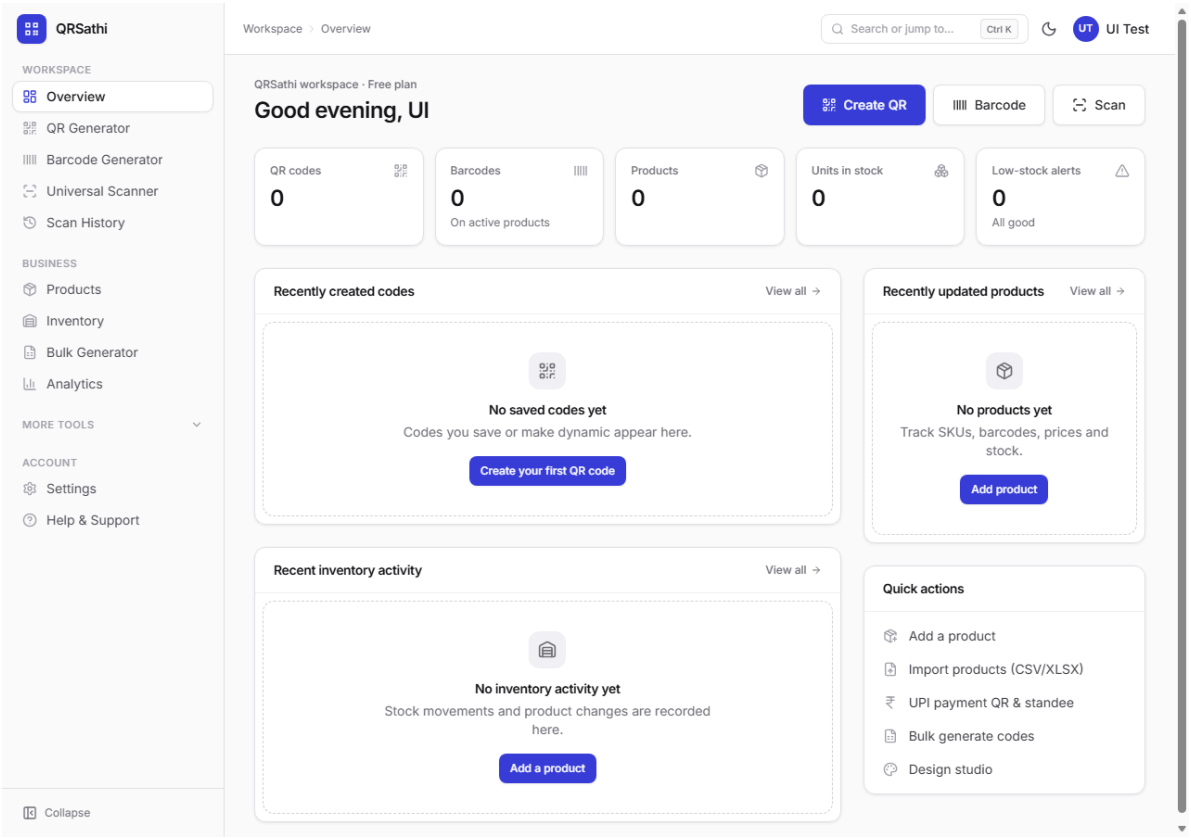


Figure 6.6: Workspace overview dashboard (new account)

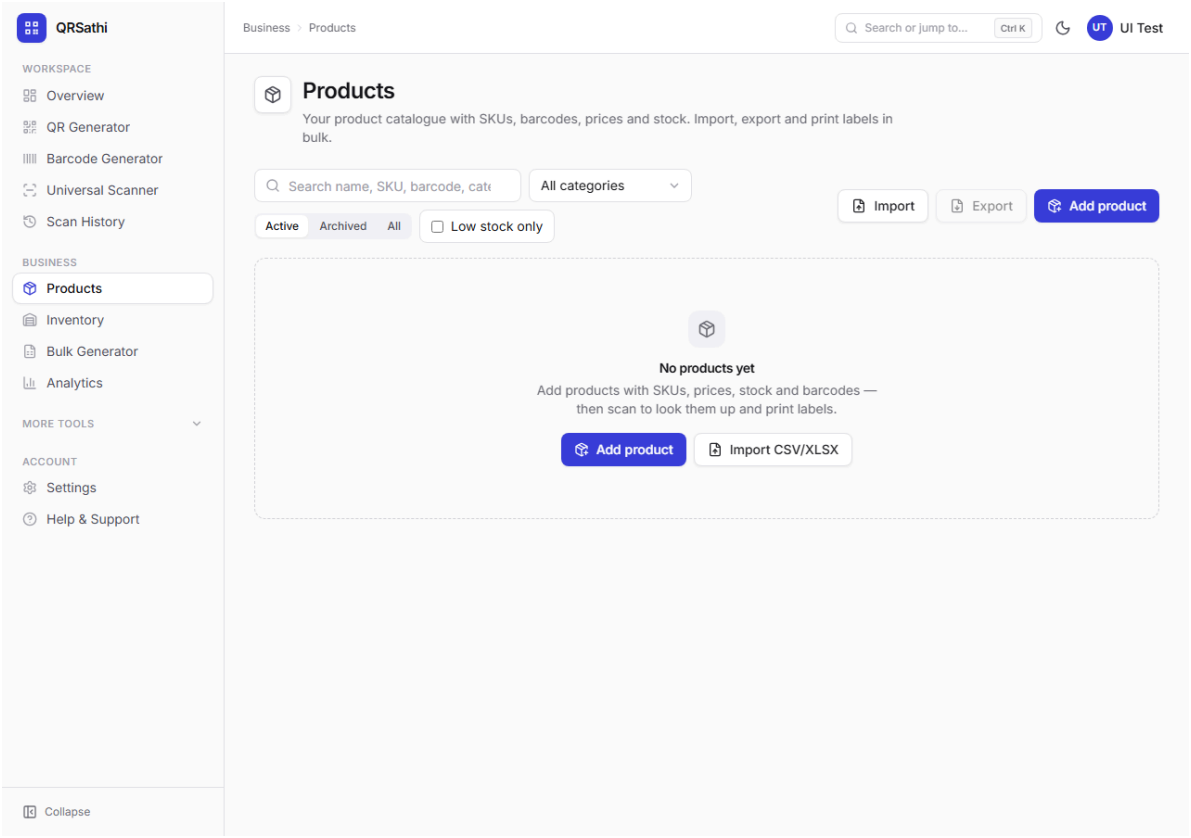


Figure 6.7: Products list (new account)

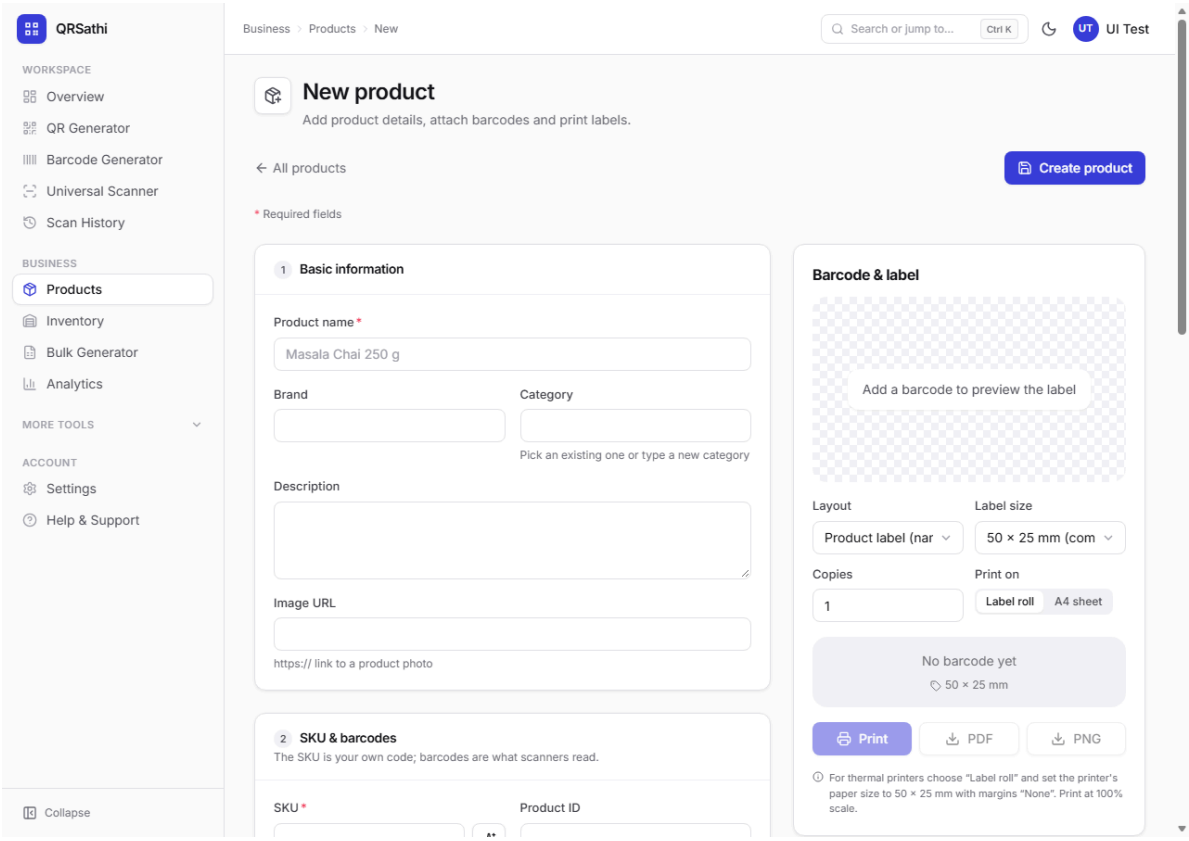


Figure 6.8: New product form

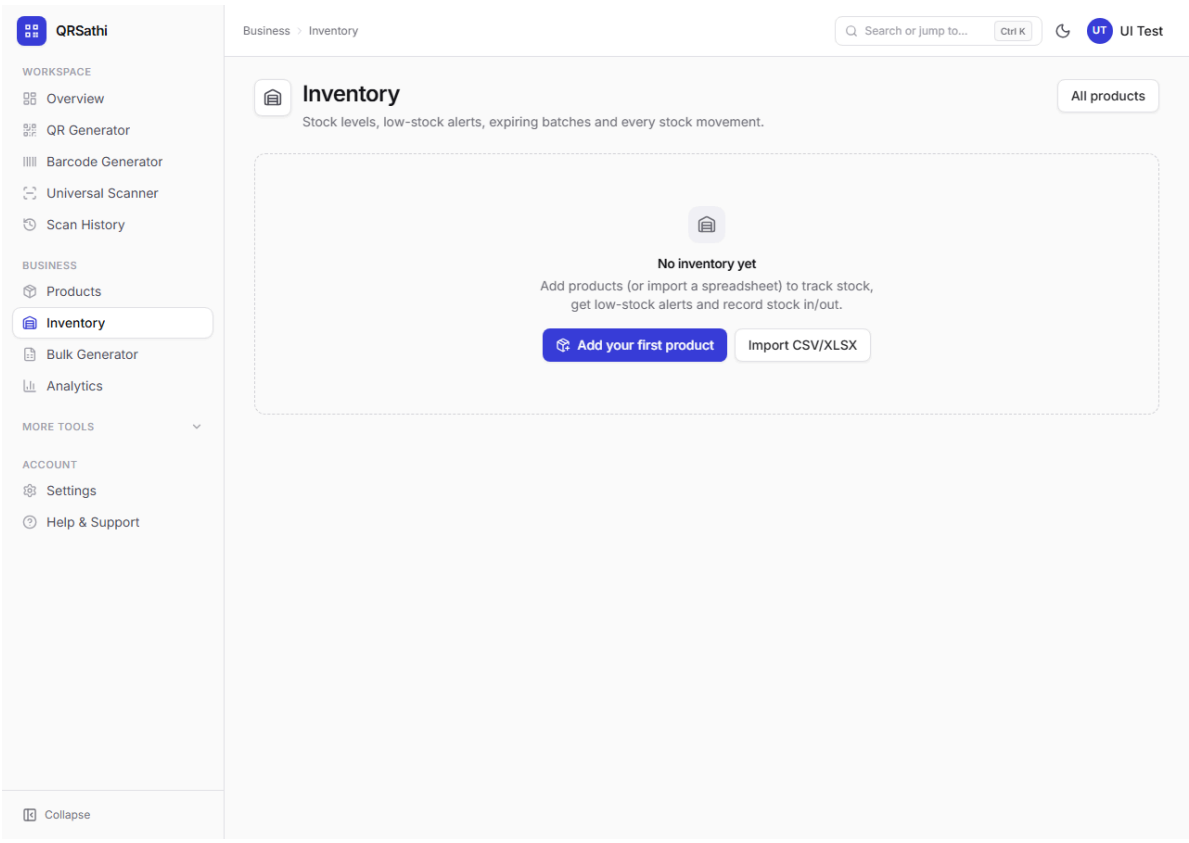


Figure 6.9: Inventory

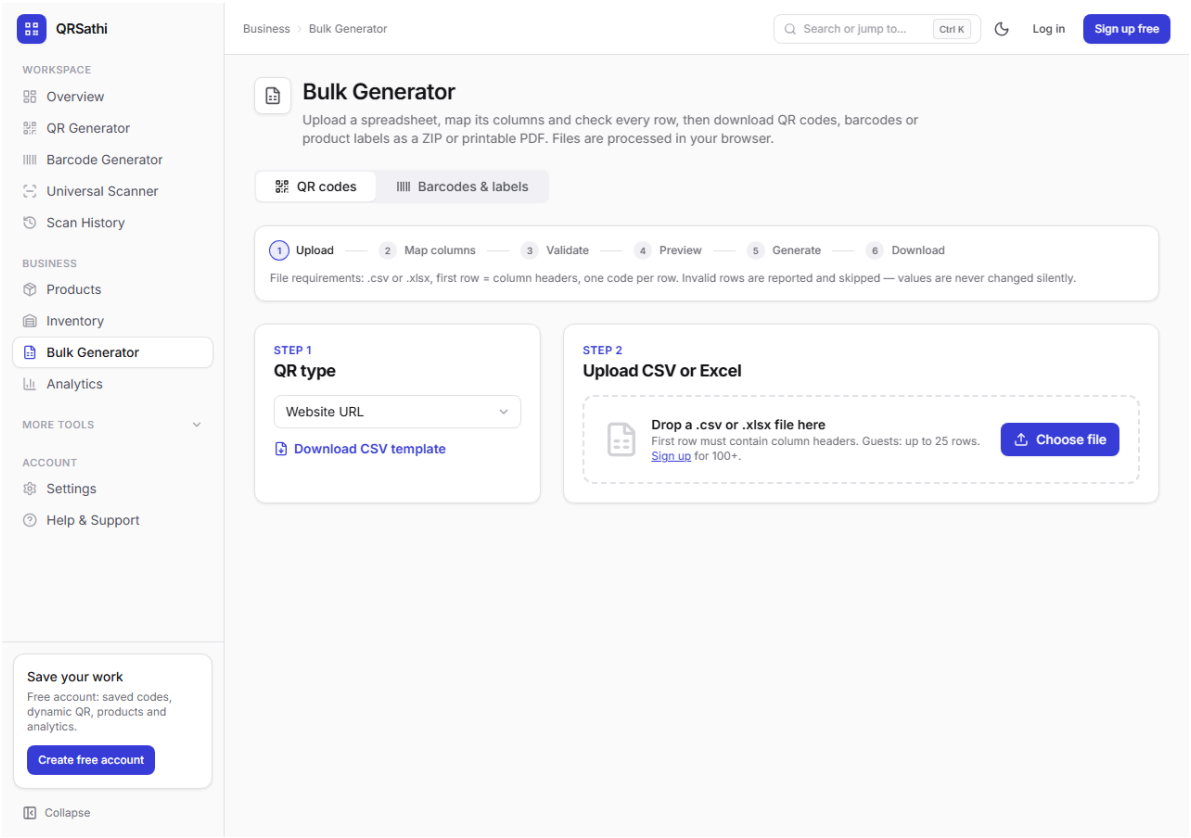


Figure 6.10: Bulk generator

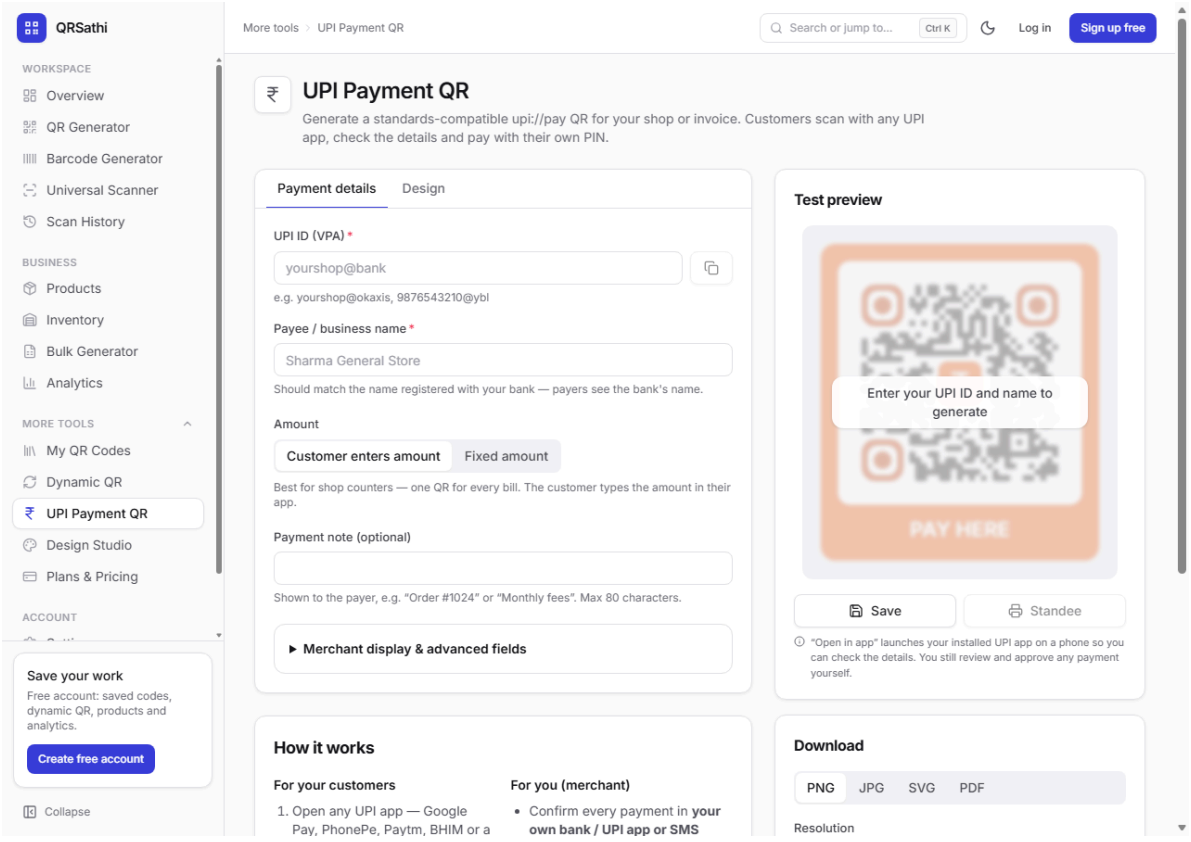


Figure 6.11: UPI payment QR tool

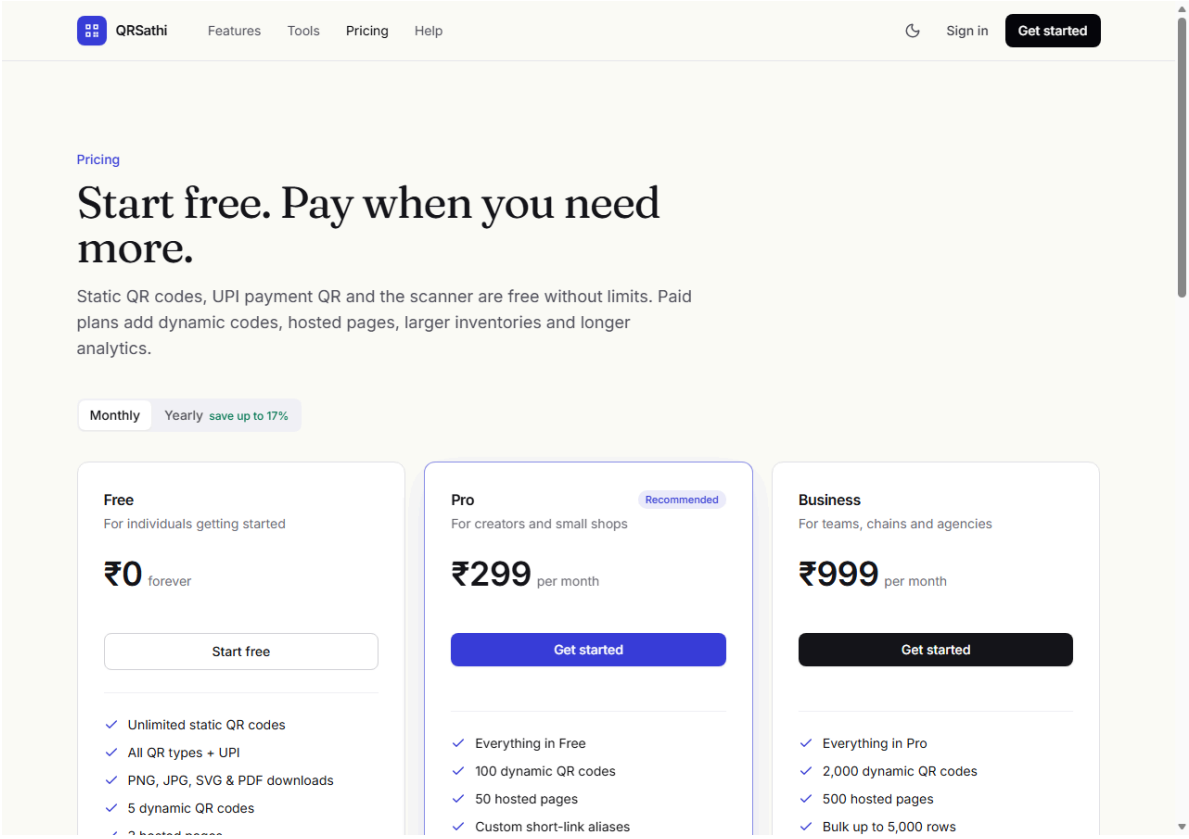


Figure 6.12: Pricing page

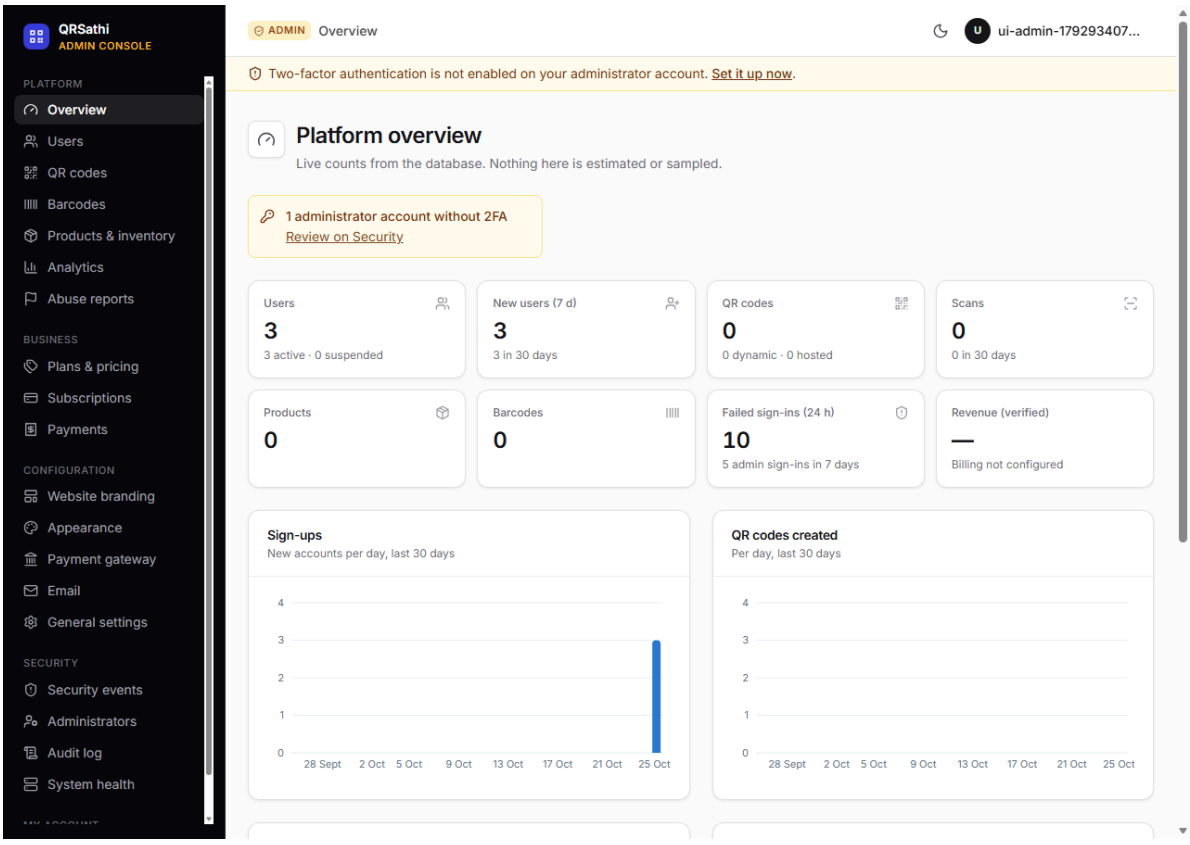


Figure 6.13: Admin console overview

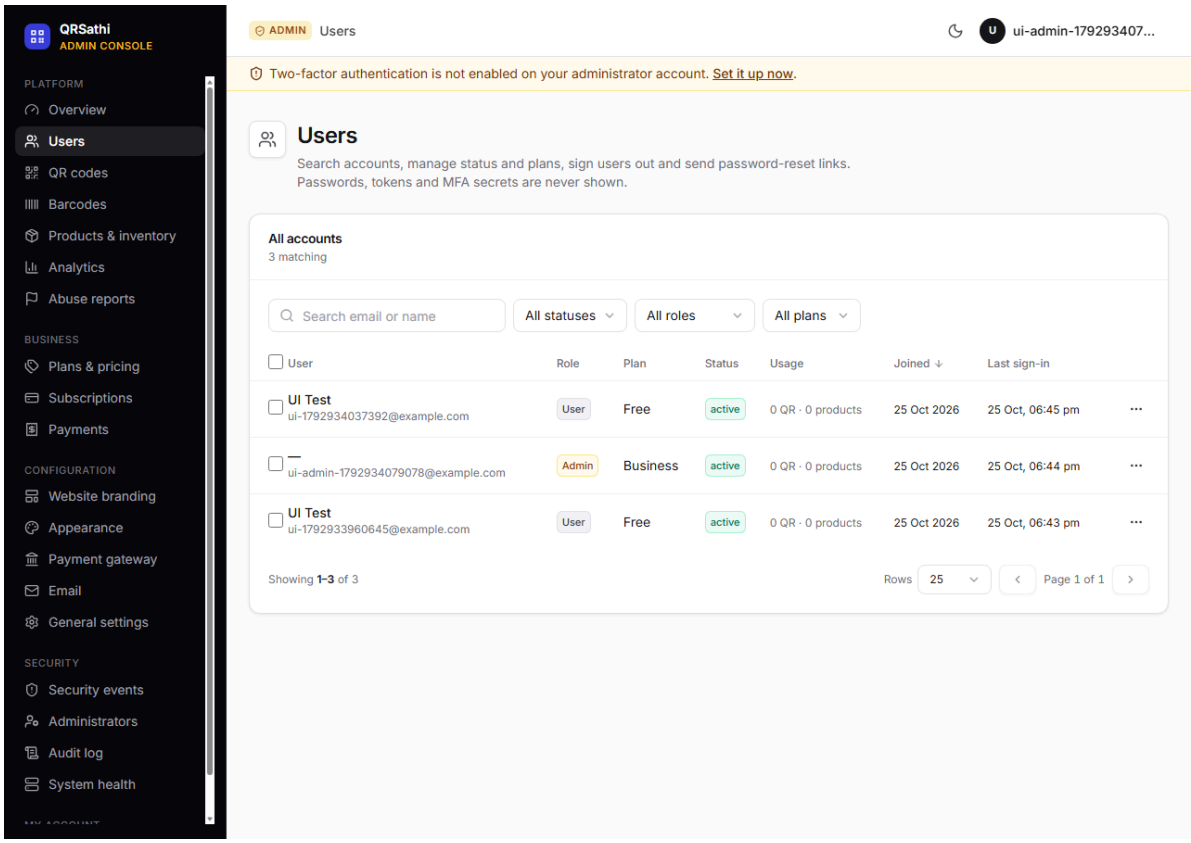


Figure 6.14: Admin user management

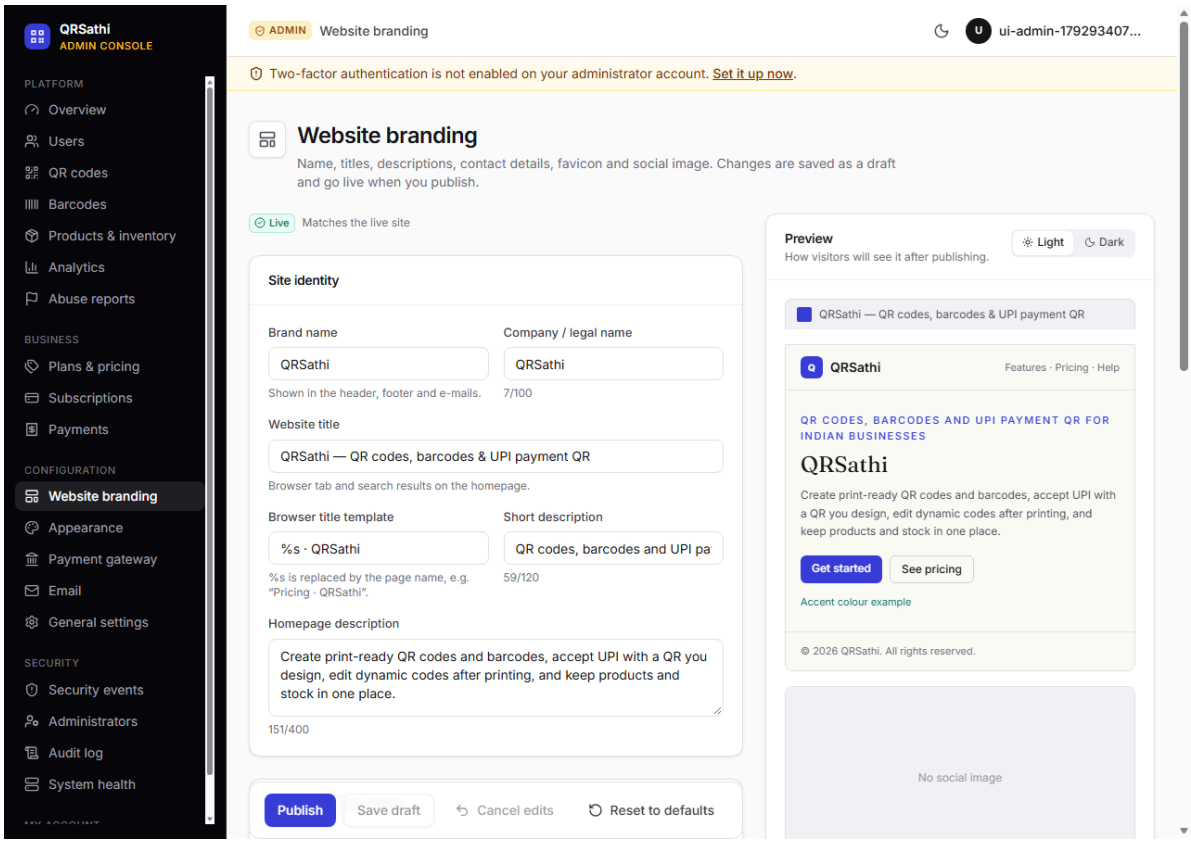


Figure 6.15: Admin branding settings

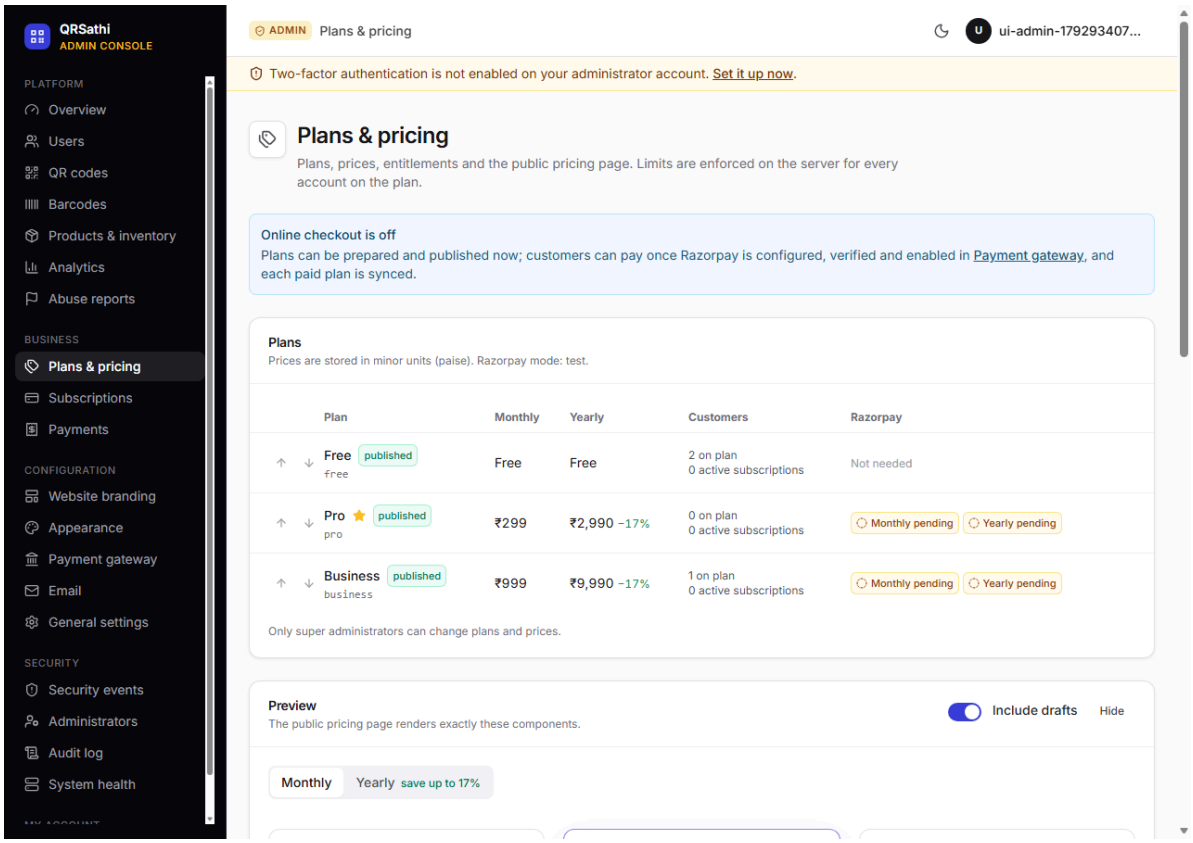


Figure 6.16: Admin pricing plans editor

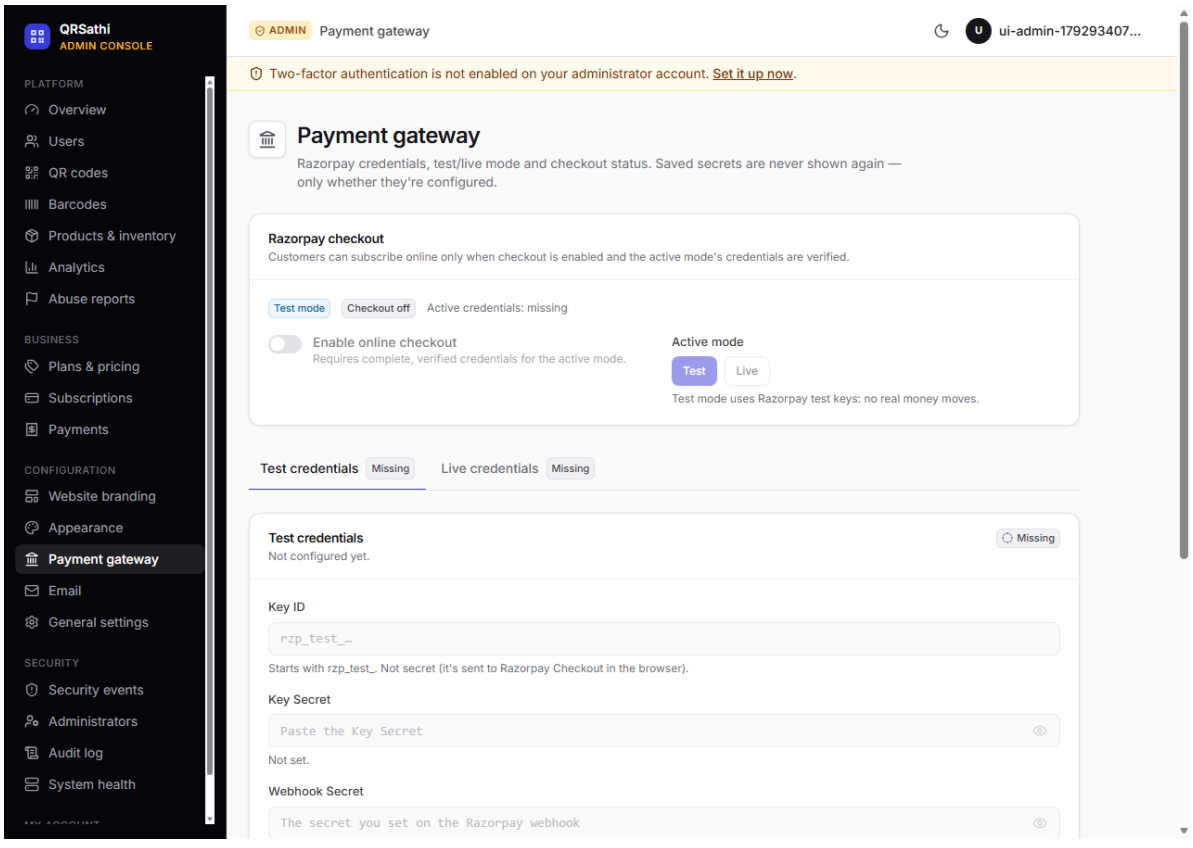


Figure 6.17: Admin payment-gateway configuration

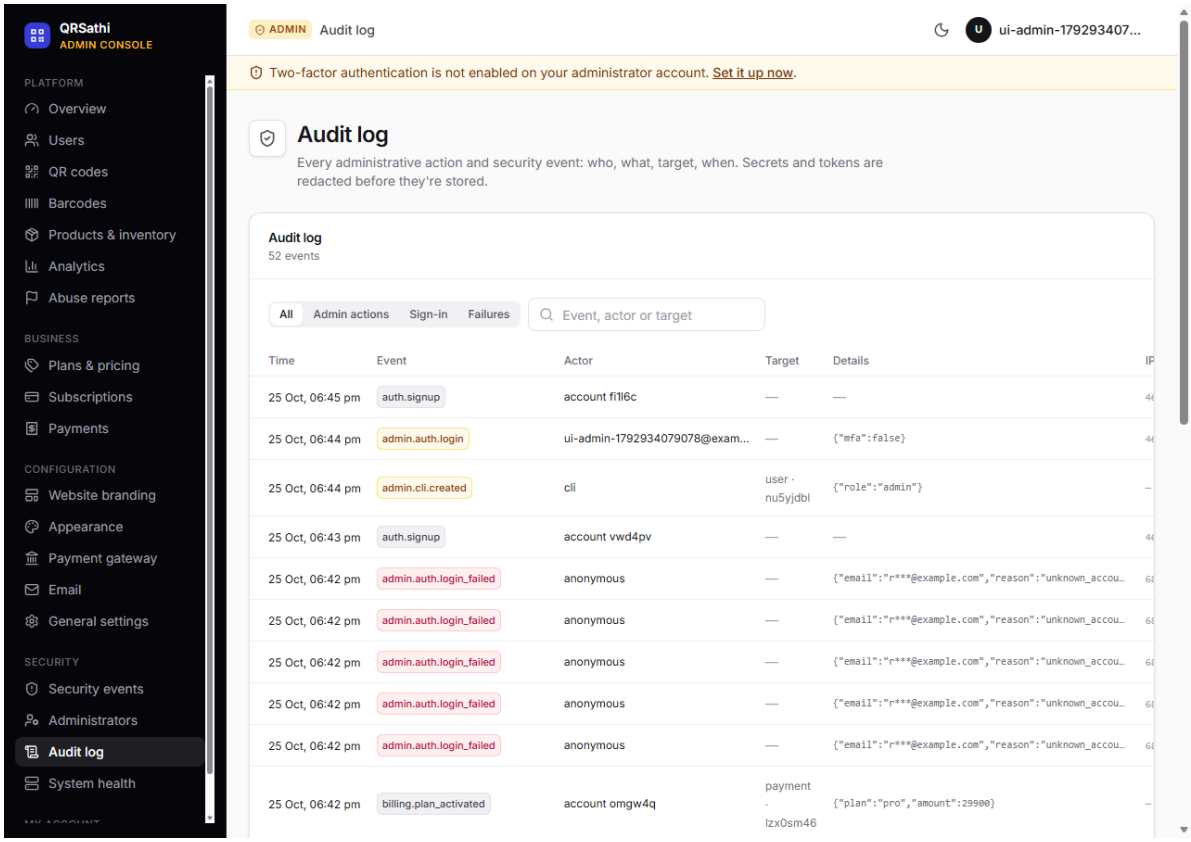


Figure 6.18: Admin security and audit log

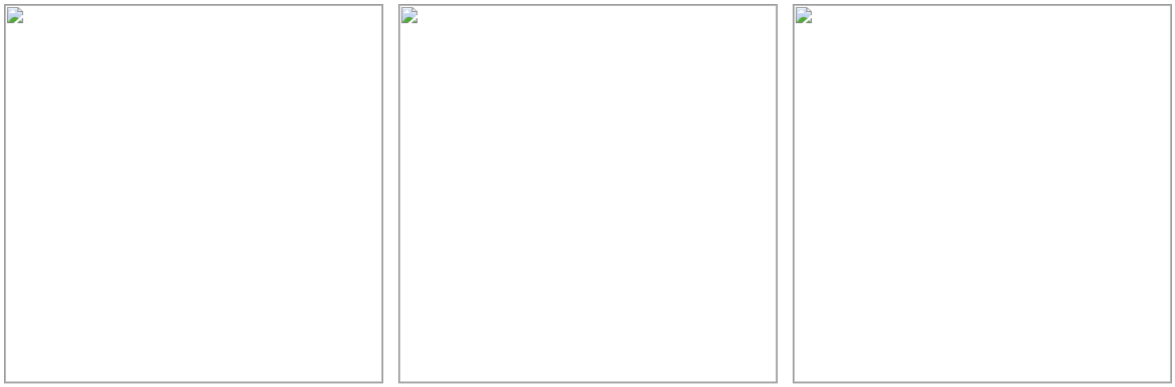


Figure 6.19: Mobile views (375 px): landing page, generator and scanner

6.22 Evaluation of Implemented Features

Table 6.2: Feature verification matrix

Feature	Expected purpose	Implementation location	Status	Verification method	Findings
Account sign-up / sign-in / sign-out	User accounts with secure sessions	src/app/api/auth/*, src/lib/auth-flow.ts, src/lib/session.ts	Implemented and verified	test:admin, test:api, probe	Generic errors, three rate limits, server-side sessions
Google sign-in	Sign in with Google (OIDC)	src/app/api/auth/google/**	Implemented, not independently verified	Source review	No Google credentials in audit environment
Password reset	One-hour single-use e-mail link	src/app/api/auth/forgot|reset, src/lib/password-reset.ts	Implemented and verified	test:admin	E-mail delivery (SMTP) not exercised
Account settings / delete account	Profile, password change, sign out other devices, deletion	src/app/api/account/**	Implemented and verified	test:admin	Deletion requires typing the e-mail
Static QR generator (27 types)	Create QR codes for URL, Wi-Fi, vCard, UPI etc.	src/lib/qr-types.ts, src/components/qr/*	Implemented and verified	test:qr, test:api, test:browser	Every type/style decodes
QR design and export	Colours, shapes, logo, frames; PNG/JPG/SVG/PDF/ZIP	src/lib/qr-render.ts, src/lib/qr-export.ts	Implemented and verified	test:qr; screenshot	Automatic ECC H with logo; scan test after export
Saved QR library	List, search, favourite, edit, duplicate, delete	src/lib/qr-service.ts, src/app/api/qrcodes/**	Implemented and verified	test:admin (IDOR), test:billing (quota)	—
Dynamic QR (short links)	Editable destination, pause, expiry, alias, UTM	src/app/r/[code]/route.ts	Implemented and verified	test:api, probe (pause/expiry/404)	Redirects are no-store
Hosted landing pages	Menu, links, business, coupon etc. at /p/{code}	src/app/p/[code]/page.tsx, src/components/hosted-page.tsx	Implemented, not independently verified	Source review	No test asserts hosted page content
Scan analytics	Device/browser/OS/country charts; CSV export	src/lib/scans.ts, src/lib/analytics.ts, src/app/api/analytics/**	Implemented and verified	test:admin (scan cap), test:billing (plan gating)	No raw IPs; DNT/GPC honoured
UPI payment QR and standees	upi://pay QR for VPA or account+IFSC	src/lib/upi.ts, src/app/(app)/upi/*	Implemented and verified	test:api, test:qr	Limit warnings ₹1 lakh / ₹5 lakh
Barcode generator (23 formats)	Valid barcodes with check digits	src/lib/barcodes.ts, src/components/barcode/*	Implemented and verified	test:barcodes (166)	Render → decode round-trip for every format
Barcode labels	Printable labels in 6 sizes, 4 layouts	src/lib/barcodes.ts (renderLabelSvg), src/lib/barcode-export.ts	Implemented and verified	test:barcodes ('Labels stay scannable', PDF)	—
Universal scanner	Camera/image decoding with safety notes	src/app/(app)/scanner/scanner.tsx, src/lib/zxing-client.ts, src/lib/content-safety.ts	Implemented and verified	test:browser (camera, upload, UPI warning)	Chromium only
Scan history	On-device history with CSV export	src/lib/scan-history.ts	Implemented and verified	test:browser ('camera scan saved to on-device history')	Stored in localStorage, not in the account
Products	Catalogue with multiple barcodes per product	src/lib/products.ts, src/app/api/products/**	Implemented and verified	test:api	Per-user SKU/barcode uniqueness incl. GTIN equivalence
Inventory movements	Stock in/out/adjust, low stock, ledger	moveStock() in src/lib/products.ts, src/app/api/inventory	Implemented and verified	test:api	Stock never negative; scanning never changes stock

Feature	Expected purpose	Implementation location	Status	Verification method	Findings
Product lookup by scan	Find product from scanned code	src/app/api/products/lookup	Implemented and verified	test:api	Read-only
Bulk QR / barcode generation	Generate many codes from a table	src/app/(app)/bulk/*	Implemented, not independently verified	Page renders (test:browser); no functional assertion	Plan bulkRows limit
Product import (CSV/XLSX)	Import or update products from spreadsheets	src/app/api/products/import, src/lib/table-file.ts	Implemented and verified	test:api	skip/update modes, per-row report
File uploads for File QR	Upload PDFs/images/Office files	src/app/api/uploads, src/lib/storage.ts, src/app/f/[name]	Partially implemented / defect	test:admin, probes	D-01: >10 MB uploads fail on paid plans; D-02 CSP fixed in working tree
Plans and entitlements	Server-enforced limits and features	src/lib/entitlements.ts, src/lib/plan-store.ts, src/lib/plans.ts	Implemented and verified	test:billing	Concurrency-safe via quotaLock
Pricing administration	Create/edit/publish/archive/reorder plans	src/app/api/admin/plans/**, src/lib/admin-plans.ts	Implemented and verified	test:billing	Super admin only; layout overflow at 768 px (D-03)
Razorpay subscriptions	Online payment for paid plans	src/lib/billing.ts, src/lib/razorpay.ts, src/app/api/billing/**	Implemented and verified	test:billing (mock Razorpay), test:admin (webhooks)	Not tested against live Razorpay
Payment gateway configuration	Sealed credentials, verification, enable checkout	src/lib/payment-config.ts, src/lib/secret-box.ts	Implemented and verified	test:billing	Secrets never returned; re-auth required
Admin console sign-in and MFA	Separate admin login with TOTP	src/app/api/admin/auth/**, src/lib/mfa.ts	Implemented and verified	test:admin	MFA mandatory for super admin only
User administration and RBAC	Suspend, roles, sessions, reset, delete	src/app/api/admin/users/**, src/lib/admin-users.ts	Implemented and verified	test:admin	D-05: plan enum hard-coded
QR moderation and abuse reports	Pause/disable codes; triage reports	src/app/api/admin/qrcodes/**,** **admin/reports/**, api/reports	Implemented, not independently verified	Source review; pages render	No functional assertion in suites
Branding and appearance	Names, colours, logos, favicon; draft/publish	src/lib/branding.ts, src/app/api/admin/branding/**	Implemented and verified	test:billing (branding), test:admin (XSS)	SVG sanitisation and sandboxing
Audit log and security events	Record security-relevant actions	src/lib/audit.ts, src/app/api/admin/security	Implemented and verified	test:admin	Redacted; IPs hashed
System health and logs	Health probe, config status, logs	src/app/api/health, src/lib/system-status.ts, src/lib/log.ts	Implemented and verified	test:admin (health)	Health reveals only ok/degraded
SQLite ↔ MySQL support	Dev on SQLite, production on MySQL	prisma/schema.prisma, prisma/mysql/**, scripts/db/*	Implemented and verified	test:mysql (32), schema check	MySQL production deployment itself not verified
Light/dark theme	User-selectable theme	src/components/providers.tsx	Implemented, not independently verified	Source review; DARK=1 mode exists in test but was not run	—
Command palette	Keyboard navigation (Ctrl K)	src/components/command-palette.tsx	Implemented, not independently verified	Visible in screenshots	—
E-mail verification at sign-up	Confirm e-mail ownership	—	Planned / not implemented	—	Recommended in docs/SECURITY-AUDIT.md

Feature	Expected purpose	Implementation location	Status	Verification method	Findings
Object storage for uploads (S3/R2)	Scalable file storage	src/lib/storage.ts comment	Planned / not implemented	—	Storage functions designed to be replaceable

6.23 Evaluation Against Project Objectives

The objectives O1–O10 of Section 1.7 are evaluated in Table 7.1. In brief: objectives O1–O7, O9 and O10 are met with test evidence; O8 (plans and payments) is met for entitlements and verified against a mock payment provider, but not against live Razorpay; and the upload limits that are part of O8's plan model are affected by defect D-01.

6.24 Known Limitations

1. **Large uploads (D-01):** files over 10 MB cannot be uploaded even on plans that allow 25–50 MB.
2. **Local file storage:** uploads live on one server's disk; horizontal scaling needs shared or object storage.
3. **No e-mail verification** at sign-up; password reset depends on SMTP being configured.
4. **Rate limiting** defaults to per-process memory; multi-instance deployments must enable the database store and correct proxy settings.
5. **External integrations untested** in this audit (Google, SMTP, live Razorpay).
6. **No performance baseline** and no load testing.
7. **Browser coverage:** automated tests only on Chromium.
8. **Single currency for UPI**; payments only via Razorpay.
9. **Scan history** is per device (browser storage) and is not synchronised to the account.
10. **Admin plan assignment** limited to the three built-in plan ids (D-05).
11. **Repository hygiene:** personal data in Git history (D-07) and a large amount of uncommitted work at audit time.

6.25 Maintenance and Operational Considerations

- Keep dependencies current (Dependabot is configured) and review the advisory allowlist before its 2027-01-31 expiry.
- Run the CI workflow on every change; add the audit probes (D-01 regression) to the suites.
- Back up the database and `UPLOAD_DIR` together; test restores (`docs/BACKUP-RECOVERY.md`).
- Rotate secrets carefully: `MFA_ENCRYPTION_KEY` and `SETTINGS_ENCRYPTION_KEY` are needed to decrypt stored secrets.
- Monitor `/api/health`, the system page and security events in the admin console; review the audit log.
- Periodically purge expired sessions, old logs and rate-limit rows (rate-limit rows are cleaned opportunistically; no scheduled job exists for sessions or logs).

6.26 Implementation Summary

QRSathi is implemented as a coherent, layered Next.js application in which validation, authorization and business rules live in a small set of server modules shared by all routes. The implementation goes beyond basic CRUD: it uses transactions for consistency, idempotency for payments, encryption for secrets and careful privacy defaults. The audit verified the large majority of features with executed tests and found one significant functional defect (D-01) and several minor issues, all of which are documented for correction.

CHAPTER 7
Conclusion and Future Enhancements

7.1 Conclusion

QRSathi set out to replace a fragmented set of tools — QR generators, barcode websites, phone scanners and stock spreadsheets — with one secure web application for small businesses. The audit carried out for this report confirms that the project achieves this aim to a substantial degree. The application integrates QR generation for 27 content types, dynamic QR codes with privacy-respecting analytics, barcode generation for 23 symbologies with standards-based validation, in-browser scanning with fraud warnings, a product catalogue with an append-only inventory ledger, bulk tools, a separate MFA-protected administration console, and plan-based billing through Razorpay.

The verified technical contribution lies less in any single feature than in the way the features are engineered: validation schemas shared by browser and server; server-side sessions that can be revoked instantly; deny-by-default authorization; transactions that keep quotas and stock consistent under concurrency; idempotent payment processing; encrypted storage of secrets; and a schema that runs on both SQLite and MySQL. These properties are not just claimed — 745 integration and end-to-end assertions, 300 unit assertions and 32 MySQL assertions passed in the final audit run.

The audit also found problems that must be addressed before production use: uploads above 10 MB fail on paid plans (D-01), a minor admin layout overflow and stale browser-test expectations (D-03, D-04), a hard-coded plan list in one admin endpoint (D-05), and — most importantly for publication — a development database with password hashes in the Git history (D-07). A large part of the latest work was also uncommitted at the time of the audit.

7.2 Achievement of Objectives

Table 7.1: Objectives versus results

Objective	Result	Evidence	Assessment
O1 QR generation with validation and scannable designs	27 types; zod validation; design warnings; post-export scan test	test:qr; TC-QR-01..08	Achieved
O2 Dynamic QR with privacy-preserving analytics	Short links with edit/pause/expiry/UTM; scan cap; no raw IPs; DNT/GPC	test:api, test:admin, probes	Achieved
O3 Standard barcodes with check digits and labels	23 formats; GS1 check digits; labels in 6 sizes	test:barcodes (166)	Achieved
O4 Browser scanning with risk warnings	Camera and multi-code image decoding; URL/UPI heuristics	test:browser	Achieved (Chromium)
O5 Products and auditable inventory	Per-user uniqueness; ledger with balances; non-negative stock	test:api	Achieved
O6 Bulk generation and import	Import verified; bulk generation UI present	test:api; source review	Largely achieved
O7 Secure authentication, RBAC, MFA console	Separate admin sessions, TOTP, recovery codes, audit	test:admin (99)	Achieved
O8 Plans with server-enforced limits and payments	Entitlements and Razorpay lifecycle verified with mock provider	test:billing (128)	Partially achieved: live payments not verified; D-01 affects upload limits
O9 SQLite and MySQL from one schema	Generated schema, migrations, data migration	test:mysql (32)	Achieved
O10 Automated verification	Unit, API, admin, billing, browser and MySQL suites; CI workflow	Chapter 5	Achieved (coverage percentage unknown)

7.3 Major Contributions

1. An integrated, open-source-ready platform combining QR codes, barcodes, scanning and inventory for Indian small businesses.

2. A type-registry design in which each QR type declares its own fields, validation and payload builder, shared by client and server.
3. GTIN normalisation (lookup keys) so that equivalent barcode forms match the same product, and enforcement of GS1 restricted-circulation ranges for internal numbers.
4. A privacy-preserving approach to scan analytics.
5. A security architecture suitable as a teaching example: revocable sessions, separate admin console, TOTP with replay protection, CSRF origin checks, sealed secrets and audit logging.
6. A comprehensive automated test set, including real-browser camera tests and real-MySQL migration tests.

7.4 Current Limitations

- File uploads above 10 MB fail (D-01); files are stored on local disk.
- No e-mail verification; resets require SMTP.
- External integrations (Google, SMTP, live Razorpay) unverified.
- No performance or load testing; no code-coverage measurement.
- Automated browser testing on Chromium only.
- Scan history not synchronised across devices.
- Personal data in Git history; much work uncommitted at the time of the audit.

7.5 Future Enhancements

The following are proposals; none of them is implemented unless stated.

Table 7.2: Proposed future enhancements

Area	Enhancement	Note
Correctness	Fix D-01 (exclude `/api/uploads` from the middleware matcher or raise the middleware body limit) and add a regression test	Highest priority
Analytics and reporting	Scheduled reports; campaign comparison; inventory valuation and movement reports	Scan analytics and CSV export already exist
Bulk processing	Background jobs for very large batches; progress tracking	Client-side bulk generation exists
Export formats	EPS/vector PDF labels for print shops; Excel export of inventory	PNG/JPG/SVG/PDF/ZIP exist
Inventory alerts	E-mail/WhatsApp low-stock and expiry alerts; multi-location stock	Low-stock flag exists in UI
Audit logs	Export and retention policies; tamper-evident hashing	Audit log exists
Permissions	Team workspaces with per-member roles (owner, editor, viewer)	Currently one owner per account
Authentication	E-mail verification; passkeys (WebAuthn); MFA for all users; breached-password checks	TOTP exists for admins
Accessibility	Manual WCAG 2.2 AA audit; screen-reader testing	Automated name checks exist
Performance	Load testing; caching of hosted pages; database indexes reviewed with real data	—
Backups and recovery	Automated scheduled backups and restore drills	Procedure documented only
Monitoring	Metrics and tracing (e.g. OpenTelemetry); alerting on health and errors	Health endpoint and logs exist
Payments	Additional providers; invoices with GST details; coupons	Razorpay subscriptions exist
Internationalisation	Hindi and regional-language UI; locale formatting	UI is English
API and integrations	Public REST API with API keys; OpenAPI documentation; webhooks for scans	Internal API only
Testing	Coverage reporting; Firefox/Safari/mobile device tests; contract tests against Razorpay test mode	Large suite exists
Storage	Object storage (S3-compatible) for uploads	`storage.ts` designed to be replaceable
Security	URL reputation checks for dynamic destinations; CSP nonces; Git history clean-up	See Table 4.12

7.6 Long-Term Scope

In the long term QRSathi could evolve into a small-business identification platform: a native or progressive-web mobile app for offline scanning and stock-taking, point-of-sale integration, GS1 Digital Link support (QR codes that carry GTINs and resolve to product information), supplier and purchase-order management, and analytics that connect scans to sales. Because the core is a typed, layered code base with a portable schema and automated tests, these extensions can be added incrementally.

7.7 Final Conclusion

QRSathi is a working, well-engineered mini project that meets most of its objectives with evidence. It demonstrates practical skills in full-stack web development, database design, security engineering and automated testing. With the documented defects fixed, the work committed and the repository history cleaned, it would be a sound foundation for a production service.

References

References follow the IEEE style. All web sources were accessed on 25 October 2026. URLs marked † returned HTTP 403 to the automated link check (bot protection on the publisher's site) and should be confirmed manually in a browser before submission; all other URLs returned HTTP 200.

[1] Vercel, "Next.js Documentation." [Online]. Available: https://nextjs.org/docs

[2] Vercel, "Next.js API Reference: middleware." [Online]. Available: https://nextjs.org/docs/app/api-reference/file-conventions/middleware

[3] Meta Open Source, "React Reference." [Online]. Available: https://react.dev/reference/react

[4] Microsoft, "TypeScript Documentation." [Online]. Available: https://www.typescriptlang.org/docs/

[5] Prisma Data, Inc., "Prisma ORM Documentation." [Online]. Available: https://www.prisma.io/docs

[6] Tailwind Labs, "Tailwind CSS v3 Documentation." [Online]. Available: https://v3.tailwindcss.com/docs

[7] Oracle Corporation, "MySQL 8.4 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/ †

[8] International Organization for Standardization, *ISO/IEC 18004:2015 Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*. Geneva, Switzerland: ISO, 2015. [Online]. Available: https://www.iso.org/standard/62021.html †

[9] GS1, "GS1 General Specifications." [Online]. Available: https://www.gs1.org/standards/barcodes-epcrfid-id-keys/gs1-general-specifications †

[10] GS1, "Check Digit Calculator." [Online]. Available: https://www.gs1.org/services/check-digit-calculator †

[11] DENSO WAVE Inc., "History of QR Code." [Online]. Available: https://www.qrcode.com/en/history/

[12] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, IETF, May 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6238

[13] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache and O. Ranen, "HOTP: An HMAC-Based One-Time Password Algorithm," RFC 4226, IETF, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4226

[14] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, IETF, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[15] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[16] OWASP Foundation, "OWASP Cheat Sheet Series" (Authentication, Session Management, Cross-Site Request Forgery Prevention, Password Storage). [Online]. Available: https://cheatsheetseries.owasp.org/

[17] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[18] P. A. Grassi *et al.*, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST SP 800-63B. [Online]. Available: https://pages.nist.gov/800-63-3/sp800-63b.html

[19] Mozilla, "Content-Security-Policy," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy

[20] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation. [Online]. Available: https://www.w3.org/TR/WCAG22/

[21] Razorpay, "Subscriptions." [Online]. Available: https://razorpay.com/docs/payments/subscriptions/

[22] Razorpay, "Validate and Test Webhooks." [Online]. Available: https://razorpay.com/docs/webhooks/validate-test/

[23] National Payments Corporation of India, "Unified Payments Interface (UPI): Product Overview." [Online]. Available: https://www.npci.org.in/what-we-do/upi/product-overview †

[24] C. McDonnell, "Zod Documentation." [Online]. Available: https://zod.dev

[25] metafloor, "bwip-js: Barcode Writer in Pure JavaScript," GitHub repository. [Online]. Available: https://github.com/metafloor/bwip-js

[26] Sec-ant, "zxing-wasm: ZXing-C++ WebAssembly build," GitHub repository. [Online]. Available: https://github.com/Sec-ant/zxing-wasm

[27] soldair, "node-qrcode: QR code generator," GitHub repository. [Online]. Available: https://github.com/soldair/node-qrcode

[28] SQLite Consortium, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html

[29] Mermaid contributors, "Mermaid: Diagramming and charting tool." [Online]. Available: https://mermaid.js.org

[30] QRSathi project repository, source code and documentation (`README.md`, `docs/*.md`), branch `main`, commit `9b5dae7` plus working-tree changes as of 25 Oct. 2026. Remote: https://github.com/druvemedia/qrsathi (not accessed during the audit; the local copy was used).

Appendix A: Project Directory Structure

The tree below lists source, configuration and documentation files. Dependencies (`node_modules`), build outputs (`.next*`), uploaded files, databases and backups, and secret files (`.env`) are excluded.

Files listed: 315 (tracked and untracked, excluding ignored files).

```
qrsathi/
├── .github/
│   ├── ISSUE_TEMPLATE/
│   │   ├── bug_report.yml
│   │   ├── config.yml
│   │   └── feature_request.yml
│   ├── workflows/
│   │   └── ci.yml
│   ├── dependabot.yml
│   └── PULL_REQUEST_TEMPLATE.md
├── docs/
│   ├── mini-project-report/
│   │   └── (this report package)
│   ├── ADMIN.md
│   ├── BACKUP-RECOVERY.md
│   ├── BARCODES.md
│   ├── DATABASE.md
│   ├── DEPLOYMENT.md
│   ├── LICENSING.md
│   ├── OPEN-SOURCE-READINESS.md
│   └── SECURITY-AUDIT.md
├── prisma/
│   ├── mysql/
│   │   ├── migrations/
│   │   │   ├── 20261025000000_init/
│   │   │   │   └── migration.sql
│   │   │   ├── 20261025180000_billing_branding/
│   │   │   │   └── migration.sql
│   │   │   └── migration_lock.toml
│   │   └── schema.prisma
│   ├── schema.prisma
│   └── seed-demo.mjs
├── scripts/
│   ├── db/
│   │   ├── mysql-schema.mjs
│   │   ├── prisma.mjs
│   │   └── sqlite-to-mysql.mts
│   ├── admin.mts
│   ├── audit-check.mjs
│   ├── copy-wasm.mjs
│   ├── license-report.mjs
│   ├── test-admin.mts
│   ├── test-api.mts
│   ├── test-barcodes.mts
│   ├── test-billing.mts
│   ├── test-browser.mts
│   ├── test-mysql.mts
│   ├── test-qr.mts
│   └── test-security.mts
├── security/
│   └── audit-allowlist.json
├── src/
│   ├── app/
│   │   ├── (app)/
│   │   │   ├── analytics/
│   │   │   │   ├── [id]/
│   │   │   │   │   └── page.tsx
│   │   │   │   └── page.tsx
│   │   │   ├── barcodes/
│   │   │   │   └── page.tsx
│   │   │   ├── billing/
│   │   │   │   └── status/
│   │   │   │       ├── checkout-status.tsx
│   │   │   │       └── page.tsx
│   │   │   ├── bulk/
│   │   │   │   ├── barcode-bulk.tsx
│   │   │   │   ├── bulk-modes.tsx
│   │   │   │   ├── bulk-tool.tsx
│   │   │   │   └── page.tsx
│   │   │   ├── codes/
│   │   │   │   ├── library.tsx
```

```
│   │   │   │   └── page.tsx
│   │   │   ├── dashboard/
│   │   │   │   ├── history/
│   │   │   │   │   ├── history.tsx
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── settings/
│   │   │   │   │   ├── billing.tsx
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── settings.tsx
│   │   │   │   ├── overview.tsx
│   │   │   │   └── page.tsx
│   │   │   ├── dynamic/
│   │   │   │   ├── dynamic-list.tsx
│   │   │   │   └── page.tsx
│   │   │   ├── generator/
│   │   │   │   └── page.tsx
│   │   │   ├── inventory/
│   │   │   │   ├── inventory-view.tsx
│   │   │   │   └── page.tsx
│   │   │   ├── products/
│   │   │   │   ├── [id]/
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── page.tsx
│   │   │   │   └── products-list.tsx
│   │   │   ├── scan-history/
│   │   │   │   ├── history-view.tsx
│   │   │   │   └── page.tsx
│   │   │   ├── scanner/
│   │   │   │   ├── page.tsx
│   │   │   │   └── scanner.tsx
│   │   │   ├── studio/
│   │   │   │   └── page.tsx
│   │   │   ├── upi/
│   │   │   │   ├── page.tsx
│   │   │   │   └── upi-tool.tsx
│   │   │   └── layout.tsx
│   │   ├── (auth)/
│   │   │   ├── forgot-password/
│   │   │   │   └── page.tsx
│   │   │   ├── login/
│   │   │   │   └── page.tsx
│   │   │   ├── reset-password/
│   │   │   │   └── page.tsx
│   │   │   ├── signup/
│   │   │   │   └── page.tsx
│   │   │   ├── auth-ui.tsx
│   │   │   └── layout.tsx
│   │   ├── (site)/
│   │   │   ├── about/
│   │   │   │   └── page.tsx
│   │   │   ├── contact/
│   │   │   │   └── page.tsx
│   │   │   ├── features/
│   │   │   │   └── page.tsx
│   │   │   ├── help/
│   │   │   │   └── page.tsx
│   │   │   ├── pricing/
│   │   │   │   ├── page.tsx
│   │   │   │   ├── pricing-view.tsx
│   │   │   │   └── upgrade-button.tsx
│   │   │   ├── privacy/
│   │   │   │   └── page.tsx
│   │   │   ├── report/
│   │   │   │   ├── page.tsx
│   │   │   │   └── report-form.tsx
│   │   │   ├── terms/
│   │   │   │   └── page.tsx
│   │   │   ├── tools/
│   │   │   │   └── page.tsx
│   │   │   ├── layout.tsx
│   │   │   └── page.tsx
│   │   ├── admin/
│   │   │   ├── (auth)/
│   │   │   │   ├── forgot-password/
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── login/
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── admin-auth-ui.tsx
│   │   │   │   └── layout.tsx
```

```
│   │   │   ├── (console)/
│   │   │   │   ├── analytics/
│   │   │   │   │   ├── analytics-view.tsx
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── barcodes/
│   │   │   │   │   ├── barcodes-view.tsx
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── inventory/
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── mfa/
│   │   │   │   │   ├── mfa-view.tsx
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── payments/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── payments-view.tsx
│   │   │   │   ├── products/
│   │   │   │   │   ├── catalog-view.tsx
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── qr-codes/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── qr-codes-view.tsx
│   │   │   │   ├── reports/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── reports-view.tsx
│   │   │   │   ├── security/
│   │   │   │   │   ├── admins/
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   ├── events/
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── security-view.tsx
│   │   │   │   ├── settings/
│   │   │   │   │   ├── appearance/
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   ├── branding/
│   │   │   │   │   │   ├── branding-editor.tsx
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   ├── email/
│   │   │   │   │   │   ├── email-view.tsx
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   ├── payments/
│   │   │   │   │   │   ├── page.tsx
│   │   │   │   │   │   └── payments-config.tsx
│   │   │   │   │   ├── pricing/
│   │   │   │   │   │   ├── page.tsx
│   │   │   │   │   │   └── pricing-admin.tsx
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── settings-view.tsx
│   │   │   │   ├── subscriptions/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── subscriptions-view.tsx
│   │   │   │   ├── system/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── system-view.tsx
│   │   │   │   ├── users/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── users-view.tsx
│   │   │   │   ├── layout.tsx
│   │   │   │   ├── overview-view.tsx
│   │   │   │   └── page.tsx
│   │   │   └── layout.tsx
│   │   ├── api/
│   │   │   ├── account/
│   │   │   │   ├── sessions/
│   │   │   │   │   └── route.ts
│   │   │   │   └── route.ts
│   │   │   ├── admin/
│   │   │   │   ├── analytics/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── auth/
│   │   │   │   │   ├── login/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   ├── logout/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   ├── me/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   ├── mfa/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── reauth/
│   │   │   │   │       └── route.ts
│   │   │   │   ├── branding/
```

```
│   │   │   │   │   ├── assets/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── catalog/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── email/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── logs/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── mfa/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── payments/
│   │   │   │   │   ├── config/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── plans/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   ├── reorder/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── qrcodes/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── reports/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── security/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── settings/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── stats/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── subscriptions/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── system/
│   │   │   │   │   └── route.ts
│   │   │   │   └── users/
│   │   │   │       ├── [id]/
│   │   │   │       │   └── route.ts
│   │   │   │       ├── bulk/
│   │   │   │       │   └── route.ts
│   │   │   │       └── route.ts
│   │   │   ├── analytics/
│   │   │   │   ├── export/
│   │   │   │   │   └── route.ts
│   │   │   │   └── route.ts
│   │   │   ├── auth/
│   │   │   │   ├── forgot/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── google/
│   │   │   │   │   ├── callback/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── login/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── logout/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── me/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── reset/
│   │   │   │   │   └── route.ts
│   │   │   │   └── signup/
│   │   │   │       └── route.ts
│   │   │   ├── billing/
│   │   │   │   ├── cancel/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── checkout/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── status/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── subscribe/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── verify/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── webhook/
│   │   │   │   │   └── route.ts
```

```
│   │   │   │   └── route.ts
│   │   │   ├── downloads/
│   │   │   │   └── route.ts
│   │   │   ├── health/
│   │   │   │   └── route.ts
│   │   │   ├── inventory/
│   │   │   │   └── route.ts
│   │   │   ├── overview/
│   │   │   │   └── route.ts
│   │   │   ├── products/
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── history/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   ├── stock/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   ├── import/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── lookup/
│   │   │   │   │   └── route.ts
│   │   │   │   ├── next-code/
│   │   │   │   │   └── route.ts
│   │   │   │   └── route.ts
│   │   │   ├── qrcodes/
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── duplicate/
│   │   │   │   │   │   └── route.ts
│   │   │   │   │   └── route.ts
│   │   │   │   └── route.ts
│   │   │   ├── reports/
│   │   │   │   └── route.ts
│   │   │   └── uploads/
│   │   │       └── route.ts
│   │   ├── brand/
│   │   │   └── [name]/
│   │   │       └── route.ts
│   │   ├── f/
│   │   │   └── [name]/
│   │   │       └── route.ts
│   │   ├── p/
│   │   │   └── [code]/
│   │   │       └── page.tsx
│   │   ├── r/
│   │   │   └── [code]/
│   │   │       └── route.ts
│   │   ├── globals.css
│   │   ├── icon.svg
│   │   ├── layout.tsx
│   │   ├── not-found.tsx
│   │   ├── robots.ts
│   │   └── sitemap.ts
│   ├── components/
│   │   ├── admin/
│   │   │   ├── admin-nav.ts
│   │   │   ├── admin-shell.tsx
│   │   │   ├── data.tsx
│   │   │   └── guards.tsx
│   │   ├── barcode/
│   │   │   ├── barcode-generator.tsx
│   │   │   └── barcode-ui.tsx
│   │   ├── home/
│   │   │   └── quick-generator.tsx
│   │   ├── pricing/
│   │   │   └── pricing-cards.tsx
│   │   ├── products/
│   │   │   └── product-editor.tsx
│   │   ├── qr/
│   │   │   ├── code-type-selector.tsx
│   │   │   ├── design-panel.tsx
│   │   │   ├── export-panel.tsx
│   │   │   ├── preview.tsx
│   │   │   ├── save-dialog.tsx
│   │   │   ├── type-form.tsx
│   │   │   └── workbench.tsx
│   │   ├── analytics-view.tsx
│   │   ├── app-shell.tsx
│   │   ├── brand.tsx
│   │   ├── charts.tsx
│   │   ├── command-palette.tsx
│   │   ├── generator-tabs.tsx
│   │   ├── hosted-page.tsx
```

```
│   │   ├── icon.tsx
│   │   ├── legal.tsx
│   │   ├── marketing.tsx
│   │   ├── nav-config.ts
│   │   ├── providers.tsx
│   │   ├── site-chrome.tsx
│   │   ├── ui.tsx
│   │   └── user-menu.tsx
│   ├── lib/
│   │   ├── admin-auth.ts
│   │   ├── admin-plans.ts
│   │   ├── admin-query.ts
│   │   ├── admin-users.ts
│   │   ├── analytics.ts
│   │   ├── api.ts
│   │   ├── audit.ts
│   │   ├── auth-flow.ts
│   │   ├── auth-helpers.ts
│   │   ├── barcode-export.ts
│   │   ├── barcodes.ts
│   │   ├── billing.ts
│   │   ├── brand.ts
│   │   ├── branding.ts
│   │   ├── config.ts
│   │   ├── content-safety.ts
│   │   ├── db.ts
│   │   ├── entitlements.ts
│   │   ├── env-check.ts
│   │   ├── fetcher.ts
│   │   ├── guard.ts
│   │   ├── image-check.ts
│   │   ├── log.ts
│   │   ├── logger.ts
│   │   ├── logos.ts
│   │   ├── mail.ts
│   │   ├── mfa.ts
│   │   ├── password-reset.ts
│   │   ├── payment-config.ts
│   │   ├── plan-store.ts
│   │   ├── plans.ts
│   │   ├── poster.ts
│   │   ├── preferences.ts
│   │   ├── products.ts
│   │   ├── qr-export.ts
│   │   ├── qr-render.ts
│   │   ├── qr-service.ts
│   │   ├── qr-types.ts
│   │   ├── rate-limit.ts
│   │   ├── razorpay.ts
│   │   ├── redact.ts
│   │   ├── roles.ts
│   │   ├── safe-redirect.ts
│   │   ├── scan-classify.ts
│   │   ├── scan-history.ts
│   │   ├── scan-merge.ts
│   │   ├── scans.ts
│   │   ├── secret-box.ts
│   │   ├── session.ts
│   │   ├── settings.ts
│   │   ├── storage.ts
│   │   ├── system-status.ts
│   │   ├── table-file.ts
│   │   ├── templates.ts
│   │   ├── ua.ts
│   │   ├── upi.ts
│   │   ├── utils.ts
│   │   └── zxing-client.ts
│   ├── instrumentation.ts
│   └── middleware.ts
├── .env.example
├── .gitignore
├── .nvmrc
├── CODE_OF_CONDUCT.md
├── CONTRIBUTING.md
├── eslint.config.mjs
├── next.config.mjs
├── package-lock.json
├── package.json
├── postcss.config.mjs
├── README.md
├── SECURITY.md
```

```
├── tailwind.config.ts
└── tsconfig.json
```

Appendix B: API Endpoint Inventory

Methods and guards were extracted automatically from each `route.ts` file at build time; purposes were written by the auditor after reading each handler. "Guard" lists the authentication/authorization and rate-limit calls found in the file. All `/api` POST/PUT/PATCH/DELETE requests (except the webhook) must come from the application's own origin.

Table B.1: API endpoint inventory

#	Route	Methods	Guard	Purpose	Source
1	`/api/account`	GET, PATCH, DELETE	user, rate-limited	GET own profile; PATCH name/password (current password required; other sessions revoked); DELETE account (type e-mail to confirm)	`src/app/api/account/route.ts`
2	`/api/account/sessions`	POST	user	POST sign out all other devices	`src/app/api/account/sessions/route.ts`
3	`/api/admin/analytics`	GET	admin	Platform analytics (admin)	`src/app/api/admin/analytics/route.ts`
4	`/api/admin/auth/login`	POST	public	Admin sign-in step 1 (password); returns MFA ticket when MFA enabled	`src/app/api/admin/auth/login/route.ts`
5	`/api/admin/auth/logout`	POST	admin session (checked in handler)	End admin-console session	`src/app/api/admin/auth/logout/route.ts`
6	`/api/admin/auth/me`	GET	admin session (checked in handler)	Current administrator	`src/app/api/admin/auth/me/route.ts`
7	`/api/admin/auth/mfa`	POST	rate-limited	Admin sign-in step 2 (TOTP or recovery code)	`src/app/api/admin/auth/mfa/route.ts`
8	`/api/admin/auth/reauth`	POST	admin session (checked in handler), rate-limited	Re-confirm password (+TOTP) for sensitive actions	`src/app/api/admin/auth/reauth/route.ts`
9	`/api/admin/branding`	GET, PUT, POST	admin, rate-limited	GET branding state; PUT draft; POST publish / discard / reset	`src/app/api/admin/branding/route.ts`
10	`/api/admin/branding/assets`	POST	admin, rate-limited	Upload logo/favicon/social image (validated)	`src/app/api/admin/branding/assets/route.ts`
11	`/api/admin/catalog`	GET	admin	Products and barcodes overview (admin)	`src/app/api/admin/catalog/route.ts`
12	`/api/admin/email`	GET, POST	admin, rate-limited	GET SMTP status (no credentials); POST test mail to own address	`src/app/api/admin/email/route.ts`
13	`/api/admin/logs`	GET	admin	System log entries	`src/app/api/admin/logs/route.ts`
14	`/api/admin/mfa`	GET, POST	admin (MFA not required), rate-limited	GET MFA status; POST setup / enable / disable / regenerate recovery codes	`src/app/api/admin/mfa/route.ts`
15	`/api/admin/payments`	GET	admin	Payment records	`src/app/api/admin/payments/route.ts`
16	`/api/admin/payments/config`	GET, PUT, PATCH, POST	admin, super admin, recent re-auth (super admin), rate-limited	GET gateway status (never secrets); PUT credentials (super admin + re-auth); PATCH enable/mode; POST verify	`src/app/api/admin/payments/config/route.ts`
17	`/api/admin/plans`	GET, POST	admin, super admin, rate-limited	GET plans; POST create plan (super admin)	`src/app/api/admin/plans/route.ts`
18	`/api/admin/plans/[id]`	PATCH, POST	super admin, rate-limited	PATCH edit plan (confirm if live subscribers); POST action: duplicate, publish, unpublish, archive, restore or sync provider plans	`src/app/api/admin/plans/[id]/route.ts`

#	Route	Methods	Guard	Purpose	Source
19	`/api/admin/plans/reorder`	POST	super admin	Reorder plans (super admin)	`src/app/api/admin/plans/reorder/route.ts`
20	`/api/admin/qrcodes`	GET	admin	List dynamic and hosted QR codes	`src/app/api/admin/qrcodes/route.ts`
21	`/api/admin/qrcodes/[id]`	PATCH, DELETE	admin	PATCH status active/paused/disabled; DELETE QR code	`src/app/api/admin/qrcodes/[id]/route.ts`
22	`/api/admin/reports`	GET	admin	List abuse reports	`src/app/api/admin/reports/route.ts`
23	`/api/admin/reports/[id]`	PATCH	admin	PATCH report status: open, resolved or dismissed (audited)	`src/app/api/admin/reports/[id]/route.ts`
24	`/api/admin/security`	GET	admin	Audit log and security events	`src/app/api/admin/security/route.ts`
25	`/api/admin/settings`	GET, PUT	admin, rate-limited	GET/PUT general settings	`src/app/api/admin/settings/route.ts`
26	`/api/admin/stats`	GET	admin	Overview statistics	`src/app/api/admin/stats/route.ts`
27	`/api/admin/subscriptions`	GET	admin	List subscriptions	`src/app/api/admin/subscriptions/route.ts`
28	`/api/admin/subscriptions/[id]`	POST	super admin, rate-limited	Cancel a subscription (super admin)	`src/app/api/admin/subscriptions/[id]/route.ts`
29	`/api/admin/system`	GET	admin	Configuration and health status	`src/app/api/admin/system/route.ts`
30	`/api/admin/users`	GET	admin	Search and page users (no secrets)	`src/app/api/admin/users/route.ts`
31	`/api/admin/users/[id]`	PATCH, POST, DELETE	admin, super admin	PATCH role/plan/status; POST revoke sessions / send reset; DELETE account (super admin)	`src/app/api/admin/users/[id]/route.ts`
32	`/api/admin/users/bulk`	POST	admin	Bulk suspend / reactivate / revoke sessions (≤100 ids)	`src/app/api/admin/users/bulk/route.ts`
33	`/api/analytics`	GET	user	Scan statistics limited to plan's analyticsDays	`src/app/api/analytics/route.ts`
34	`/api/analytics/export`	GET	user, plan feature	CSV export of scans (requires analyticsExport feature)	`src/app/api/analytics/export/route.ts`
35	`/api/auth/forgot`	POST	rate-limited	Request password-reset e-mail (same response for any e-mail)	`src/app/api/auth/forgot/route.ts`
36	`/api/auth/google`	GET	public	Start Google OAuth flow	`src/app/api/auth/google/route.ts`
37	`/api/auth/google/callback`	GET	public	Google OAuth callback (state check, verified e-mail)	`src/app/api/auth/google/callback/route.ts`
38	`/api/auth/login`	POST	public	Workspace sign-in	`src/app/api/auth/login/route.ts`
39	`/api/auth/logout`	POST	public	Workspace sign-out	`src/app/api/auth/logout/route.ts`
40	`/api/auth/me`	GET	optional user	Current user (or null)	`src/app/api/auth/me/route.ts`
41	`/api/auth/reset`	POST	rate-limited	Set new password with a reset token	`src/app/api/auth/reset/route.ts`

#	Route	Methods	Guard	Purpose	Source
42	`/api/auth/signup`	POST	rate-limited	Create account (role always 'user')	`src/app/api/auth/signup/route.ts`
43	`/api/billing`	GET	user	Own billing overview and payment history	`src/app/api/billing/route.ts`
44	`/api/billing/cancel`	POST	user, rate-limited	Cancel own subscription (at period end by default)	`src/app/api/billing/cancel/route.ts`
45	`/api/billing/status`	GET	user	Status of a subscription after checkout	`src/app/api/billing/status/route.ts`
46	`/api/billing/subscribe`	POST	user, rate-limited	Start subscription checkout (planId + interval only)	`src/app/api/billing/subscribe/route.ts`
47	`/api/billing/verify`	POST	user, rate-limited	Verify Checkout signature and confirm payment	`src/app/api/billing/verify/route.ts`
48	`/api/billing/webhook`	POST	HMAC signature	Razorpay webhook (HMAC over raw body; CSRF-exempt)	`src/app/api/billing/webhook/route.ts`
49	`/api/downloads`	GET, POST, DELETE	user	Record / list / clear download history	`src/app/api/downloads/route.ts`
50	`/api/health`	GET	public	Liveness/readiness: {status: ok|degraded}	`src/app/api/health/route.ts`
51	`/api/inventory`	GET	user	Inventory summary and low-stock list	`src/app/api/inventory/route.ts`
52	`/api/overview`	GET	user	Dashboard counts (real data)	`src/app/api/overview/route.ts`
53	`/api/products`	GET, POST	user	GET list/search/filter; POST create product	`src/app/api/products/route.ts`
54	`/api/products/[id]`	GET, PATCH, DELETE	user	GET / PATCH / DELETE product (delete only when archived)	`src/app/api/products/[id]/route.ts`
55	`/api/products/[id]/history`	GET	user	Inventory ledger of a product	`src/app/api/products/[id]/history/route.ts`
56	`/api/products/[id]/stock`	POST	user	POST stock in / out / adjust	`src/app/api/products/[id]/stock/route.ts`
57	`/api/products/import`	POST	user	Import products from parsed CSV/XLSX rows (skip/update)	`src/app/api/products/import/route.ts`
58	`/api/products/lookup`	GET	user	Find product by scanned code or SKU (read-only)	`src/app/api/products/lookup/route.ts`
59	`/api/products/next-code`	GET	user	Generate unused internal barcode number	`src/app/api/products/next-code/route.ts`
60	`/api/qrcodes`	GET, POST	user, rate-limited	GET list/search/filter; POST create QR code	`src/app/api/qrcodes/route.ts`
61	`/api/qrcodes/[id]`	GET, PATCH, DELETE	user	GET / PATCH / DELETE own QR code	`src/app/api/qrcodes/[id]/route.ts`
62	`/api/qrcodes/[id]/duplicate`	POST	user	Duplicate a QR code (quota-checked)	`src/app/api/qrcodes/[id]/duplicate/route.ts`
63	`/api/reports`	POST	rate-limited	Public abuse report (rate-limited)	`src/app/api/reports/route.ts`
64	`/api/uploads`	GET, POST, DELETE	user, rate-limited	GET list; POST upload (multipart); DELETE ?id=	`src/app/api/uploads/route.ts`
65	`/brand/[name]`	GET	public	Public branding image (immutable, sandboxed SVG)	`src/app/brand/[name]/route.ts`

#	Route	Methods	Guard	Purpose	Source
66	`/f/[name]`	GET	public	Uploaded file with safe headers	`src/app/f/[name]/route.ts`
67	`/r/[code]`	GET	public	Dynamic QR redirect (records scan)	`src/app/r/[code]/route.ts`

Appendix C: Database Data Dictionary

Generated from `prisma/schema.prisma`; the MySQL column type is taken from the generated `prisma/mysql/schema.prisma` (Prisma's default mapping `VARCHAR(191)` applies where no explicit type is given). No data from any database was read for this dictionary.

User

Table C.1: Data dictionary — User

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
email	String	VARCHAR(191)	No	UNIQUE		
name	String	VARCHAR(191)	No		""	
passwordHash	String	VARCHAR(191)	Yes			
googleId	String	VARCHAR(191)	Yes	UNIQUE		
role	String	VARCHAR(191)	No		"user"	user / admin / super_admin (assigned only server-side)
plan	String	VARCHAR(191)	No		"free"	free / pro / business
planExpiresAt	DateTime	DATETIME(3)	Yes			null = no expiry (free or admin-assigned)
status	String	VARCHAR(191)	No		"active"	active / suspended
sessionVersion	Int	INT	No		0	
createdAt	DateTime	DATETIME(3)	No		now()	
updatedAt	DateTime	DATETIME(3)	No		auto (updatedAt)	
lastLoginAt	DateTime	DATETIME(3)	Yes			
mfaSecret	String	TEXT	Yes			
mfaEnabledAt	DateTime	DATETIME(3)	Yes			
mfaLastStep	Int	INT	No		0	last accepted TOTP time-step (replay protection)
quotaLock	Int	INT	No		0	bumped inside quota transactions to serialise concurrent creates per user

Relations: `sessions` → Session (one-to-many); `subscriptions` → Subscription (one-to-many); `recoveryCodes` → MfaRecoveryCode (one-to-many); `qrCodes` → QrCode (one-to-many); `downloads` → Download (one-to-many); `uploads` → Upload (one-to-many); `resets` → PasswordReset (one-to-many); `payments` → Payment (one-to-many); `products` → Product (one-to-many); `barcodes` → Barcode (one-to-many); `inventoryTx` → InventoryTransaction (one-to-many).

PasswordReset

Table C.2: Data dictionary — PasswordReset

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
tokenHash	String	VARCHAR(191)	No	UNIQUE		
expiresAt	DateTime	DATETIME(3)	No			
usedAt	DateTime	DATETIME(3)	Yes			
createdAt	DateTime	DATETIME(3)	No		now()	

Relations: `user` → User (many-to-one), on delete Cascade.

QrCode

Table C.3: Data dictionary — QrCode

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
name	String	VARCHAR(191)	No			
type	String	VARCHAR(191)	No			
content	String	MEDIUMTEXT	No			JSON: form data for the QR type
design	String	MEDIUMTEXT	No			JSON: design settings
payload	String	TEXT	No			exact text encoded in the QR image
isDynamic	Boolean	TINYINT(1)	No		false	
isHosted	Boolean	TINYINT(1)	No		false	landing page served at /p/[shortCode]
shortCode	String	VARCHAR(191)	Yes	UNIQUE		
destination	String	TEXT	Yes			dynamic redirect target
status	String	VARCHAR(191)	No		"active"	active / paused / disabled (admin)
expiresAt	DateTime	DATETIME(3)	Yes			
utm	String	TEXT	Yes			JSON: { source, medium, campaign, term, content }
campaign	String	VARCHAR(191)	No		""	
trackScans	Boolean	TINYINT(1)	No		true	
favorite	Boolean	TINYINT(1)	No		false	
isDemo	Boolean	TINYINT(1)	No		false	
createdAt	DateTime	DATETIME(3)	No		now()	
updatedAt	DateTime	DATETIME(3)	No		auto (updatedAt)	

Relations: `user` → User (many-to-one), on delete Cascade; `scans` → Scan (one-to-many); `downloads` → Download (one-to-many).

Composite constraints / indexes: `@@index([userId])`.

Scan

Table C.4: Data dictionary — Scan

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
qrId	String	VARCHAR(191)	No	FK → QrCode.id		
createdAt	DateTime	DATETIME(3)	No		now()	
device	String	VARCHAR(191)	No		"unknown"	mobile / tablet / desktop / bot / unknown
browser	String	VARCHAR(191)	No		"unknown"	
os	String	VARCHAR(191)	No		"unknown"	
country	String	VARCHAR(191)	No		""	ISO code from hosting platform header, if any
visitorHash	String	VARCHAR(191)	No		""	salted daily hash, never raw IP
isDemo	Boolean	TINYINT(1)	No		false	

Relations: `qr` → QrCode (many-to-one), on delete Cascade.

Composite constraints / indexes: `@@index([qrId, createdAt])`.

Download

Table C.5: Data dictionary — Download

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
qrId	String	VARCHAR(191)	Yes	FK → QrCode.id		
name	String	VARCHAR(191)	No			
format	String	VARCHAR(191)	No			
createdAt	DateTime	DATETIME(3)	No		now()	

Relations: `user` → User (many-to-one), on delete Cascade; `qr` → QrCode (optional many-to-one), on delete SetNull.

Composite constraints / indexes: `@@index([userId, createdAt])`.

Upload

Table C.6: Data dictionary — Upload

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
filename	String	VARCHAR(255)	No			
storedName	String	VARCHAR(191)	No	UNIQUE		
mime	String	VARCHAR(191)	No			
size	Int	INT	No			
createdAt	DateTime	DATETIME(3)	No		now()	

Relations: `user` → User (many-to-one), on delete Cascade.

AbuseReport

Table C.7: Data dictionary — AbuseReport

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
shortCode	String	VARCHAR(191)	No		""	
url	String	VARCHAR(2000)	No		""	
reason	String	VARCHAR(191)	No			
details	String	VARCHAR(2000)	No		""	
reporterEmail	String	VARCHAR(191)	No		""	
status	String	VARCHAR(191)	No		"open"	open / resolved / dismissed
createdAt	DateTime	DATETIME(3)	No		now()	

Payment

Table C.8: Data dictionary — Payment *One money movement. Amount, currency and plan are snapshots taken when the payment happened — later plan or price edits never rewrite history.*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
provider	String	VARCHAR(191)	No			razorpay

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
providerRef	String	VARCHAR(191)	No	UNIQUE		payment link id (legacy) or razorpay payment id
kind	String	VARCHAR(191)	No		"payment_link"	payment_link / subscription
plan	String	VARCHAR(191)	No			plan id snapshot
planName	String	VARCHAR(191)	No		""	display-name snapshot
interval	String	VARCHAR(191)	No		""	month / year / "" (30-day pass)
amount	Int	INT	No			minor units (paise)
currency	String	VARCHAR(191)	No		"INR"	
status	String	VARCHAR(191)	No		"created"	created / pending / paid / failed / expired / refunded / partially_refunded
refundedAmount	Int	INT	No		0	
failureReason	String	VARCHAR(500)	No		""	
subscriptionId	String	VARCHAR(191)	Yes	FK → Subscription.id		
createdAt	DateTime	DATETIME(3)	No		now()	
updatedAt	DateTime	DATETIME(3)	No		auto (updatedAt)	

Relations: `user` → User (many-to-one), on delete Cascade; `subscription` → Subscription (optional many-to-one), on delete SetNull.

Composite constraints / indexes: `@@index([userId, createdAt])`, `@@index([subscriptionId])`.

PricingPlan

Table C.9: Data dictionary — PricingPlan *Admin-managed plan definition (catalogue). Entitlements live in `limits` (JSON). Plans are archived, never deleted, so history and existing subscribers keep a reference.*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK		internal identifier, e.g. "pro"
name	String	VARCHAR(191)	No			
description	String	VARCHAR(500)	No		""	
currency	String	VARCHAR(191)	No		"INR"	
monthlyPrice	Int	INT	No		0	minor units; 0 = not offered monthly (paid plans)
annualPrice	Int	INT	No		0	minor units per year; 0 = not offered yearly
features	String	VARCHAR(4000)	No		"[]"	JSON string[] shown on pricing cards
limits	String	VARCHAR(4000)	No		"{}"	JSON entitlements (see src/lib/plans.ts)
featured	Boolean	TINYINT(1)	No		false	
status	String	VARCHAR(191)	No		"draft"	draft / published / archived
sortOrder	Int	INT	No		0	
createdAt	DateTime	DATETIME(3)	No		now()	
updatedAt	DateTime	DATETIME(3)	No		auto (updatedAt)	

Relations: `providerPlans` → ProviderPlan (one-to-many).

ProviderPlan

Table C.10: Data dictionary — ProviderPlan *A Razorpay plan created for one (plan, interval, price, currency, mode). Price changes create a new provider plan; existing subscriptions keep theirs.*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
planId	String	VARCHAR(191)	No	FK → PricingPlan.id		
interval	String	VARCHAR(191)	No			month / year
amount	Int	INT	No			minor units
currency	String	VARCHAR(191)	No			
mode	String	VARCHAR(191)	No			test / live
providerPlanId	String	VARCHAR(191)	No	UNIQUE		
active	Boolean	TINYINT(1)	No		true	used for new checkouts
createdAt	DateTime	DATETIME(3)	No		now()	

Relations: `plan` → PricingPlan (many-to-one), on delete Restrict.

Composite constraints / indexes: `@@index([planId, interval, mode, active])`.

Subscription

Table C.11: Data dictionary — Subscription *Customer subscription (Razorpay Subscriptions). Status mirrors the provider and changes only from verified server-side confirmations (API fetch or signed webhook).*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
planId	String	VARCHAR(191)	No			PricingPlan id snapshot
planName	String	VARCHAR(191)	No		""	
interval	String	VARCHAR(191)	No			month / year
amount	Int	INT	No			minor units per period (snapshot)
currency	String	VARCHAR(191)	No		"INR"	
provider	String	VARCHAR(191)	No		"razorpay"	
mode	String	VARCHAR(191)	No		"test"	test / live
providerSubscriptionId	String	VARCHAR(191)	No	UNIQUE		
providerPlanId	String	VARCHAR(191)	No			
status	String	VARCHAR(191)	No		"created"	created / authenticated / active / pending / halted / cancelled / completed / expired
currentPeriodEnd	DateTime	DATETIME(3)	Yes			
cancelAtPeriodEnd	Boolean	TINYINT(1)	No		false	
createdAt	DateTime	DATETIME(3)	No		now()	
updatedAt	DateTime	DATETIME(3)	No		auto (updatedAt)	

Relations: `user` → User (many-to-one), on delete Cascade; `payments` → Payment (one-to-many).

Composite constraints / indexes: `@@index([userId, status])`.

WebhookEvent

Table C.12: Data dictionary — WebhookEvent *Ledger of processed provider webhooks. The unique key makes duplicate deliveries no-ops.*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
provider	String	VARCHAR(191)	No			
eventId	String	VARCHAR(191)	No			

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
eventType	String	VARCHAR(191)	No			
status	String	VARCHAR(191)	No		"processed"	processed / ignored / rejected
summary	String	VARCHAR(500)	No		""	short redacted description, never the raw payload
receivedAt	DateTime	DATETIME(3)	No		now()	

Composite constraints / indexes: `@@unique([provider, eventId])`, `@@index([receivedAt])`.

BrandAsset

Table C.13: Data dictionary — BrandAsset *Uploaded branding image (logo, favicon, social image). Served from /brand/[name].*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
kind	String	VARCHAR(191)	No			logo / logoDark / favicon / ogImage
storedName	String	VARCHAR(191)	No	UNIQUE		
mime	String	VARCHAR(191)	No			
width	Int	INT	No		0	
height	Int	INT	No		0	
size	Int	INT	No			
createdBy	String	VARCHAR(191)	No		""	
createdAt	DateTime	DATETIME(3)	No		now()	

Composite constraints / indexes: `@@index([kind, createdAt])`.

Log

Table C.14: Data dictionary — Log

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
level	String	VARCHAR(191)	No			info / warn / error
event	String	VARCHAR(191)	No			
message	String	VARCHAR(2000)	No		""	
userId	String	VARCHAR(191)	Yes			
createdAt	DateTime	DATETIME(3)	No		now()	

Composite constraints / indexes: `@@index([createdAt])`.

Setting

Table C.15: Data dictionary — Setting

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
key	String	VARCHAR(191)	No	PK		
value	String	TEXT	No			

Product

Table C.16: Data dictionary — Product

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
name	String	VARCHAR(191)	No			

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
sku	String	VARCHAR(191)	No			
productCode	String	VARCHAR(191)	No		""	internal product ID
brand	String	VARCHAR(191)	No		""	
category	String	VARCHAR(191)	No		""	
variant	String	VARCHAR(191)	No		""	size / colour / variant
description	String	VARCHAR(2000)	No		""	
manufacturer	String	VARCHAR(191)	No		""	
imageUrl	String	VARCHAR(1000)	No		""	
priceMinor	Int	INT	No		0	price in minor units (paise)
currency	String	VARCHAR(191)	No		"INR"	
stock	Int	INT	No		0	
lowStockAt	Int	INT	No		5	
batch	String	VARCHAR(191)	No		""	
expiryDate	DateTime	DATETIME(3)	Yes			
status	String	VARCHAR(191)	No		"active"	active / archived
createdAt	DateTime	DATETIME(3)	No		now()	
updatedAt	DateTime	DATETIME(3)	No		auto (updatedAt)	

Relations: `user` → User (many-to-one), on delete Cascade; `barcodes` → Barcode (one-to-many); `transactions` → InventoryTransaction (one-to-many).

Composite constraints / indexes: `@@unique([userId, sku])`, `@@index([userId, status])`, `@@index([userId, category])`.

Barcode

Table C.17: Data dictionary — Barcode

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
productId	String	VARCHAR(191)	No	FK → Product.id		
format	String	VARCHAR(32)	No			id from src/lib/barcodes.ts
value	String	VARCHAR(1500)	No			exact encoded text
lookupKey	String	VARCHAR(1500)	No			canonical key for scan matching (GTIN-14 for EAN/UPC family)
source	String	VARCHAR(191)	No		"internal"	internal / assigned (GS1-allocated) / existing
isPrimary	Boolean	TINYINT(1)	No		false	
createdAt	DateTime	DATETIME(3)	No		now()	

Relations: `user` → User (many-to-one), on delete Cascade; `product` → Product (many-to-one), on delete Cascade.

Composite constraints / indexes: `@@unique([userId, format, value])`, `@@index([userId, lookupKey])`, `@@index([productId])`.

InventoryTransaction

Table C.18: Data dictionary — InventoryTransaction

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
userId	String	VARCHAR(191)	No	FK → User.id		
productId	String	VARCHAR(191)	No	FK → Product.id		
type	String	VARCHAR(191)	No			created / in / out / adjust / updated / archived / restored / imported
quantity	Int	INT	No		0	signed change in stock
balance	Int	INT	No		0	stock after this entry
note	String	VARCHAR(500)	No		""	
reference	String	VARCHAR(191)	No		""	
createdAt	DateTime	DATETIME(3)	No		now()	

Relations: `user` → User (many-to-one), on delete Cascade; `product` → Product (many-to-one), on delete Cascade.

Composite constraints / indexes: `@@index([productId, createdAt])`, `@@index([userId, createdAt])`.

Session

Table C.19: Data dictionary — Session *Server-side session. The cookie holds a signed token naming this row, so logout and revocation take effect immediately.*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
kind	String	VARCHAR(191)	No		"user"	user / admin (admin = created through /admin/login)
mfa	Boolean	TINYINT(1)	No		false	MFA verified for this session
createdAt	DateTime	DATETIME(3)	No		now()	
expiresAt	DateTime	DATETIME(3)	No			
lastSeenAt	DateTime	DATETIME(3)	No		now()	
reauthAt	DateTime	DATETIME(3)	Yes			last password(+MFA) re-confirmation, for sensitive admin actions
userAgent	String	VARCHAR(255)	No		""	

Relations: `user` → User (many-to-one), on delete Cascade.

Composite constraints / indexes: `@@index([userId])`, `@@index([expiresAt])`.

MfaRecoveryCode

Table C.20: Data dictionary — MfaRecoveryCode

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
userId	String	VARCHAR(191)	No	FK → User.id		
codeHash	String	VARCHAR(191)	No	UNIQUE		SHA-256 of a high-entropy one-time code
usedAt	DateTime	DATETIME(3)	Yes			
createdAt	DateTime	DATETIME(3)	No		now()	

Relations: `user` → User (many-to-one), on delete Cascade.

Composite constraints / indexes: `@@index([userId])`.

AuditLog

Table C.21: Data dictionary — AuditLog *Security-relevant events and administrator actions (secrets never stored).*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
id	String	VARCHAR(191)	No	PK	cuid()	
actorId	String	VARCHAR(191)	Yes			
actorEmail	String	VARCHAR(191)	No		""	
action	String	VARCHAR(191)	No			e.g. admin.user.suspend, auth.login_failed, auth.mfa_enabled
targetType	String	VARCHAR(191)	No		""	
targetId	String	VARCHAR(191)	No		""	
details	String	VARCHAR(4000)	No		""	JSON, redacted
ip	String	VARCHAR(191)	No		""	truncated / hashed per privacy settings
createdAt	DateTime	DATETIME(3)	No		now()	

Composite constraints / indexes: `@@index([createdAt])`, `@@index([action])`, `@@index([actorId])`.

RateLimit

Table C.22: Data dictionary — RateLimit *Fixed-window counters for RATE_LIMIT_STORE=database (multi-instance deployments).*

Field	Prisma type	MySQL column	Null	Key / constraint	Default	Notes
key	String	VARCHAR(191)	No	PK		
count	Int	INT	No		0	
resetAt	DateTime	DATETIME(3)	No			

Composite constraints / indexes: `@@index([resetAt])`.

Appendix D: Detailed Test Cases

Status values: **Pass** — executed and met the expected result; **Fail** — executed and did not; **Not executed** — documented procedure, not run (reason given). Evidence quotes the assertion text printed by the suite in the final run, or names the audit probe.

Table D.1: Detailed test cases

Test case ID	Module	Test scenario	Preconditions	Test steps	Input	Expected result	Actual result	Status	Evidence or reason
TC-AUTH-01	Auth	Sign-up creates an ordinary user even if a role is injected	Sign-ups enabled	POST /api/auth/signup with extra field role=super_admin	name, e-mail, valid password, role=super_admin	201; DB role = user	201; role stored as 'user'	Pass	test:admin: "sign-up with injected role succeeds as a normal user"; test:admin: "database row has role 'user'"
TC-AUTH-02	Auth	Sign-up response and session cookie are safe	—	Sign up; inspect response body and Set-Cookie	valid data	No hash/token fields; cookie HttpOnly, SameSite=Lax	As expected	Pass	test:admin: "sign-up response has no hash, token or secret fields"; test:admin: "workspace cookie is HttpOnly + SameSite=Lax"
TC-AUTH-03	Auth	Duplicate e-mail rejected at sign-up	Account exists	POST /api/auth/signup with the same e-mail	existing e-mail	409 Conflict	409 'An account with this email already exists. Try signing in.'	Pass	Audit probe (probe-cases.mts) on the final isolated build: duplicate e-mail → 409
TC-AUTH-04	Auth	Weak password rejected	—	POST /api/auth/signup with 'abcdefgh'	8 letters, no digit	400 'Use at least one letter and one number'	400 'password: Use at least one letter and one number'	Pass	Audit probe (probe-cases.mts) on the final isolated build: weak password → 400
TC-AUTH-05	Auth	Private API requires sign-in	No session	GET/POST /api/products, /lookup, /inventory, /overview, /import	no cookie	401 for each	401 for each	Pass	test:api: "GET /api/products requires sign-in (401)" (+5 similar)
TC-AUTH-06	Auth	Cross-site state-changing request blocked	Signed in	POST with Origin of another site	Origin: https://evil.example	403	403	Pass	test:api: "cross-site POST blocked (403)"
TC-AUTH-07	Auth	Generic error for wrong password, unknown e-mail and non-admin	Accounts exist	POST /api/admin/auth/login three ways	wrong pw; unknown e-mail; ordinary user	Identical 401 message	Identical	Pass	test:admin: "same generic error for non-admin, wrong password and unknown e-mail"
TC-AUTH-08	Auth	Session fixation prevented	Signed in	Sign in again in same browser	—	New session id; old revoked	As expected	Pass	test:admin: "re-login issues a new session id (fixation)"; test:admin: "the replaced session is revoked"
TC-AUTH-09	Auth	Logout invalidates token server-side	Signed in	POST /api/auth/logout; reuse old cookie	old cookie	401	401	Pass	test:admin: "user logout invalidates the token server-side"
TC-AUTH-10	Auth	Suspended user cannot sign in; sessions end immediately	Admin suspends user	Use old cookie; then sign in	valid credentials	401 on old session; 403 on login	As expected	Pass	test:admin: "suspension ends the user's sessions immediately"; test:admin: "suspended user cannot sign in"
TC-AUTH-11	Auth	Forgot-password does not reveal accounts	—	POST /api/auth/forgot for existing and unknown e-mail	two e-mails	Identical responses	Identical	Pass	test:admin: "forgot-password: identical response for existing and unknown e-mail"
TC-AUTH-12	Auth	Reset token works once and is stored hashed	Token issued	POST /api/auth/reset twice	valid token, new password	1st 200; 2nd 400; DB holds only hash	As expected	Pass	test:admin: "reset token is single-use"; test:admin: "token is stored only as a hash"
TC-AUTH-13	Auth	Expired reset token rejected	Token past expiresAt	POST /api/auth/reset	expired token	400	400	Pass	test:admin: "expired token rejected"
TC-AUTH-14	Auth	Reset signs out existing sessions	Two sessions	Reset password; reuse other session	—	Other session 401	As expected	Pass	test:admin: "reset signs out existing sessions"
TC-AUTH-15	Account	Password change signs out other devices but keeps current	Two sessions	PATCH /api/account with current + new password	valid passwords	Other session 401; current works	As expected	Pass	test:admin: "password change signs out other sessions"; test:admin: "…but keeps the current browser signed in"

Test case ID	Module	Test scenario	Preconditions	Test steps	Input	Expected result	Actual result	Status	Evidence or reason
TC-AUTH-16	Auth	Brute-force limit on admin sign-in	—	6 failed admin sign-ins for one account	wrong password ×6	6th → 429	429	Pass	test:admin: "6th admin sign-in attempt for one account → 429"
TC-AUTH-17	Auth	Google sign-in (OAuth) end-to-end	Google client configured	Click 'Continue with Google', consent, return	Google account	Account created/linked; session	Not run (no Google credentials in audit)	Not executed	Requires live Google OAuth client
TC-QR-01	QR	Every QR type and style renders and decodes	—	npm run test:qr renders each type/style and decodes with a scanner	type defaults	Decoded text equals payload	'All QR checks passed'	Pass	test:qr (npm test) output: All QR checks passed
TC-QR-02	QR	Generator page loads	Server running	GET /generator	—	200	200	Pass	test:api: "GET /generator → 200"
TC-QR-03	QR	Generator page fits mobile/tablet/desktop with no CSP errors	Browser	Load /generator at 375/768/1280 px	—	No overflow, no CSP violation	As expected	Pass	test:browser: "375px /generator fits the screen" (+ 768/1280, CSP)
TC-QR-04	QR	User B cannot read or delete A's QR code	Two users	GET/DELETE /api/qrcodes/{A's id} as B	—	404	404	Pass	test:admin: "user B can't read A's QR code (404)"; test:admin: "user B can't delete A's QR code"
TC-QR-05	QR	Dynamic-QR quota enforced under concurrency	Free plan (5 dynamic)	Concurrent POST /api/qrcodes dynamic=true beyond the limit	valid dynamic QRs	At most 5 created; clear 403	As expected	Pass	test:billing: "concurrent creates can't exceed the Free dynamic limit (5)"; test:billing: "over-limit request gets a clear 403"
TC-QR-06	QR	Custom alias collision / reserved word	Pro plan	POST /api/qrcodes alias='admin'	alias=admin	400 reserved	400 'That alias is reserved. Try another.'	Pass	Audit probe (probe-cases.mts) on the final isolated build: reserved alias 'admin' → 400
TC-QR-07	UPI	UPI QR saved with correct URI	Signed in	POST /api/qrcodes type=upi	valid VPA, amount	upi://pay?… payload	As expected	Pass	test:api: "UPI QR saved with correct URI"
TC-QR-08	UPI	Invalid UPI ID rejected	Signed in	POST /api/qrcodes type=upi	pa='not-a-vpa'	400	400	Pass	test:api: "invalid UPI ID rejected"
TC-DYN-01	Dynamic QR	Dynamic QR created	Signed in	POST /api/qrcodes dynamic=true	https URL	201 with /r/{code} payload	201	Pass	test:api: "dynamic QR created"
TC-DYN-02	Dynamic QR	Dynamic QR redirects	Code exists	GET /r/{code}	—	302 to destination	302	Pass	test:api: "dynamic QR redirects (302)"
TC-DYN-03	Dynamic QR	Paused / expired code shows 410 page	Code paused or expired	GET /r/{code}	—	410 explanatory page	Paused → 410; expired (2.5 s expiry) → 410; unknown code → 404	Pass	Audit probe (probe-cases.mts) on the final isolated build: paused 410, expired 410, unknown 404
TC-DYN-04	Analytics	Scan recording capped per client	Code exists	25 visits from one client	—	All redirected; 20 scans stored	As expected	Pass	test:admin: "scan recording capped per client (25 visits → 20 scans; every visitor still redirected)"
TC-BC-01	Barcodes	All 23 formats render and decode with the app's scanner	—	npm run test:barcodes	format examples	Decoded value matches	166 assertions passed	Pass	test:barcodes (npm test): 166 passed, 0 failed
TC-BC-02	Barcodes	Wrong EAN-13 check digit rejected	Signed in	POST /api/products with bad EAN-13	EAN-13 with wrong check digit	400	400	Pass	test:api: "wrong EAN-13 check digit rejected (400)"
TC-BC-03	Barcodes	Internal GTIN outside restricted range rejected	Signed in	POST /api/products source=internal	EAN-13 starting 89…	400	400	Pass	test:api: "'internal' barcode outside restricted range rejected (400)"
TC-BC-04	Barcodes	Internal EAN-13 generated in restricted range	Signed in	GET /api/products/next-code?format=ean13	—	Starts with 2	As expected	Pass	test:api: "internal EAN-13 is generated in restricted range (starts with 2)"
TC-SCN-01	Scanner	Camera scan of EAN-13, Code 128, QR and Data Matrix	Chromium with fake webcam	Open /scanner?camera=1 with Y4M video	4 generated codes	Each decoded and classified	All 4 decoded	Pass	test:browser: "camera → ean13 "8901234567890" shown as Product barcode" (+3)

Test case ID	Module	Test scenario	Preconditions	Test steps	Input	Expected result	Actual result	Status	Evidence or reason
TC-SCN-02	Scanner	Image with several codes decoded	Chromium	Upload one image with 4 codes	PNG	'4 codes detected'	As expected	Pass	test:browser: "upload → "4 codes detected""
TC-SCN-03	Scanner	UPI QR shows payment warning	Chromium	Upload UPI QR image	upi://pay…	Warning text, not a payment	As expected	Pass	test:browser: "UPI QR from upload shows payment warning, not a payment"
TC-PRD-01	Products	Create product with barcodes; price in paise	Signed in	POST /api/products	name, SKU, price 149.00, 2 barcodes	201; priceMinor 14900; EAN-13 primary	As expected	Pass	test:api: "create product (201)"; test:api: "price stored in paise"
TC-PRD-02	Products	Duplicate SKU rejected	Product exists	POST same SKU	duplicate SKU	409	409	Pass	test:api: "duplicate SKU rejected (409)"
TC-PRD-03	Products	Duplicate barcode rejected (incl. UPC-A/EAN-13 equivalence)	Product exists	POST same barcode / EAN-13 form of a UPC-A	barcode	409	409	Pass	test:api: "EAN-13 equivalent of an existing UPC-A rejected as duplicate (409)"
TC-PRD-04	Products	Missing name / invalid price rejected	Signed in	POST /api/products	name=''; price='abc'	400	400	Pass	test:api: "missing name rejected (400)"; test:api: "invalid price rejected (400)"
TC-PRD-05	Products	Other user cannot read, edit or move stock	Users A and B	B calls A's product endpoints	—	404	404	Pass	test:api: "user B cannot read A's product (404)" (+edit, stock)
TC-PRD-06	Products	Same SKU allowed for a different user	Users A and B	B creates product with A's SKU	same SKU	201	201	Pass	test:api: "same SKU/barcode allowed for a different user"
TC-PRD-07	Products	Partial edit keeps untouched fields	Product exists	PATCH price & category only	partial body	Other fields unchanged	As expected	Pass	test:api: "partial edit keeps untouched fields and barcodes"
TC-PRD-08	Products	Delete requires archive first	Active product	DELETE then archive then DELETE	—	400, then 200	As expected	Pass	test:api: "deleting an active product is refused (400)"; test:api: "delete archived product"
TC-PRD-09	Products	Barcode lookup is read-only and finds equivalents	Products exist	GET /api/products/lookup?code=…	EAN-13 / UPC-A / SKU / unknown	Found / found:false; stock unchanged	As expected	Pass	test:api: "scanned EAN-13 form finds UPC-A product"; test:api: "lookups did not change stock"
TC-PRD-10	Import	CSV/XLSX import with per-row report	Signed in	POST /api/products/import mode=skip	rows incl. invalid	created/skipped/error statuses	As expected	Pass	test:api: "import report statuses"; test:api: "import summary"
TC-PRD-11	Import	Import update mode logs stock as adjustment	SKU exists	POST import mode=update	changed name, stock	Updated; ledger 'adjust'	As expected	Pass	test:api: "import stock change recorded as adjustment"
TC-INV-01	Inventory	Stock in increases stock	Stock 10	POST stock type=in qty=5	5	15	15	Pass	test:api: "stock in +5 → 15"
TC-INV-02	Inventory	Stock out decreases stock	Stock 15	POST type=out qty=3	3	12	12	Pass	test:api: "stock out −3 → 12"
TC-INV-03	Inventory	Stock out beyond available rejected	Stock 12	POST type=out qty=999	999	400; stock unchanged	400	Pass	test:api: "stock out beyond available rejected (400)"
TC-INV-04	Inventory	Zero quantity rejected; adjust sets value and flags low stock	Product	POST qty=0; POST adjust=2	0; 2	400; stock 2, lowStock true	As expected	Pass	test:api: "zero quantity rejected (400)"; test:api: "adjust (stock-take) sets 2, low-stock flagged"
TC-INV-05	Inventory	History records every movement with balances; archived blocks movement	Movements made	GET history; archive; POST stock	—	Ledger complete; 400 when archived	As expected	Pass	test:api: "activity history records every movement with balances"; test:api: "archived product: stock changes refused (400)"
TC-UPL-01	Uploads	Oversized upload rejected before parsing (Free plan)	Free plan (10 MB)	POST /api/uploads > limit	>10 MB body	413	413	Pass	test:admin: "oversized upload rejected before parsing (413)"
TC-UPL-02	Uploads	Paid-plan upload between 10 MB and plan limit	Pro plan (25 MB)	POST /api/uploads with 5, 9.5 and 12 MB PDFs (audit probe)	valid %PDF- files	201 for all three	201, 201, then 400 'No file received.' for 12 MB	Fail	Audit probe scripts/probe-upload (logs/probe-upload.log); defect D-01

Test case ID	Module	Test scenario	Preconditions	Test steps	Input	Expected result	Actual result	Status	Evidence or reason
TC-UPL-03	Uploads	Uploaded files served with sandboxing CSP	File exists	GET /f/{name}; check CSP	—	CSP contains 'sandbox'	First build: global CSP (fail); final tree: sandbox present	Pass	test:billing: "user uploads (/f) are also served with a sandboxing CSP" — failed in run 1, passed in final run (D-02 fixed in working tree)
TC-UPL-04	Uploads	File whose content does not match its extension	Signed in	Upload an HTML file renamed .pdf	text/html bytes	415	415 'The file content doesn't match its extension.'	Pass	Audit probe (probe-cases.mts) on the final isolated build: HTML renamed .pdf → 415
TC-BIL-01	Billing	Forged webhook signature rejected	Webhook secret set	POST /api/billing/webhook with bad signature	payload + wrong HMAC	400	400	Pass	test:admin: "forged signature rejected (400)"
TC-BIL-02	Billing	Underpaid payment does not upgrade	Payment link	Signed webhook with lower amount	amount 100 vs 29900	Plan unchanged	As expected	Pass	test:admin: "underpaid link does not upgrade"
TC-BIL-03	Billing	Duplicate webhook deliveries applied once	Valid event	Send same event concurrently	duplicate eventId	Plan extended once	As expected	Pass	test:admin: "concurrent duplicate deliveries extend the plan only once (~30 days)"
TC-BIL-04	Billing	Subscription checkout lifecycle against mock Razorpay	Mock server, sealed creds	npm run test:billing	plans, subscriptions, payments, refunds	All assertions	128 passed, 0 failed	Pass	test:billing final run (mock Razorpay; no real payments)
TC-BIL-05	Entitlements	Plan features enforced on server (alias, analytics, export)	Free plan	Use custom alias; open analytics; export analytics	—	403 for each	403 for each	Pass	test:billing: "custom alias blocked on Free (403)"; test:billing: "analytics export disabled by plan → 403"
TC-BIL-06	Billing	Live Razorpay payment in test mode	Real Razorpay test keys	Subscribe and pay with test card/UPI	—	Plan active after webhook	Not run (no Razorpay account in audit)	Not executed	Requires real Razorpay test account and public webhook URL
TC-ADM-01	Admin	All admin APIs deny without admin session	—	Call 13 admin GET endpoints and mutations	no cookie / user cookie	401	401	Pass	test:admin: "all 13 admin GET endpoints → 401 without a session"; test:admin: "a workspace (user) session is rejected by every admin endpoint"
TC-ADM-02	Admin	User token replayed as admin cookie rejected	User session	Send qs_session value as qs_admin	—	401	401	Pass	test:admin: "user token replayed as the admin cookie is rejected"
TC-ADM-03	Admin	Admin cookie flags	Admin login	Inspect Set-Cookie	—	HttpOnly, SameSite=Strict, ≤12 h	As expected	Pass	test:admin: "admin cookie is HttpOnly + SameSite=Strict, 12 h max"
TC-ADM-04	Admin	Super admin forced to enrol MFA	Super admin without MFA	Sign in; call admin API; open console page	—	403 / redirect to /admin/mfa	As expected	Pass	test:admin: "super admin without MFA: admin APIs → 403"
TC-ADM-05	Admin	MFA enrolment stores encrypted secret and hashed recovery codes	Admin	setup → enable with valid TOTP	TOTP	10 recovery codes once; secret encrypted	As expected	Pass	test:admin: "secret stored encrypted (not plaintext)"; test:admin: "recovery codes stored only as hashes"
TC-ADM-06	Admin	Wrong, replayed or forged MFA input rejected	MFA enabled	Submit wrong TOTP, reused TOTP, forged ticket	—	401	401	Pass	test:admin: "replayed TOTP (already used to enable) rejected"; test:admin: "forged ticket rejected"
TC-ADM-07	Admin	Recovery code single use	MFA enabled	Sign in with recovery code twice	code	1st OK, 2nd 401	As expected	Pass	test:admin: "used recovery code can't be reused"
TC-ADM-08	Admin	Admin CLI refuses weak or command-line passwords	—	npm run admin create	short pw; --password arg	Refused	Refused	Pass	test:admin: "CLI refuses a weak administrator password"
TC-ADM-09	Admin	Admin cannot change roles, touch super admin or delete	Admin session	PATCH role; PATCH super admin; DELETE user	—	403	403	Pass	test:admin: "admin cannot change roles (403)" (+2)

Test case ID	Module	Test scenario	Preconditions	Test steps	Input	Expected result	Actual result	Status	Evidence or reason
TC-ADM-10	Admin	Admin changes ordinary user's plan; unknown fields ignored	Admin	PATCH plan	plan=pro, extra fields	200; extras ignored	As expected	Pass	test:admin: "admin can change an ordinary user's plan"
TC-ADM-11	Admin	Super admin promotes/demotes; cannot change own role	Super admin	PATCH role on self and others	—	Self 400; others 200, audited	As expected	Pass	test:admin: "super admin cannot change own role"; test:admin: "super admin can promote (role change audited)"
TC-ADM-12	Admin	Bulk action skips own account	Admin	POST /api/admin/users/bulk incl. self	ids	Self skipped	As expected	Pass	test:admin: "bulk action skips the actor's own account"
TC-ADM-13	Admin	Admin triggers reset e-mail without seeing token	Admin	POST action=send_password_reset	—	200; no token in response	As expected	Pass	test:admin: "admin can trigger a reset e-mail (never sees the token)"
TC-ADM-14	Admin	User search returns no secrets; pagination	Admin	GET /api/admin/users?q=	query	No hashes/secrets	As expected	Pass	test:admin: "user search works and returns no secrets"
TC-ADM-15	Settings	Settings validation rejects invalid e-mail/colour	Admin	PUT /api/admin/settings	bad e-mail, bad colour	400	400	Pass	test:admin: "invalid support e-mail rejected"; test:admin: "invalid brand colour rejected"
TC-ADM-16	Settings	Stored XSS via brand name / announcement rejected	Admin	PUT with markup	<script>…	400; JSON-LD safe	As expected	Pass	test:admin: "markup in the brand name rejected"; test:admin: "site name can't break out of the JSON-LD script"
TC-ADM-17	Branding	Branding publish, assets and SVG sandboxing	Super admin	Upload assets, publish, fetch /brand/{name}	SVG/PNG	Published; SVG sandboxed + nosniff	First build failed sandbox check; final run passed	Pass	test:billing: "SVG asset served sandboxed with nosniff" (final run)
TC-ADM-18	Audit	Audit log records admin actions without secrets	Actions performed	Inspect AuditLog rows	—	Entries present; no passwords/codes	As expected	Pass	test:admin: "audit records contain no passwords, codes or secrets"
TC-ADM-19	Admin UI	Admin sign-in through real form and console pages render	Chromium; admin created by CLI	Sign in at /admin/login; visit each admin console page at 3 widths	—	All pages render expected headings, fit screen	445 browser assertions passed; 6 heading checks failed (stale expectations 'Paid accounts', 'Payment records'); 1 overflow at 768 px on /admin/settings/pricing	Fail	test:browser final run: 445 passed, 7 failed; defects D-03, D-04
TC-SYS-01	Platform	Health endpoint reveals only status	Server running	GET /api/health	—	{status:'ok'} only	As expected	Pass	test:admin: "health returns only {status:'ok'}"
TC-SYS-02	Platform	Security headers present; no X-Powered-By	Server running	Inspect response headers	—	CSP, nosniff, frame, referrer; no X-Powered-By	As expected	Pass	test:admin: "CSP set (object-src none, frame-ancestors, only Razorpay Checkout as remote script)"; test:admin: "no X-Powered-By"
TC-SYS-03	Platform	No secrets in client bundle	Production build	Scan .next-audit/static files	208 files	No secret values or names	As expected	Pass	test:admin: "no server secrets or secret env names in 208 client files"
TC-SYS-04	Platform	Unsafe production configuration refused	—	Unit tests of checkEnv()	short secret, http URL, DEMO_MODE…	Errors reported	As expected	Pass	test:security (npm test): 134 passed, 0 failed
TC-SYS-05	Database	MySQL migrations and SQLite→MySQL data migration	MySQL 8.4 container	npm run test:mysql	edge-case SQLite source	Exact round-trip; failures write nothing	32 passed, 0 failed	Pass	test:mysql run against mysql:8.4 (Docker), 74 s
TC-SYS-06	Database	MySQL schema in sync with canonical schema	—	npm run db:mysql:schema -- --check	—	Up to date	Up to date	Pass	unit-db_mysql_schema log
TC-SYS-07	Responsive	Public and workspace pages fit 375/768/1280 px without CSP or console errors	Chromium	test:browser responsive section	24 public and workspace pages × 3 widths	No overflow / CSP / console errors; accessible names	All public & workspace checks passed	Pass	test:browser final run

Test case ID	Module	Test scenario	Preconditions	Test steps	Input	Expected result	Actual result	Status	Evidence or reason
TC-SYS-08	Performance	Redirect latency and page load under load	Production-like server	Load test /r/{code} (e.g. k6, 50 VUs, 5 min)	—	p95 < 200 ms (target)	Not run	Not executed	No load-testing tool or environment in the audit
TC-SYS-09	Compatibility	Firefox and Safari	Browsers installed	Manual walkthrough of main flows	—	Same behaviour as Chromium	Not run	Not executed	Only Chromium automated
TC-SYS-10	Database	Database unavailable	Stop DB	GET /api/health	—	503 'degraded'	Not run	Not executed	checkDatabase() in src/lib/system-status.ts

Appendix E: Installation and Execution Guide

Prerequisites. Node.js ≥ 22.5 and npm ≥ 10; Git; a Chromium-based browser for the browser tests; optionally MySQL 8 (or Docker) and SMTP, Google and Razorpay accounts.

Development set-up.

```
git clone https://github.com/druvemedia/qrsathi.git
cd qrsathi
npm install                      # also generates Prisma client and copies the scanner WASM
cp .env.example .env             # then edit: DATABASE_URL, AUTH_SECRET (32+ random chars), NEXT_PUBLIC_APP_URL
npm run db:push                  # create the SQLite database
npm run admin -- create --email you@example.com --super   # password via stdin
npm run dev                      # http://localhost:3000
```

Quality checks.

```
npm run typecheck && npm run lint && npm test
```

Integration tests (isolated, as performed in this audit).

```
# build into a separate folder and use a throw-away database
export NEXT_DIST_DIR=.next-test DATABASE_URL="file:/absolute/path/test.db" \
       NEXT_PUBLIC_APP_URL=http://localhost:3100 AUTH_SECRET=<random 32+ chars>
npx prisma db push --skip-generate
npx next build
NODE_ENV=production npx next start -p 3100 &
BASE_URL=http://localhost:3100 npm run test:api
BASE_URL=http://localhost:3100 npm run test:admin
BASE_URL=http://localhost:3100 npm run test:browser
# billing: restart the server with RAZORPAY_API_BASE=http://127.0.0.1:3999 and SETTINGS_ENCRYPTION_KEY set,
# without RAZORPAY_* keys, then: BASE_URL=http://localhost:3100 npm run test:billing
MYSQL_TEST_URL="mysql://root:<pw>@127.0.0.1:3306/" npm run test:mysql
```

Never point the integration tests at a real database: they create and delete accounts.

Production (summary). `npm ci` → set production variables → `npm run db:mysql:deploy` → `npm run build` → `npm start` behind an HTTPS reverse proxy → create the super administrator and enrol MFA. See Section 6.20 and `docs/DEPLOYMENT.md`.

Appendix F: Configuration Reference

The complete list of environment variables with purposes is Table 3.6. Additional configuration lives in:

Table F.1: Configuration files

File	Purpose
`next.config.mjs`	Security headers (CSP etc.), separate build directory, server-external packages, admin no-store/noindex headers, sandbox CSP for `/f` and `/brand`
`src/middleware.ts`	CSRF origin rule; matcher `/api/:path*`, `/admin`, `/admin/:path*`
`src/lib/env-check.ts`	Start-up validation rules
`prisma/schema.prisma`	Canonical database schema (SQLite)
`prisma/mysql/schema.prisma`	Generated MySQL schema (do not edit by hand)
`src/lib/plans.ts`	Default plan catalogue (seeded only into an empty table) and limit schema
`tailwind.config.ts`, `postcss.config.mjs`	Styling
`tsconfig.json`, `eslint.config.mjs`	TypeScript and lint rules
`.github/workflows/ci.yml`	CI pipeline (quality job and MySQL job)
`.github/dependabot.yml`	Dependency update automation
`security/audit-allowlist.json`	Accepted dependency advisories with expiry
Admin console → Settings	Runtime settings stored in the `Setting` table (announcement, sign-ups, branding, sealed payment credentials)

Appendix G: Screenshot Index

All screenshots were captured by `scripts/test-browser.mts` (with `SHOT_DIR`) from the production build during the final audit run (2026-10-25, 18:41–18:47 IST) using throw-away test accounts. They are stored unmodified in `screenshots/desktop-1280/` and `screenshots/mobile-375/`. Screenshots of 768 px widths were captured but not included.

Table G.1: Screenshot index

#	Desktop file	Mobile file	Route	Used in report
1	desktop-1280/about.png	mobile-375/about.png	/about	—
2	desktop-1280/admin.png	mobile-375/admin.png	/admin	Figure 6.13
3	desktop-1280/admin_analytics.png	mobile-375/admin_analytics.png	/admin/analytics	—
4	desktop-1280/admin_barcodes.png	mobile-375/admin_barcodes.png	/admin/barcodes	—
5	desktop-1280/admin_forgot-password.png	mobile-375/admin_forgot-password.png	/admin/forgot-password	—
6	desktop-1280/admin_login.png	mobile-375/admin_login.png	/admin/login	—
7	desktop-1280/admin_mfa.png	mobile-375/admin_mfa.png	/admin/mfa	—
8	desktop-1280/admin_payments.png	mobile-375/admin_payments.png	/admin/payments	—
9	desktop-1280/admin_products.png	mobile-375/admin_products.png	/admin/products	—
10	desktop-1280/admin_qr-codes.png	mobile-375/admin_qr-codes.png	/admin/qr-codes	—
11	desktop-1280/admin_reports.png	mobile-375/admin_reports.png	/admin/reports	—
12	desktop-1280/admin_security.png	mobile-375/admin_security.png	/admin/security	Figure 6.18
13	desktop-1280/admin_security_admins.png	mobile-375/admin_security_admins.png	/admin/security/admins	—
14	desktop-1280/admin_security_events.png	mobile-375/admin_security_events.png	/admin/security/events	—
15	desktop-1280/admin_settings.png	mobile-375/admin_settings.png	/admin/settings	—
16	desktop-1280/admin_settings_appearance.png	mobile-375/admin_settings_appearance.png	/admin/settings/appearance	—
17	desktop-1280/admin_settings_branding.png	mobile-375/admin_settings_branding.png	/admin/settings/branding	Figure 6.15
18	desktop-1280/admin_settings_email.png	mobile-375/admin_settings_email.png	/admin/settings/email	—
19	desktop-1280/admin_settings_payments.png	mobile-375/admin_settings_payments.png	/admin/settings/payments	Figure 6.17
20	desktop-1280/admin_settings_pricing.png	mobile-375/admin_settings_pricing.png	/admin/settings/pricing	Figure 6.16
21	desktop-1280/admin_subscriptions.png	mobile-375/admin_subscriptions.png	/admin/subscriptions	—
22	desktop-1280/admin_system.png	mobile-375/admin_system.png	/admin/system	—
23	desktop-1280/admin_users.png	mobile-375/admin_users.png	/admin/users	Figure 6.14
24	desktop-1280/analytics.png	mobile-375/analytics.png	/analytics	—
25	desktop-1280/barcodes.png	mobile-375/barcodes.png	/barcodes	Figure 6.4
26	desktop-1280/bulk.png	mobile-375/bulk.png	/bulk	Figure 6.10
27	desktop-1280/codes.png	mobile-375/codes.png	/codes	—
28	desktop-1280/contact.png	mobile-375/contact.png	/contact	—
29	desktop-1280/dashboard.png	mobile-375/dashboard.png	/dashboard	Figure 6.6
30	desktop-1280/dashboard_settings.png	mobile-375/dashboard_settings.png	/dashboard/settings	—
31	desktop-1280/dynamic.png	mobile-375/dynamic.png	/dynamic	—
32	desktop-1280/features.png	mobile-375/features.png	/features	—
33	desktop-1280/forgot-password.png	mobile-375/forgot-password.png	/forgot-password	—
34	desktop-1280/generator.png	mobile-375/generator.png	/generator	Figure 6.3
35	desktop-1280/help.png	mobile-375/help.png	/help	—
36	desktop-1280/home.png	mobile-375/home.png	/	Figure 6.1
37	desktop-1280/inventory.png	mobile-375/inventory.png	/inventory	Figure 6.9
38	desktop-1280/login.png	mobile-375/login.png	/login	Figure 6.2

#	Desktop file	Mobile file	Route	Used in report
39	desktop-1280/pricing.png	mobile-375/pricing.png	/pricing	Figure 6.12
40	desktop-1280/products.png	mobile-375/products.png	/products	Figure 6.7
41	desktop-1280/products_new.png	mobile-375/products_new.png	/products/new	Figure 6.8
42	desktop-1280/scan-history.png	mobile-375/scan-history.png	/scan-history	—
43	desktop-1280/scanner.png	mobile-375/scanner.png	/scanner	Figure 6.5
44	desktop-1280/signup.png	mobile-375/signup.png	/signup	—
45	desktop-1280/studio.png	mobile-375/studio.png	/studio	—
46	desktop-1280/tools.png	mobile-375/tools.png	/tools	—
47	desktop-1280/upi.png	mobile-375/upi.png	/upi	Figure 6.11

Appendix H: Requirement Traceability Matrix

Table H.1: Requirement traceability matrix

Requirement ID	Requirement description	Module	Source file(s)	Test case(s)	Verification status	Notes
FR-01	User registration with validated input; role always 'user'	Accounts	src/app/api/auth/signup/route.ts	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04	Verified	Duplicate and weak-password cases by audit probe
FR-02	Sign-in (password, Google) and sign-out with revocable sessions	Accounts	src/lib/auth-flow.ts; src/lib/session.ts; src/app/api/auth/login|logout|google	TC-AUTH-05..10, TC-AUTH-16, TC-AUTH-17	Verified	Google flow not executed (TC-AUTH-17)
FR-03	Password reset by one-hour single-use token	Accounts	src/lib/password-reset.ts; src/app/api/auth/forgot|reset	TC-AUTH-11..14	Verified	SMTP delivery not exercised
FR-04	Account management (name, password, other sessions, delete)	Accounts	src/app/api/account/route.ts; account/sessions	TC-AUTH-15	Verified	—
FR-05	Static QR generation for 27 types with design and export	QR	src/lib/qr-types.ts; qr-render.ts; qr-export.ts	TC-QR-01..03	Verified	—
FR-06	Save and manage QR codes within plan quota	QR	src/lib/qr-service.ts; src/app/api/qrcodes/**	TC-QR-04, TC-QR-05, TC-QR-06	Verified	—
FR-07	Dynamic QR codes and hosted pages	Dynamic QR	src/app/r/[code]/route.ts; src/app/p/[code]/page.tsx	TC-DYN-01..03	Verified	Hosted page content not asserted
FR-08	Privacy-preserving scan analytics and export	Analytics	src/lib/scans.ts; src/lib/analytics.ts; api/analytics/**	TC-DYN-04, TC-BIL-05	Partially verified	Charts not asserted
FR-09	Barcode generation (23 formats) and labels	Barcodes	src/lib/barcodes.ts; src/components/barcode/*	TC-BC-01..04	Verified	—
FR-10	Camera/image scanner with safety notes and history	Scanner	src/app/(app)/scanner/scanner.tsx; src/lib/zxing-client.ts; content-safety.ts	TC-SCN-01..03	Verified	Chromium only
FR-11	Product management with per-user unique SKU/barcodes	Products	src/lib/products.ts; src/app/api/products/**	TC-PRD-01..09	Verified	—
FR-12	Stock in/out/adjust with ledger and low-stock flag	Inventory	moveStock() in src/lib/products.ts; api/products/[id]/stock; api/inventory	TC-INV-01..05	Verified	—
FR-13	Bulk generation and CSV/XLSX import	Bulk/Import	src/app/(app)/bulk/*; src/app/api/products/import/route.ts	TC-PRD-10, TC-PRD-11	Partially verified	Bulk generation UI not functionally asserted
FR-14	File uploads for File QR codes	Uploads	src/app/api/uploads/route.ts; src/lib/storage.ts; src/app/f/[name]/route.ts	TC-UPL-01..04	Partially verified	Defect D-01 (>10 MB); D-02 fixed in working tree
FR-15	Server-enforced plan entitlements	Plans	src/lib/entitlements.ts; plan-store.ts; plans.ts	TC-QR-05, TC-BIL-05	Verified	—
FR-16	Razorpay subscription billing	Billing	src/lib/billing.ts; razorpay.ts; src/app/api/billing/**	TC-BIL-01..04, TC-BIL-06	Partially verified	Mock provider only; live Razorpay not executed
FR-17	Administrator sign-in with TOTP MFA	Admin auth	src/app/api/admin/auth/**; api/admin/mfa; src/lib/mfa.ts	TC-ADM-01..08	Verified	—
FR-18	User administration with RBAC rules	Admin	src/app/api/admin/users/**; src/lib/admin-users.ts	TC-ADM-09..14	Verified	D-05 plan enum
FR-19	QR moderation and abuse reports	Moderation	src/app/api/admin/qrcodes/**; admin/reports/**; api/reports	—	Implemented, not verified	No assertions in suites
FR-20	Site configuration: settings, branding, pricing, payment gateway	Configuration	api/admin/settings|branding|plans|payments/config; src/lib/branding.ts; payment-config.ts	TC-ADM-15..17, TC-BIL-04	Verified	D-03 layout at 768 px
FR-21	Audit log, system log and health	Audit	src/lib/audit.ts; log.ts; system-status.ts; api/health	TC-ADM-18, TC-SYS-01	Verified	—
FR-22	UPI payment QR with validation and limits	UPI	src/lib/upi.ts; src/app/(app)/upi/*	TC-QR-07, TC-QR-08	Verified	—
NFR-01	Authenticated access to private data; admin-only admin APIs	Security	src/lib/session.ts; guard.ts	TC-AUTH-05, TC-ADM-01, TC-ADM-02	Verified	—
NFR-02	CSRF protection	Security	src/middleware.ts	TC-AUTH-06	Verified	—
NFR-03	Secure storage of passwords, tokens and secrets	Security	auth-helpers.ts; mfa.ts; secret-box.ts; password-reset.ts	TC-AUTH-12, TC-ADM-05, TC-SYS-03	Verified	—
NFR-04	Security headers incl. sandbox CSP for files	Security	next.config.mjs; app/f, app/brand	TC-SYS-02, TC-UPL-03, TC-ADM-17	Verified	After D-02 fix in working tree
NFR-05	Privacy: no raw IPs; DNT/GPC	Privacy	src/lib/scans.ts; audit.ts	TC-DYN-04	Partially verified	DNT path not asserted

Requirement ID	Requirement description	Module	Source file(s)	Test case(s)	Verification status	Notes
NFR-06	Atomic/idempotent quotas, stock and payments	Reliability	entitlements.ts; products.ts; billing.ts	TC-QR-05, TC-INV-03, TC-BIL-03	Verified	—
NFR-07	Fail-closed production configuration	Reliability	src/lib/env-check.ts; src/instrumentation.ts	TC-SYS-04	Verified	—
NFR-08	Responsive layout at 375/768/1280 px	Usability	UI components	TC-SYS-07, TC-ADM-19	Partially verified	D-03 on one admin page
NFR-09	Accessible names for controls	Accessibility	UI components	TC-SYS-07	Verified (automated)	No manual WCAG audit
NFR-10	Type-checked, linted, CI	Maintainability	tsconfig.json; eslint.config.mjs; .github/workflows/ci.yml	Commands 1–2 (Table 5.2)	Verified	5 lint warnings
NFR-11	SQLite and MySQL portability	Portability	prisma/**; scripts/db/*	TC-SYS-05, TC-SYS-06	Verified	—
NFR-12	Performance targets	Performance	—	TC-SYS-08	Not verified	Not measured
NFR-13	Multi-instance scalability	Scalability	rate-limit.ts (database store)	—	Not verified	Not load-tested

Appendix I: Glossary

Table I.1: Glossary of terms

Term	Meaning
Alias	A user-chosen short code for a dynamic QR (paid plans)
App Router	The Next.js routing system based on the `src/app` directory
Check digit	A digit computed from the others to detect misreads (GS1 modulo-10)
Dynamic QR	A QR code that encodes a short link whose destination can be changed
Entitlement	A plan limit or feature checked by the server
GTIN	Global Trade Item Number (8, 12, 13 or 14 digits)
Hosted page	A mobile landing page served by QRSathi at `/p/{code}`
Idempotent	Producing the same result when repeated (e.g. a webhook delivered twice)
Ledger	Append-only list of stock movements with resulting balances
Lookup key	Normalised form of a barcode used to match scans to products
Middleware	Code that runs before route handlers for matching paths
Prisma	Type-safe ORM used for database access and migrations
Quiet zone	Blank margin around a code needed by scanners
Restricted circulation	GS1 prefix range 20–29 reserved for internal numbering
Route handler	A Next.js server function handling an HTTP method for a path
Sealed secret	A credential encrypted with AES-256-GCM using a key held only in the environment
Session version	Counter on the user that invalidates all tokens when incremented
Short code	The random identifier in a dynamic link `/r/{code}`
Snapshot	A copy of data (e.g. plan price) stored with a record so history does not change
Super administrator	Highest role; manages roles, pricing and payment credentials
TOTP	Six-digit code that changes every 30 seconds (RFC 6238)
UPI	India's real-time payment system; QR codes use `upi://pay` links
UTM parameters	Campaign tags appended to destination URLs
Webhook	An HTTP callback from a provider (Razorpay) to the application
zod	Schema library used to validate input on client and server